

(10) **Patent No.:** US 12,400,015 B2
(45) **Date of Patent:** Aug. 26, 2025

(58) **Field of Classification Search**

CPC G06F 21/6218; G06F 9/45558; G06F
16/172; G06F 16/192; G06F 2009/45579;
G06F 2009/45595; G06F 2221/2141
(Continued)

(71) Applicant: **Nutanix, Inc.**, San Jose, CA (US)

(72) Inventors: **Saji Kumar Vijaya Kumari**
Rajendran Nair, Cupertino, CA (US);
Hemanth Kumar Thummala, San
Jose, CA (US); **Veerraju Tammineedi**,
San Jose, CA (US); **Shyamsunder**
Prayagchand Rathi, Sunnyvale, CA
(US); **Manoj Naik**, San Jose, CA (US);
Manish Gupta, Bangalore (IN); **Durga**
Manesh Arikatla, San Jose, CA (US);
Gaurav Kumar, Milpitas, CA (US)

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,095,421 A 3/1992 Freund
5,276,867 A 1/1994 Kenley et al.
(Continued)

FOREIGN PATENT DOCUMENTS

CN	103746997	A	4/2014
CN	105100210	A	11/2015

(Continued)

OTHER PUBLICATIONS

US 11,048,595 B2, 06/2021, Venkatesh et al. (withdrawn)
(Continued)

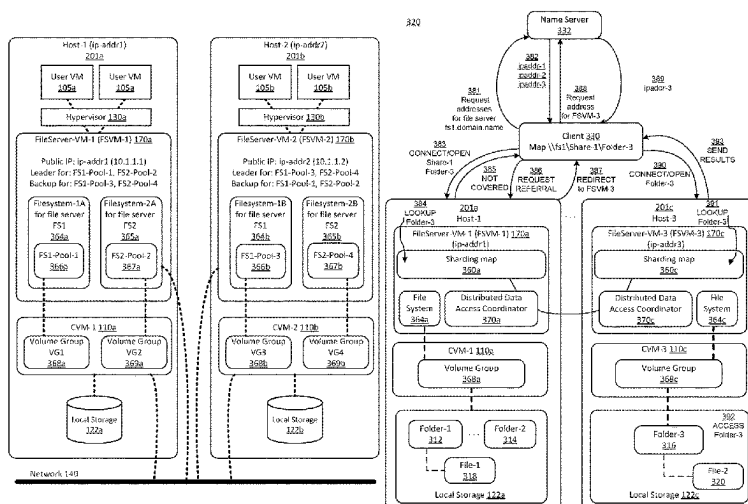
Primary Examiner — Michael Pham

(74) *Attorney, Agent, or Firm* — Stoel Rives LLP

(57) **ABSTRACT**

Examples of systems described herein include a file server virtual machine of a virtualized file server configured to manage storage of a plurality of storage items. The file server virtual machine including a file system configured to receive an access request directed to a storage item of the plurality of storage items and associated with a user. The file system is further configured to retrieve an access control list having permissions information associated with the storage item, and to cache a permissions profile for the user including all permissions pertaining to the user for the storage item. The file system is further configured to determine whether the access request is permissible based on the cached permissions profile.

32 Claims, 21 Drawing Sheets



Related U.S. Application Data					
(60)	Provisional application No. 62/430,264, filed on Dec. 5, 2016, provisional application No. 62/429,283, filed on Dec. 2, 2016.			8,447,728 B2	5/2013 Prahlad et al.
				8,473,462 B1	6/2013 Banerjee
(51)	Int. Cl. G06F 16/188 (2019.01) G06F 21/62 (2013.01)			8,484,163 B1	7/2013 Yucel et al.
				8,484,356 B1	7/2013 Dougliis et al.
(52)	U.S. Cl. CPC .. G06F 16/192 (2019.01); G06F 2009/45579 (2013.01); G06F 2009/45595 (2013.01); G06F 2221/2141 (2013.01)			8,539,076 B2	9/2013 Nakano et al.
				8,543,790 B2	9/2013 Chen et al.
(58)	Field of Classification Search USPC 707/785 See application file for complete search history.			8,549,518 B1	10/2013 Aron et al.
				8,595,595 B1	11/2013 Grcanac et al.
(56)	References Cited U.S. PATENT DOCUMENTS			8,601,473 B1	12/2013 Aron et al.
				8,620,870 B2	12/2013 Dwarampudi et al.
				8,635,351 B2	1/2014 Astete et al.
				8,646,089 B2	2/2014 Jayanthi et al.
				8,688,660 B1	4/2014 Sivasubramanian et al.
				8,725,679 B2	5/2014 Nair et al.
				8,751,515 B1	6/2014 Xing et al.
				8,762,335 B2	6/2014 Prahlad et al.
				8,805,951 B1	8/2014 Faibish et al.
				8,812,566 B2	8/2014 Aizman et al.
				8,838,923 B2	9/2014 Prahlad et al.
				8,850,130 B1	9/2014 Aron et al.
				8,863,124 B1	10/2014 Aron
				8,914,429 B2	12/2014 Pitts
				8,935,563 B1	1/2015 Rajaa et al.
				8,943,203 B1	1/2015 Lent et al.
				8,949,557 B2	2/2015 Kamei et al.
				8,966,188 B1	2/2015 Bardale
				8,983,952 B1	3/2015 Zhang et al.
				8,996,783 B2	3/2015 Huang et al.
				9,009,106 B1	4/2015 Aron et al.
				9,043,567 B1	5/2015 Modukuri et al.
				9,060,014 B2	6/2015 Crowley
				9,069,708 B2	6/2015 Gill et al.
				9,152,628 B1	10/2015 Stacey et al.
				9,154,535 B1	10/2015 Harris
				9,165,003 B1	10/2015 Tummala et al.
				9,171,019 B1	10/2015 Donlan
				9,201,698 B2	12/2015 Ashok et al.
				9,201,704 B2	12/2015 Chang et al.
				9,201,887 B1	12/2015 Earl et al.
				9,213,513 B2	12/2015 Hartz et al.
				9,229,850 B1	1/2016 Wang et al.
				9,244,674 B2	1/2016 Waterman et al.
				9,244,969 B1	1/2016 Love et al.
				9,250,823 B1	2/2016 Kamat et al.
				9,256,475 B1	2/2016 Aron et al.
				9,256,612 B1	2/2016 Bhatt et al.
				9,268,586 B2	2/2016 Voccio et al.
				9,268,947 B1	2/2016 Jarlstrom et al.
				9,274,817 B1	3/2016 Fan et al.
				9,286,298 B1	3/2016 Gillett
				9,292,327 B1	3/2016 Von Thenen et al.
				9,336,132 B1	5/2016 Aron et al.
				9,348,702 B2	5/2016 Hsu et al.
				9,405,482 B2	8/2016 Varadharajan et al.
				9,405,566 B2	8/2016 Chawla et al.
				9,411,628 B2	8/2016 Bezbaruah et al.
				9,430,646 B1	8/2016 Mushtaq et al.
				9,448,887 B1	9/2016 Dayan et al.
				9,465,953 B2	10/2016 Sharma et al.
				9,497,257 B1	11/2016 Love et al.
				9,513,946 B2	12/2016 Sevigny et al.
				9,519,596 B2	12/2016 Coppola et al.
				9,535,907 B1	1/2017 Stringham
				9,563,555 B2	2/2017 Flynn et al.
				9,571,561 B2	2/2017 Jang
				9,590,843 B2	3/2017 Cui et al.
				9,600,307 B1	3/2017 Pulkayath et al.
				9,602,341 B1	3/2017 Degwekar et al.
				9,613,064 B1	4/2017 Chou et al.
				9,619,257 B1	4/2017 Aron et al.
				9,632,892 B1	4/2017 Sledz et al.
				9,634,990 B2	4/2017 Lee
				9,639,428 B1	5/2017 Boda et al.
				9,652,265 B1	5/2017 Narayanasamy et al.
				9,658,899 B2	5/2017 Jenkins
				9,690,670 B1	6/2017 Paulzagade et al.
				9,733,958 B2	8/2017 Cui et al.
				9,740,436 B2	8/2017 Fiebrich-Kandler et al.
				9,740,472 B1	8/2017 Sohi et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

9,740,723 B2	8/2017	Prahlad et al.	11,106,442 B1	8/2021	Gupta et al.
9,747,287 B1	8/2017	Bhardwaj et al.	11,106,447 B2	8/2021	Gupta et al.
9,760,447 B2	9/2017	Durge et al.	11,144,407 B1	10/2021	Shankar et al.
9,772,866 B1	9/2017	Aron et al.	11,194,680 B2	12/2021	Konka et al.
9,798,486 B1	10/2017	Singh	11,204,710 B2	12/2021	Varadharajan et al.
9,832,136 B1	11/2017	Gibson	11,218,418 B2	1/2022	Gupta et al.
9,846,706 B1	12/2017	Basov et al.	11,281,484 B2	3/2022	Bafna et al.
9,853,978 B2	12/2017	Tellvik et al.	11,288,239 B2	3/2022	Bafna
9,870,291 B2	1/2018	Bezbaruah et al.	11,294,777 B2	4/2022	Venkatesh et al.
9,893,988 B2	2/2018	Agarwal et al.	11,310,286 B2	4/2022	Cui et al.
9,922,201 B2 *	3/2018	von Muhlen H04L 63/104	11,474,796 B1	10/2022	Mather
9,940,154 B2	4/2018	Ramani et al.	11,489,839 B2	11/2022	Sankavaram et al.
9,940,460 B1	4/2018	Derbeko et al.	11,537,384 B2	12/2022	Sharpe et al.
9,946,573 B2	4/2018	McDermott	11,544,049 B2	1/2023	Gopalapura Venkatesh et al.
9,961,145 B1	5/2018	Faibish et al.	11,546,337 B2	1/2023	Le et al.
9,965,357 B1	5/2018	Earl et al.	11,550,557 B2	1/2023	Sharpe et al.
10,009,215 B1	6/2018	Shorey	11,550,558 B2	1/2023	Bafna et al.
10,019,159 B2	7/2018	Wires et al.	11,550,559 B2	1/2023	Bafna et al.
10,050,862 B2	8/2018	Nambiar et al.	11,562,034 B2	1/2023	Arikatla et al.
10,069,909 B1	9/2018	Chopra et al.	11,563,800 B1	1/2023	Trendafilov et al.
10,078,583 B1	9/2018	Wallace	11,568,073 B2	1/2023	Nair et al.
10,083,022 B2	9/2018	Fukui et al.	11,579,861 B2	2/2023	Sharpe et al.
10,084,873 B2	9/2018	Dornemann	11,609,826 B2	3/2023	Alonzo et al.
10,095,506 B2	10/2018	Venkatesh et al.	11,645,065 B2	5/2023	Gupta et al.
10,101,989 B2	10/2018	Sinha et al.	11,669,320 B2	6/2023	Gopalapura Venkatesh et al.
10,108,502 B1	10/2018	Gopinath et al.	11,675,746 B2	6/2023	Thummala et al.
10,114,706 B1	10/2018	Chougala et al.	11,775,397 B2	10/2023	Venkatesh et al.
10,127,059 B2	11/2018	Astete et al.	11,922,157 B2	3/2024	Sharpe et al.
10,140,115 B2	11/2018	Fukui et al.	11,922,203 B2	3/2024	Bafna et al.
10,152,233 B2	12/2018	Xu et al.	11,947,952 B2	4/2024	Gopalapura Venkatesh et al.
10,210,048 B2	2/2019	Sancheti	11,954,078 B2	4/2024	Bafna et al.
10,248,657 B2	4/2019	Prahlad et al.	11,966,729 B2	4/2024	Sharpe et al.
10,262,023 B1	4/2019	Kuang et al.	11,966,730 B2	4/2024	Sharpe et al.
10,311,153 B2	6/2019	Mason, Jr. et al.	12,014,166 B2	6/2024	Gupta et al.
10,333,901 B1	6/2019	Bauman et al.	12,131,192 B2	10/2024	Venkatesh et al.
10,362,092 B1	7/2019	Parthasarathy	12,135,963 B2	11/2024	Sharpe et al.
10,367,753 B2	7/2019	Schultze et al.	12,153,913 B2	11/2024	Bafna et al.
10,379,759 B2	8/2019	Bhardwaj et al.	12,217,039 B2	2/2025	Gopalapura Venkatesh et al.
10,382,963 B2	8/2019	Jassal et al.	2001/0047400 A1	11/2001	Coates et al.
10,394,412 B2	8/2019	Torman et al.	2002/0069196 A1	6/2002	Betros et al.
10,394,547 B2	8/2019	Fukui et al.	2002/0120763 A1	8/2002	Miloushev et al.
10,409,653 B2	9/2019	von Muhlen et al.	2003/0014442 A1	1/2003	Shiigi et al.
10,419,426 B2	9/2019	Bakshan et al.	2003/0033308 A1	2/2003	Patel et al.
10,452,456 B2	10/2019	von Muhlen et al.	2003/0115218 A1	6/2003	Bobbitt et al.
10,496,321 B2	12/2019	Varadharajan et al.	2003/0163597 A1	8/2003	Hellman et al.
10,523,592 B2	12/2019	Byers et al.	2003/0195942 A1	10/2003	Muhlestein et al.
10,530,742 B2	1/2020	Shah et al.	2004/0054777 A1	3/2004	Ackaouy et al.
10,540,164 B2	1/2020	Bafna et al.	2004/0111608 A1	6/2004	Oom et al.
10,540,165 B2	1/2020	Bafna et al.	2004/0199734 A1	10/2004	Rajamani et al.
10,540,166 B2	1/2020	Arikatla et al.	2004/0210591 A1	10/2004	Hirschfeld et al.
10,542,049 B2	1/2020	Cui et al.	2004/0225742 A1	11/2004	Loaiza et al.
10,552,266 B1	2/2020	Kogtev et al.	2004/0267832 A1	12/2004	Wong et al.
10,572,317 B2	2/2020	von Muhlen et al.	2005/0120160 A1	6/2005	Plouffe et al.
10,579,443 B2	3/2020	von Muhlen et al.	2005/0120180 A1	6/2005	Schornbach et al.
10,594,730 B1	3/2020	Summers et al.	2005/0125503 A1	6/2005	Iyengar et al.
10,635,544 B1	4/2020	Kitson	2005/0193221 A1	9/2005	Yoneyama
10,642,507 B2	5/2020	Gupta et al.	2005/0193245 A1	9/2005	Hayden et al.
10,642,800 B2	5/2020	Gummaraju et al.	2005/0226059 A1	10/2005	Kavuri et al.
10,678,651 B1	6/2020	Borodin et al.	2005/0228798 A1	10/2005	Shepard et al.
10,705,889 B2	7/2020	von Muhlen et al.	2006/0010227 A1	1/2006	Atluri
10,719,305 B2	7/2020	Sinha et al.	2006/0047685 A1	3/2006	Dearing et al.
10,719,306 B2	7/2020	Deshmukh et al.	2006/0080445 A1	4/2006	Chang et al.
10,719,307 B2	7/2020	Kanada et al.	2006/0112222 A1	5/2006	Barrall
10,728,090 B2	7/2020	Deshmukh et al.	2006/0167921 A1	7/2006	Grebus et al.
10,789,138 B2	9/2020	Wang et al.	2006/0206901 A1	9/2006	Chan
10,809,998 B2	10/2020	Venkatesh et al.	2006/0224918 A1	10/2006	Koike
10,824,455 B2	11/2020	Arikatla et al.	2006/0225065 A1	10/2006	Chandhok et al.
10,831,465 B2	11/2020	Sharpe et al.	2006/0253856 A1	11/2006	Hu et al.
10,838,708 B2	11/2020	Sinha et al.	2006/0271510 A1	11/2006	Harward et al.
10,949,192 B2	3/2021	Venkatesh et al.	2006/0271931 A1	11/2006	Harris et al.
10,949,387 B1	3/2021	Grider	2007/0022129 A1	1/2007	Bahar et al.
11,010,011 B2	5/2021	Varadharajan et al.	2007/0038913 A1	2/2007	Allen et al.
11,025,626 B1	6/2021	Todd et al.	2007/0041556 A1	2/2007	Rana et al.
11,074,138 B2	7/2021	Alonzo et al.	2007/0055703 A1	3/2007	Zimran et al.
11,086,826 B2	8/2021	Thummala et al.	2007/0100905 A1	5/2007	Masters et al.
			2007/0171921 A1	7/2007	Wookey et al.
			2007/0179995 A1	8/2007	Prahlad et al.
			2007/0300220 A1	12/2007	Seliger et al.
			2008/0040483 A1	2/2008	Nakatani et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

2008/0059474 A1	3/2008	Lim	2011/0307729 A1	12/2011	Matsuzawa et al.
2008/0071804 A1	3/2008	Gunda	2011/0320690 A1	12/2011	Petersen et al.
2008/0071997 A1	3/2008	Loaiza et al.	2012/0017114 A1	1/2012	Timashev et al.
2008/0098194 A1	4/2008	Hashimoto et al.	2012/0023495 A1	1/2012	Machida
2008/0104349 A1	5/2008	Maruyama et al.	2012/0030456 A1	2/2012	Wu et al.
2008/0104589 A1	5/2008	Mccrory et al.	2012/0054736 A1	3/2012	Arcese et al.
2008/0133486 A1	6/2008	Fitzgerald et al.	2012/0081395 A1	4/2012	Adi et al.
2008/0134178 A1	6/2008	Fitzgerald et al.	2012/0084381 A1	4/2012	Alladi et al.
2008/0168154 A1	7/2008	Skyrm et al.	2012/0151249 A1	6/2012	Swan et al.
2008/0189468 A1	8/2008	Schmidt et al.	2012/0166866 A1	6/2012	Rao et al.
2008/0195827 A1	8/2008	Saika	2012/0185437 A1	7/2012	Pavlov et al.
2008/0201414 A1	8/2008	Amir et al.	2012/0209983 A1	8/2012	Bronner et al.
2008/0201457 A1	8/2008	London	2012/0210066 A1	8/2012	Joshi et al.
2008/0263113 A1	10/2008	Krishnaiyer et al.	2012/0233463 A1	9/2012	Holt et al.
2008/0270677 A1	10/2008	Kolakowski	2012/0254445 A1	10/2012	Kawamoto et al.
2008/0271017 A1	10/2008	Herington	2012/0254567 A1	10/2012	Umbehocker
2008/0320499 A1	12/2008	Suit	2012/0266162 A1	10/2012	Baron
2008/0320583 A1	12/2008	Sharma et al.	2012/0272237 A1	10/2012	Baron
2009/0006801 A1	1/2009	Shultz et al.	2012/0290630 A1	11/2012	Aizman et al.
2009/0100248 A1	4/2009	Kami	2012/0304247 A1	11/2012	Badger et al.
2009/0150885 A1	6/2009	Safari et al.	2012/0310881 A1	12/2012	Shadmon
2009/0158082 A1	6/2009	Jain et al.	2012/0310892 A1	12/2012	Dam et al.
2009/0171971 A1	7/2009	Goddard et al.	2012/0324183 A1	12/2012	Chiruvolu et al.
2009/0182860 A1	7/2009	Hwang et al.	2012/0331119 A1	12/2012	Bose et al.
2009/0193272 A1	7/2009	Matsuzawa et al.	2013/0046740 A1	2/2013	Li et al.
2009/0216975 A1	8/2009	Halperin et al.	2013/0047160 A1	2/2013	Conover
2009/0248847 A1	10/2009	Sutoh et al.	2013/0054979 A1	2/2013	Basnov et al.
2009/0248870 A1	10/2009	Kamei et al.	2013/0055018 A1	2/2013	Joshi et al.
2009/0249470 A1	10/2009	Litvin et al.	2013/0061110 A1	3/2013	Zvibel
2009/0254572 A1	10/2009	Redlich et al.	2013/0061167 A1	3/2013	Rhodes et al.
2009/0271412 A1	10/2009	Lacapra et al.	2013/0066930 A1	3/2013	Kamei et al.
2009/0287887 A1	11/2009	Matsuki et al.	2013/0117744 A1	5/2013	Klein et al.
2009/0288084 A1	11/2009	Astete et al.	2013/0132674 A1	5/2013	Sundrani
2010/0023521 A1	1/2010	Arcese et al.	2013/0151888 A1	6/2013	Bhattiprolu et al.
2010/0070725 A1	3/2010	Prahlad et al.	2013/0152085 A1 *	6/2013	D'Amore G06F 9/45558
2010/0082716 A1	4/2010	Agestsuma et al.			718/1
2010/0082774 A1	4/2010	Pitts	2013/0174165 A1	7/2013	Chopra
2010/0095289 A1	4/2010	Nguyen et al.	2013/0185716 A1	7/2013	Yin et al.
2010/0138921 A1	6/2010	Na et al.	2013/0198738 A1	8/2013	Reddin et al.
2010/0174745 A1	7/2010	Ryan et al.	2013/0212345 A1	8/2013	Nakajima
2010/0214908 A1	8/2010	Ralev	2013/0227379 A1	8/2013	Gupta et al.
2010/0241785 A1	9/2010	Chen et al.	2013/0227552 A1	8/2013	Reddin et al.
2010/0250824 A1	9/2010	Belay	2013/0227566 A1	8/2013	Higuchi et al.
2010/0275205 A1	10/2010	Nakajima	2013/0232491 A1	9/2013	Radhakrishnan et al.
2010/0299377 A1	11/2010	Rimer et al.	2013/0246705 A1	9/2013	Diare
2010/0306256 A1	12/2010	Blackman	2013/0247036 A1	9/2013	Fujiwara
2011/0022694 A1	1/2011	Dalal et al.	2013/0262396 A1	10/2013	Kripalani et al.
2011/0022695 A1	1/2011	Dalal et al.	2013/0282994 A1	10/2013	Wires et al.
2011/0022812 A1	1/2011	Van et al.	2013/0283267 A1	10/2013	Cooper et al.
2011/0022883 A1	1/2011	Hansen	2013/0297869 A1	11/2013	Mills et al.
2011/0047340 A1	2/2011	Olson et al.	2014/0006465 A1	1/2014	Davis et al.
2011/0078318 A1	3/2011	Desai et al.	2014/0006708 A1	1/2014	Huynh et al.
2011/0119668 A1	5/2011	Calder et al.	2014/0025796 A1	1/2014	Vibhor et al.
2011/0119763 A1	5/2011	Wade et al.	2014/0059392 A1	2/2014	Ren et al.
2011/0125835 A1	5/2011	Soltis	2014/0075029 A1	3/2014	Lipchuk et al.
2011/0137879 A1	6/2011	Dubey et al.	2014/0095544 A1	4/2014	Eshel et al.
2011/0153561 A1	6/2011	Sawdon et al.	2014/0095555 A1	4/2014	Kim et al.
2011/0161299 A1	6/2011	Prahlad et al.	2014/0095816 A1	4/2014	Hsu et al.
2011/0179414 A1	7/2011	Goggin et al.	2014/0108587 A1	4/2014	Goldberg et al.
2011/0184993 A1	7/2011	Chawla et al.	2014/0115182 A1	4/2014	Sabaa et al.
2011/0185292 A1	7/2011	Chawla et al.	2014/0123138 A1	5/2014	Lee et al.
2011/0225574 A1	9/2011	Khalidi et al.	2014/0146055 A1	5/2014	Bala et al.
2011/0238949 A1	9/2011	Archer et al.	2014/0149794 A1	5/2014	Shetty et al.
2011/0239039 A1	9/2011	Dieffenbach et al.	2014/0149983 A1	5/2014	Bonilla et al.
2011/0239213 A1	9/2011	Aswani et al.	2014/0164449 A1	6/2014	Kim et al.
2011/0251992 A1	10/2011	Bethlehem et al.	2014/0173199 A1	6/2014	Gupta et al.
2011/0252208 A1	10/2011	Ali et al.	2014/0181048 A1	6/2014	Varadharajan et al.
2011/0255538 A1	10/2011	Srinivasan et al.	2014/0181116 A1	6/2014	Wang
2011/0265076 A1	10/2011	Thorat et al.	2014/0181438 A1	6/2014	Varadharajan et al.
2011/0271279 A1	11/2011	Pate	2014/0188808 A1	7/2014	Wolf et al.
2011/0276578 A1	11/2011	Allalouf et al.	2014/0189429 A1	7/2014	Gill et al.
2011/0276963 A1	11/2011	Wu et al.	2014/0189677 A1	7/2014	Curzi et al.
2011/0283277 A1	11/2011	Castillo et al.	2014/0189685 A1	7/2014	Kripalani
2011/0289561 A1	11/2011	Ivanov et al.	2014/0189686 A1	7/2014	Masters et al.
2011/0296069 A1	12/2011	Tarta et al.	2014/0196038 A1	7/2014	Kottomtharayil et al.
			2014/0201725 A1	7/2014	Tian et al.
			2014/0207824 A1	7/2014	Brandwine et al.
			2014/0230024 A1	8/2014	Uehara et al.
			2014/0237464 A1	8/2014	Waterman et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

2014/0250300 A1	9/2014	Runkis et al.	2016/0328007 A1	11/2016	Aranjani et al.
2014/0279909 A1	9/2014	Sudarsanam et al.	2016/0328226 A1	11/2016	Arya et al.
2014/0298185 A1	10/2014	Chen et al.	2016/0335134 A1	11/2016	Gupta et al.
2014/0310710 A1	10/2014	Lubsey et al.	2016/0359697 A1	12/2016	Scheib et al.
2014/0359612 A1	12/2014	D'Amato et al.	2016/0371020 A1	12/2016	Sarkar et al.
2014/0365549 A1	12/2014	Jenkins	2016/0378528 A1	12/2016	Zamir
2015/0006788 A1	1/2015	Liu et al.	2016/0378616 A1	12/2016	Wigmore et al.
2015/0007180 A1	1/2015	Sharp et al.	2017/0004131 A1	1/2017	Ben Dayan et al.
2015/0026682 A1	1/2015	Singh et al.	2017/0005990 A1	1/2017	Birger et al.
2015/0032690 A1	1/2015	Hoque et al.	2017/0012904 A1	1/2017	Matzek et al.
2015/0039735 A1	2/2015	Zeyliger et al.	2017/0024152 A1	1/2017	Bhagi et al.
2015/0039837 A1	2/2015	Quan et al.	2017/0024224 A1	1/2017	Bakke et al.
2015/0058475 A1	2/2015	Earl et al.	2017/0039078 A1	2/2017	Chen et al.
2015/0074536 A1	3/2015	Varadharajan et al.	2017/0039082 A1	2/2017	Ganesan et al.
2015/0074659 A1	3/2015	Madsen et al.	2017/0039218 A1	2/2017	Prahlad et al.
2015/0081644 A1	3/2015	Pitts	2017/0041296 A1	2/2017	Ford et al.
2015/0095597 A1	4/2015	Ayanam et al.	2017/0048223 A1	2/2017	Anantha et al.
2015/0095788 A1	4/2015	Thiele et al.	2017/0068469 A1	3/2017	Shankar et al.
2015/0106802 A1	4/2015	Ivanov et al.	2017/0075921 A1	3/2017	Benton et al.
2015/0120925 A1	4/2015	Das	2017/0090776 A1	3/2017	Kowles
2015/0142745 A1	5/2015	Tekade et al.	2017/0091047 A1	3/2017	Bangalore et al.
2015/0142747 A1	5/2015	Zou	2017/0091262 A1	3/2017	Beard et al.
2015/0143164 A1	5/2015	Veerla et al.	2017/0109184 A1	4/2017	Ramani et al.
2015/0172412 A1	6/2015	Escriva et al.	2017/0123890 A1	5/2017	Haridas et al.
2015/0178019 A1	6/2015	Hegdal et al.	2017/0155691 A1	6/2017	Knauff et al.
2015/0186635 A1	7/2015	Nakhjiri et al.	2017/0160983 A1	6/2017	Fiske et al.
2015/0193144 A1	7/2015	Bilas et al.	2017/0177638 A1	6/2017	Bhosale et al.
2015/0205639 A1	7/2015	Matsumoto et al.	2017/0193021 A1	7/2017	Deng et al.
2015/0213032 A1	7/2015	Powell et al.	2017/0206074 A1	7/2017	Arcese et al.
2015/0220324 A1	8/2015	Arcese et al.	2017/0206207 A1	7/2017	Bondurant et al.
2015/0242291 A1	8/2015	Chang et al.	2017/0220661 A1	8/2017	Cao et al.
2015/0248402 A1	9/2015	Patterson, III et al.	2017/0228300 A1	8/2017	Thomas et al.
2015/0256617 A1	9/2015	Klose et al.	2017/0235507 A1	8/2017	Sinha et al.
2015/0278046 A1	10/2015	Zellermayer et al.	2017/0235562 A1	8/2017	Bafna et al.
2015/0293830 A1	10/2015	Bhide et al.	2017/0235563 A1	8/2017	Bafna et al.
2015/0293896 A1	10/2015	Runkis et al.	2017/0235589 A1	8/2017	Gopalapura et al.
2015/0301903 A1	10/2015	Mutha et al.	2017/0235590 A1	8/2017	Sinha et al.
2015/0309825 A1	10/2015	Farkas et al.	2017/0235591 A1	8/2017	Kanada et al.
2015/0324217 A1	11/2015	Shilmover et al.	2017/0235653 A1	8/2017	Arikatla et al.
2015/0326531 A1	11/2015	Cui et al.	2017/0235654 A1	8/2017	Deshmukh et al.
2015/0331757 A1	11/2015	Durge et al.	2017/0235751 A1	8/2017	Gupta et al.
2015/0355862 A1	12/2015	Hayes et al.	2017/0235758 A1	8/2017	Gopalapura et al.
2015/0356125 A1	12/2015	Golander et al.	2017/0235760 A1	8/2017	Sharpe et al.
2015/0378761 A1	12/2015	Sevigny et al.	2017/0235761 A1	8/2017	Bafna et al.
2015/0378853 A1	12/2015	Sevigny et al.	2017/0235762 A1	8/2017	Sharpe et al.
2016/0011898 A1	1/2016	Lee et al.	2017/0235763 A1	8/2017	Gopalapura et al.
2016/0070492 A1	3/2016	Cherubini et al.	2017/0235764 A1	8/2017	Sharpe et al.
2016/0077988 A1	3/2016	Tipton et al.	2017/0235950 A1	8/2017	Gopalapura et al.
2016/0078068 A1	3/2016	Agrawal et al.	2017/0242599 A1	8/2017	Patnaik et al.
2016/0085480 A1	3/2016	Chiu et al.	2017/0242871 A1	8/2017	Kilaru et al.
2016/0085574 A1	3/2016	Dornemann et al.	2017/0262346 A1	9/2017	Pradhan et al.
2016/0087861 A1	3/2016	Kuan et al.	2017/0264684 A1	9/2017	Spillane et al.
2016/0100016 A1	4/2016	Mu et al.	2017/0277556 A1	9/2017	Ishii et al.
2016/0110214 A1	4/2016	Vincent et al.	2017/0277903 A1	9/2017	Christodorescu et al.
2016/0110267 A1	4/2016	Earl et al.	2017/0279674 A1	9/2017	Zhu
2016/0124663 A1	5/2016	Mitkar et al.	2017/0286228 A1	10/2017	Redko et al.
2016/0124665 A1	5/2016	Jain et al.	2017/0302589 A1	10/2017	Leafe et al.
2016/0162371 A1	6/2016	Prabhu et al.	2017/0302731 A1	10/2017	Cui
2016/0171241 A1	6/2016	Yun	2017/0329554 A1	11/2017	Voigt et al.
2016/0179416 A1	6/2016	Mutha et al.	2018/0004766 A1	1/2018	Darling
2016/0179419 A1	6/2016	Yamaguchi et al.	2018/0062993 A1	3/2018	Wu et al.
2016/0180107 A1 *	6/2016	Panchbudhe H04L 63/102 726/28	2018/0107838 A1	4/2018	Amarendran et al.
2016/0188232 A1	6/2016	Ramachandran et al.	2018/0129426 A1	5/2018	Aron et al.
2016/0188407 A1	6/2016	Bronnikov et al.	2018/0143845 A1	5/2018	Chawla et al.
2016/0202916 A1	7/2016	Cui et al.	2018/0143880 A1	5/2018	Dornemann
2016/0203008 A1	7/2016	Cui et al.	2018/0145960 A1	5/2018	Bakshan et al.
2016/0216993 A1	7/2016	Beckwith et al.	2018/0157521 A1	6/2018	Arikatla et al.
2016/0224363 A1	8/2016	Joy	2018/0157522 A1	6/2018	Bafna et al.
2016/0274926 A1	9/2016	Narasimhamurthy et al.	2018/0157561 A1	6/2018	Venkatesh et al.
2016/0301766 A1	10/2016	Ionescu et al.	2018/0157677 A1	6/2018	Bafna et al.
2016/0306558 A1	10/2016	Varadharajan et al.	2018/0157752 A1	6/2018	Arikatla et al.
2016/0306751 A1	10/2016	Amarendran et al.	2018/0157860 A1	6/2018	Nair et al.
2016/0314046 A1	10/2016	Kumarasamy	2018/0159729 A1	6/2018	Deshmukh et al.
2016/0316003 A1	10/2016	Snider et al.	2018/0159826 A1	6/2018	Msan et al.
			2018/0173731 A1	6/2018	Nazari et al.
			2018/0181266 A1	6/2018	von Muhlen et al.
			2018/0181448 A1	6/2018	von Muhlen et al.
			2018/0181449 A1	6/2018	von Muhlen et al.
			2018/0181587 A1	6/2018	von Muhlen et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

2018/0181723	A1	6/2018	von Muhlen et al.
2018/0196719	A1	7/2018	Glass
2018/0205787	A1	7/2018	Ben Dayan et al.
2018/0225204	A1	8/2018	Choudhari et al.
2018/0278602	A1	9/2018	Koushik et al.
2018/0332105	A1	11/2018	Huang et al.
2018/0357251	A1	12/2018	Kumarasamy et al.
2019/0026101	A1	1/2019	Venkatesh et al.
2019/0034240	A1	1/2019	Nabi et al.
2019/0079747	A1	3/2019	Sinha et al.
2019/0087279	A1	3/2019	Kumar et al.
2019/0087280	A1	3/2019	Kumar et al.
2019/0087281	A1	3/2019	Kumar et al.
2019/0087282	A1	3/2019	Kumar et al.
2019/0087285	A1	3/2019	Kumar et al.
2019/0087286	A1	3/2019	Kumar et al.
2019/0129808	A1	5/2019	Acharya et al.
2019/0196718	A1	6/2019	Pai et al.
2019/0207925	A1	7/2019	Anantha et al.
2019/0286330	A1	9/2019	Varadharajan et al.
2019/0332683	A1	10/2019	Thummala et al.
2019/0339883	A1	11/2019	Aron et al.
2020/0007530	A1	1/2020	Mohamad Abdul et al.
2020/0034069	A1	1/2020	Batra et al.
2020/0036647	A1	1/2020	Gupta et al.
2020/0042537	A1	2/2020	Kaushik et al.
2020/0050522	A1	2/2020	Coleman et al.
2020/0081704	A1	3/2020	Bafna et al.
2020/0081733	A1	3/2020	Buck et al.
2020/0106669	A1	4/2020	Dhillon et al.
2020/0125426	A1	4/2020	von Muhlen et al.
2020/0125580	A1	4/2020	Shao
2020/0137157	A1	4/2020	Joseph et al.
2020/0274869	A1	8/2020	Tahenakos et al.
2020/0387575	A1	12/2020	Palekar et al.
2021/0049240	A1	2/2021	Earl et al.
2021/0081358	A1	3/2021	Khurana et al.
2021/0141630	A1	5/2021	Sharpe et al.
2021/0165759	A1	6/2021	Bar-Nissan et al.
2021/0200641	A1	7/2021	Bafna et al.
2021/0224233	A1	7/2021	Bafna et al.
2021/0247973	A1	8/2021	Gupta et al.
2021/0334178	A1	10/2021	Yang et al.
2021/0344772	A1	11/2021	Arikatla et al.
2021/0349859	A1	11/2021	Bafna et al.
2021/0365257	A1	11/2021	Venkatesh et al.
2021/0382636	A1	12/2021	Perumal et al.
2021/0390080	A1	12/2021	Tripathi et al.
2021/0397587	A1	12/2021	Thummala et al.
2021/0406136	A1	12/2021	Venkatesh et al.
2022/0004377	A1	1/2022	Sharpe et al.
2022/0147342	A1	5/2022	Sharpe et al.
2022/0147495	A1	5/2022	Sharpe et al.
2022/0155964	A1	5/2022	Varadharajan et al.
2022/0156107	A1	5/2022	Bafna et al.
2022/0229687	A1	7/2022	Singhal et al.
2022/0300335	A1	9/2022	Venkatesh et al.
2022/0350591	A1	11/2022	Bafna et al.
2022/0350592	A1	11/2022	Gopalapura et al.
2023/0237102	A1	7/2023	Arikatla et al.
2023/0289170	A1	9/2023	Sharpe et al.
2023/0325173	A1	10/2023	Gupta et al.
2024/0103985	A1	3/2024	Venkatesh et al.
2025/0097231	A1	3/2025	Lai et al.

FOREIGN PATENT DOCUMENTS

CN	106357744	A	1/2017
CN	110516005	A	11/2019
CN	110519112	A	11/2019
CN	110569269	A	12/2019
CN	111259376	A	6/2020
CN	109076126	B	9/2020
EP	1062581	B1	10/2003
EP	1214663	B1	6/2006

EP	1979814	A2	10/2008
WO	2010050944	A1	5/2010
WO	2012126177	A2	9/2012
WO	2016018446	A1	2/2016
WO	2018014650	A1	1/2018

OTHER PUBLICATIONS

“How to Efficiently Deploy Virtual Machines from VMware vSphere Content Library”, VMware VROOM! Performance Blog; retrieved from: <https://blogs.vmware.com/performance/2015/07/efficiently-deploy-vms-vmware-vsphere-content-library.html>, Jul. 9, 2015.

“New VMware Icons Visio Stencil Download”, VM Today; obtained from: <https://web.archive.org/web/20230130190532/https://vmtoday.com/2012/09/new-vmware-icons-visio-stencil-download/>, Sep. 16, 2012.

“Open Virtualization Format White Paper”, DMTF Informational, Version 2.0.0; Distributed Management Task Force, Inc. (DMTF); obtained from https://www.dmtf.org/sites/default/files/standards/documents/DSP2017_2.0.0.pdf, Apr. 14, 2014.

U.S. Appl. No. 18/054,490 titled “Virtualized File Server Distribution Across Clusters”, filed Nov. 10, 2022.

U.S. Appl. No. 18/069,920 titled “Transparent Referrals for Distributed File Servers”, filed Dec. 21, 2022.

Isilon OneFS, Version 8.0.1; Web Administration Guide, Oct. 2016. “Administering VMware vSAN—VMware vSphere 7.0”, Update 1, 2015-2020, pp. 1-114.

“Citrix XenDesktop 7.1 on Microsoft Hyper-V Server 2012 R2 on Nutanix Virtual Computing Platform”, Citrix APAC Solutions, Jun. 25, 2014, pp. 1-94.

“Clustered Data Ontap 8.2 File Access Management Guide for CIFS”, NetApp; (year 2014), Feb. 2014, pp. all.

“Designing and Sizing Virtual SAN Fault Domains”, Administering VMware Virtual SAN; VMware vSphere 6.5; vSAN 6.6; <https://docs.vmware.com/en/VMware-vSphere/6.5/virtual-san-66-administration-guide.pdf>, captured Aug. 20, 2021, 2017, pp. 34.

“Hybrid Cloud Storage with Cloudian HyperStore and Amazon S3”, Cloudian Inc.; www.cloudian.com, 2014, pp. all.

“Preparing Storage Systems for Snapmirror Replication”, NetApp; NetApp, Apr. 2005, pp. all.

“Setting up and Using Acropolis File Services (AFS) on Nutanix AOS 5.0”, Virtual Dennis—Sharing Technical Tips Learned the Hard Way, Dec. 30, 2016, pp. all.

“Tech TopX: AHV One Click Upgrade”, Screen captures from YouTube video clip, 13 pages, uploaded on Dec. 8, 2015 by user “Nutanix University”. Retrieved from Internet: <https://www.youtube.com/watch?v=3dALdzw6qZM>, Dec. 8, 2015, pp. all.

“Technical Overview and Best Practices”, VMware vSphere VMFS ; a VMware Technical White Paper updated for VMware vSphere 5.1, Version 3.0, Nov. 27, 2012, pp. all.

“Virtual Disk Manager User’s Guide: Virtual Disk Development Kit”, vmware.com, 2008, 1-12.

“VMware vCenter Server: Centrally Managed Virtual Infrastructure Delivered with Confidence”, VMware DATASHEET; <https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/products/vcenter/vmware-vcenter-server-datasheet.pdf>, captured Aug. 20, 2021, 2015, 2015, pp. 1-2.

Bounds, Jay , “High-Availability (HA) Pair Controller Configuration Overview and Best Practices”, NetApp, Feb. 2016, pp. all.

Cano, Ignacio et al., “Curator: Self-Managing Storage for Enterprise Clusters”, University of Washington, Mar. 2017, pp. all.

Costa, Jorge , “High Availability Setup Using Veritas Cluster Server and NetApp Synchronous SnapMirror—One button Failover/Failback with SnapMirror Sync and Veritas Cluster Server”, NetApp Community, Nov. 18, 2010, pp. all.

Hogan, Cormac , “New updates from Nutanix—NOS 3.0 and NX-3000”, <https://cormachogan.com/2012/12/20/new-from-nutanix-nos-3-0-nx-3000/>, Dec. 20, 2012, pp. 1-7.

Jeffrey, Hemmes et al., “Cacheable Decentralized Groups for Grid Resource Access Control”, 2006 7th IEEE/ACM International Con-

(56)

References Cited**OTHER PUBLICATIONS**

ference on Grid Computing Department of Computer Science and Engineering, University of Notre Dame, doi: 10.1109/ICGRID.2006.311015, Sep. 2006, pp. 192-199.

Jung, Young-Woo et al., "Standard-Based Virtual Infrastructure Resource Management for Distributed and Heterogeneous Servers", ICACT, Feb. 15, 2009, pp. all.

Kaam, Bas V., "New in AOS 5.0: Nutanix Acropolis File Services", basvankaam.com, Jan. 5, 2017, pp. all.

Kemp, Erik, "NetApp SolidFire SnapMirror Architecture and Configuration", Technical Report, NetApp, Dec. 2017, pp. all.

Kleyman, Bill, "How Cloud Computing Changes Storage Tiering", <https://www.datacenterknowledge.com>; captured Jun. 4, 2019, Nov. 12, 2015, pp. all.

Leibovici, Andre, "Nutanix One-Click Upgrade now takes care of Firmware and Hypervisor too!", myvirtualcloud.net <https://myvirtualcloud.net/nutanix-one-click-upgrade-now-takes-care-of-firmware-and-hypervisor-too/>, Jul. 31, 2014, pp. 1-4.

Lye, Ben, "Implementing Windows Server 2008 FileSystem Quotas", Nov. 19, 2009, pp. 1-17.

Poitras, Steven, "The Nutanix Bible", <http://stevenpoitras.com/the-nutanix-bible/> (Publication date based on indicated capture date by Archive.org; first publication date unknown), Jan. 11, 2014, pp. all.

Poitras, Steven, "The Nutanix Bible", <https://nutanixbible.com/>, Jan. 12, 2016, pp. all.

Poitras, Steven, "The Nutanix Bible", <https://nutanixbible.com/>, Jan. 3, 2017, pp. all.

Poitras, Steven, "The Nutanix Bible", <http://stevenpoitras.com/the-nutanix-bible/> (Publication date based on indicated capture date by Archive.org; first publication date unknown), Jan. 7, 2015, pp. all.

Poitras, Steven, "The Nutanix Bible", <http://stevenpoitras.com/the-nutanix-bible/> (Publication date based on indicated capture date by Archive.org; first publication date unknown), Jun. 20, 2014, pp. all.

Poitras, Steven, "The Nutanix Bible", <https://nutanixbible.com/>, Jun. 8, 2017, pp. all.

Poitras, Steven, "The Nutanix Bible", <http://stevenpoitras.com/the-nutanix-bible/> (Publication date based on indicated capture date by Archive.org; first publication date unknown), Jun. 9, 2015, pp. all.

Poitras, Steven, "The Nutanix Bible", <https://nutanixbible.com/>, Jun. 9, 2016, pp. all.

Poitras, Steven, "The Nutanix Bible", <http://stevenpoitras.com/the-nutanix-bible/> (Publication date based on indicated capture date by Archive.org; first publication date unknown), Oct. 15, 2013, pp. all.

Poitras, Steven, "The Nutanix Bible", <https://nutanixbible.com/>, Sep. 4, 2015, pp. all.

Ruth, Paul, "Autonomic Live Adaption of Virtual Computational Environments in a Multi-Domain Infrastructure", 2006 IEEE International Conference on Autonomic Computing, 2006, pp. 5-14.

U.S. Appl. No. 18/183,142 titled "Virtualized File Server User Views", filed Mar. 13, 2023.

U.S. Appl. No. 18/306,595 titled "Self-Healing Virtualized File Server", filed Apr. 25, 2023.

"Virtual Machine Backup Guide", ESX Server 3.0 and VirtualCenter 2.0; VMware; https://www.vmware.com/pdf/vi3_30_20_vm_backup.pdf, 2006.

"VMWare High Availability & Data Recovery Configuration Guide v1", IBM; <https://www.ibm.com/support/pages/vmware-high-availability-data-recovery-configuration-guide-v1>, Nov. 2015.

"VSan Specialist: Stretched Clusters & Two-Node Clusters—Part 1", vCallaway; <http://vcallaway.com/vsan-specialist-stretched-clusters-two-node-clusters/>, Sep. 26, 2017.

"VSphere Replication Target Storage Consumption", VMware vSphere Blog; <https://blogs.vmware.com/vsphere/2015/04/vsphere-replication-target-storage-consumption.html>, Apr. 17, 2015.

Bhardwaj, Rishi, "The Wonderful World of Distributed Systems and the Art of Metadata Management", Nutanix, Inc., <https://www.nutanix.com/blog/the-wonderful-world-of-distributed-systems-and-metadata-management>; captured Aug. 19, 2021, Sep. 24, 2015, pp. 1-8.

US Patent Application for U.S. Appl. No. 18/393,105 titled "Virtualized File Server Smart Data Ingestion", filed Dec. 21, 2023.

Adam, Michael, "Samba's Way Toward SMB 3.0", vol. 38, No. 1 | https://www.usenix.org/system/files/login/articles/03adam_016-025_online.pdf, Feb. 2013, pp. 16-25.

Koehler, "Options in scalability with vSAN", VMware; Virtual Blocks Blog; retrieved from: <https://blogs.vmware.com/virtualblocks/2016/09/15/options-scalability-virtual-san/>, Sep. 15, 2016.

U.S. Appl. No. 18/450,319 titled "Disaster Recovery for Distributed File Servers, Including Metadata Fixers", filed Aug. 15, 2023.

U.S. Appl. No. 18/497,919 titled "Virtualized Server Systems and Methods Including Scaling of File System Virtual Machines", filed Oct. 30, 2023.

"The Open Virtual Machine Format Whitepaper for OVF Specification", VMware; retrieved from https://www.vmware.com/pdf/ovf_whitepaper_specification.pdf, Sep. 9, 2007.

"Changing the ESXi Host Name using vSphere Web Client", Pavan's Blog; retrieved from: <https://pchawda.wordpress.com/category/esxi/page/3/>, May 11, 2016.

"How NFS Referrals Work", From: Managing Network File Systems in Oracle® Solaris 11.2; Oracle.com; retrieved from: https://docs.oracle.com/cd/E36784_01/html/E36825/gjuaj.html#scrolltoc, Jul. 2014.

"The Architecture of VMware ESXi", VMware White Paper; retrieved from: https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/techpaper/ESXi_architecture.pdf, Oct. 24, 2008.

Bergamasco, Davide, "Content Library Performance Tuning", VMware VROOM! Performance Blog; received from: <https://blogs.vmware.com/performance/2015/09/content-library-performance-tuning.html>, Sep. 15, 2015.

Bergamasco, Davide, "How to Efficiently Synchronize, Import and Export Content in VMware vSphere Content Library", VMware VROOM! Performance Blog; retrieved from: <https://blogs.vmware.com/performance/2015/09/efficiently-synchronize-import-export-content-vmware-vsphere-content-library.html>, Sep. 3, 2015.

"Administering VMware vSAN—Update 1 VMware vSphere 8.0 VMware vSAN 8.0", <https://docs.vmware.com/en/VMware-vSphere/8.0/vsan-801-administration-guide.pdf>, 2015, pp. 1-151.

* cited by examiner

FIG. 1A

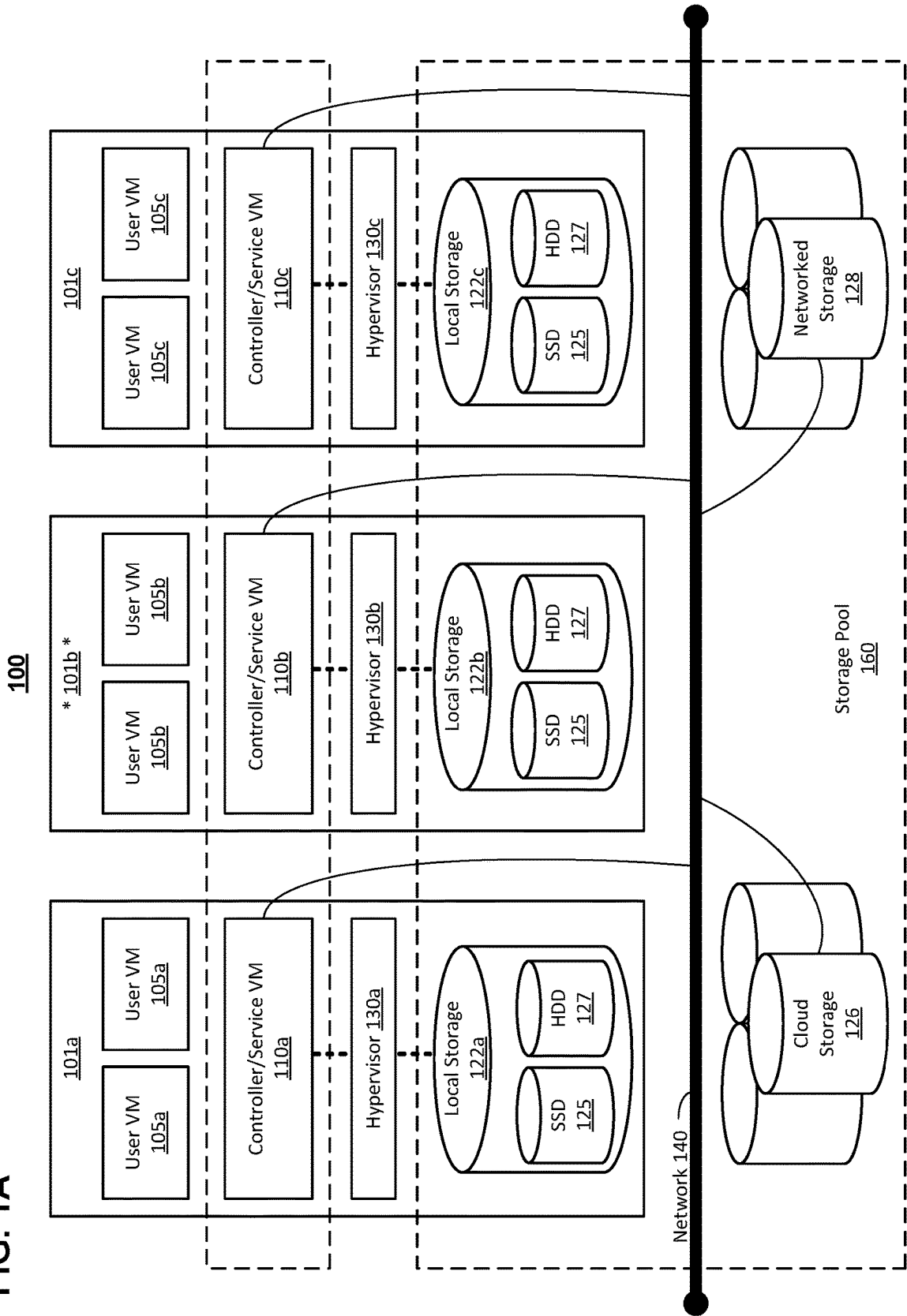


FIG. 1B

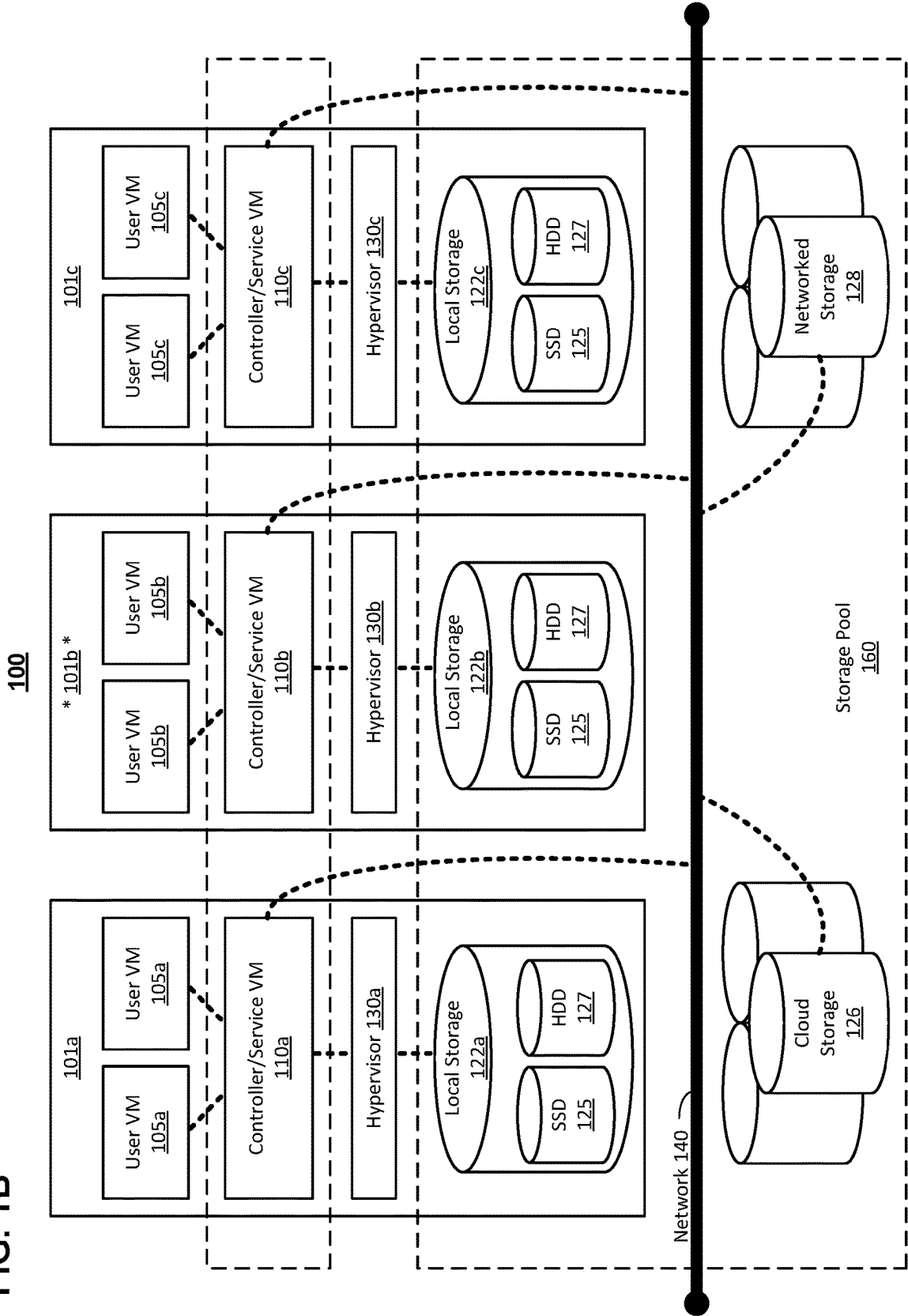


FIG. 2A

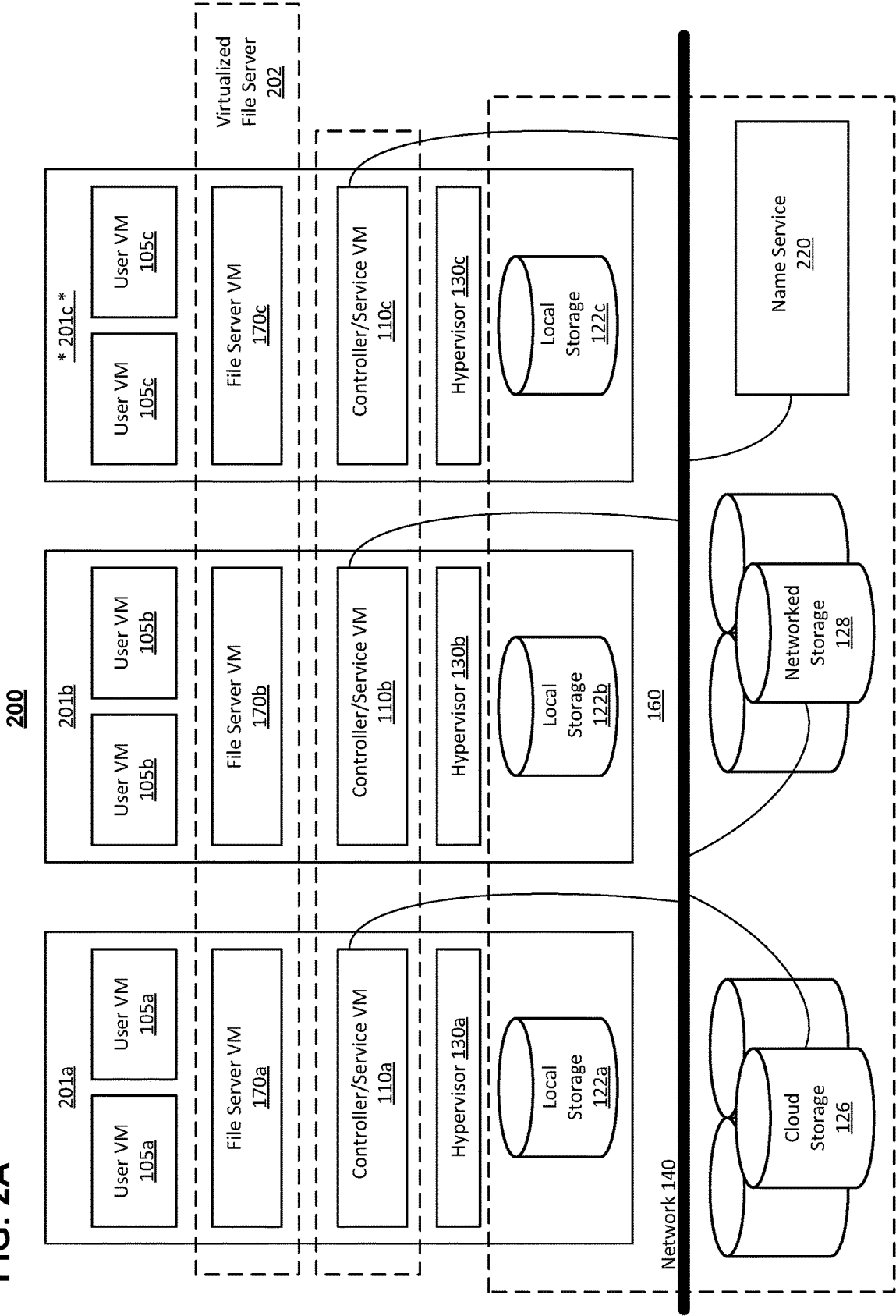
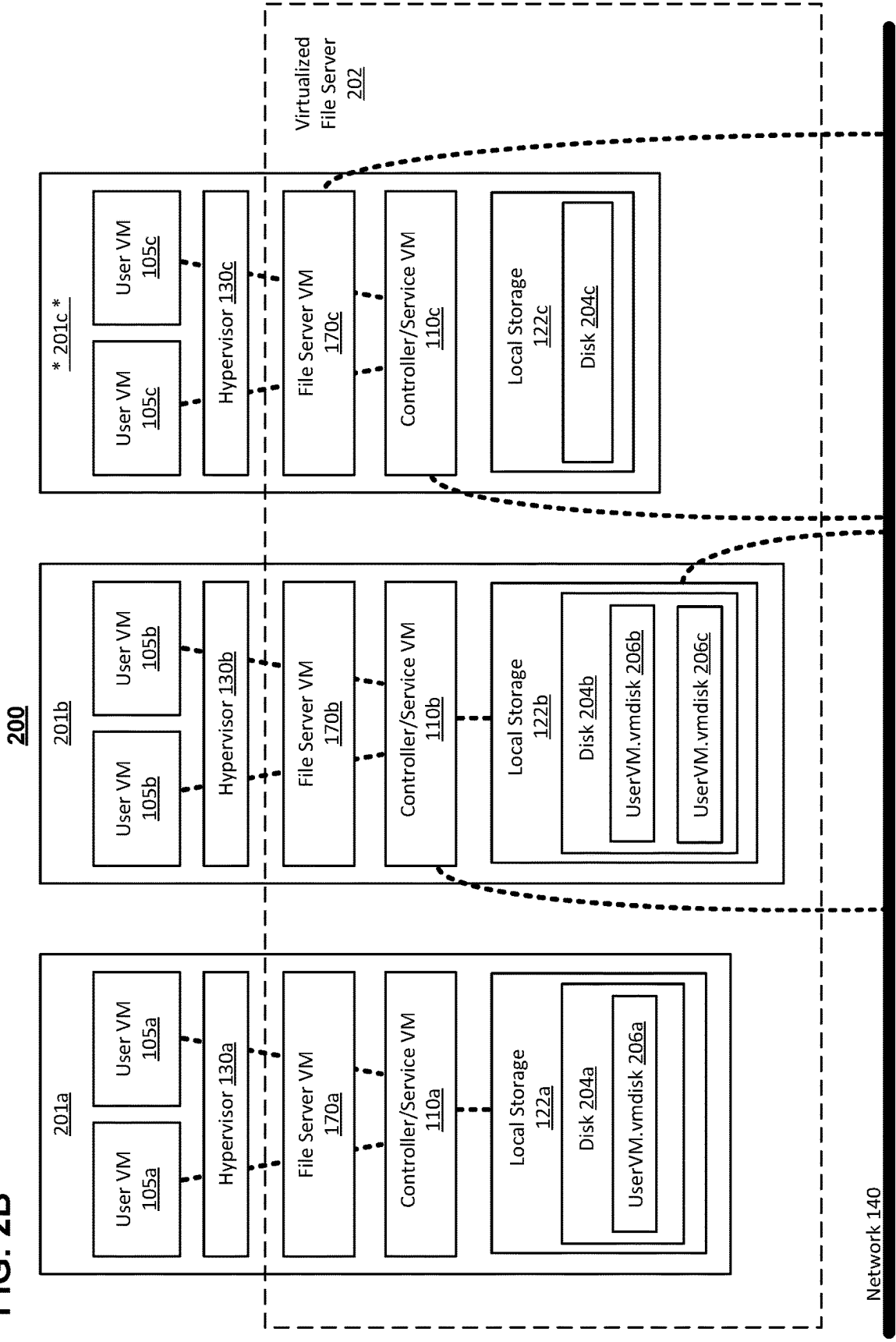
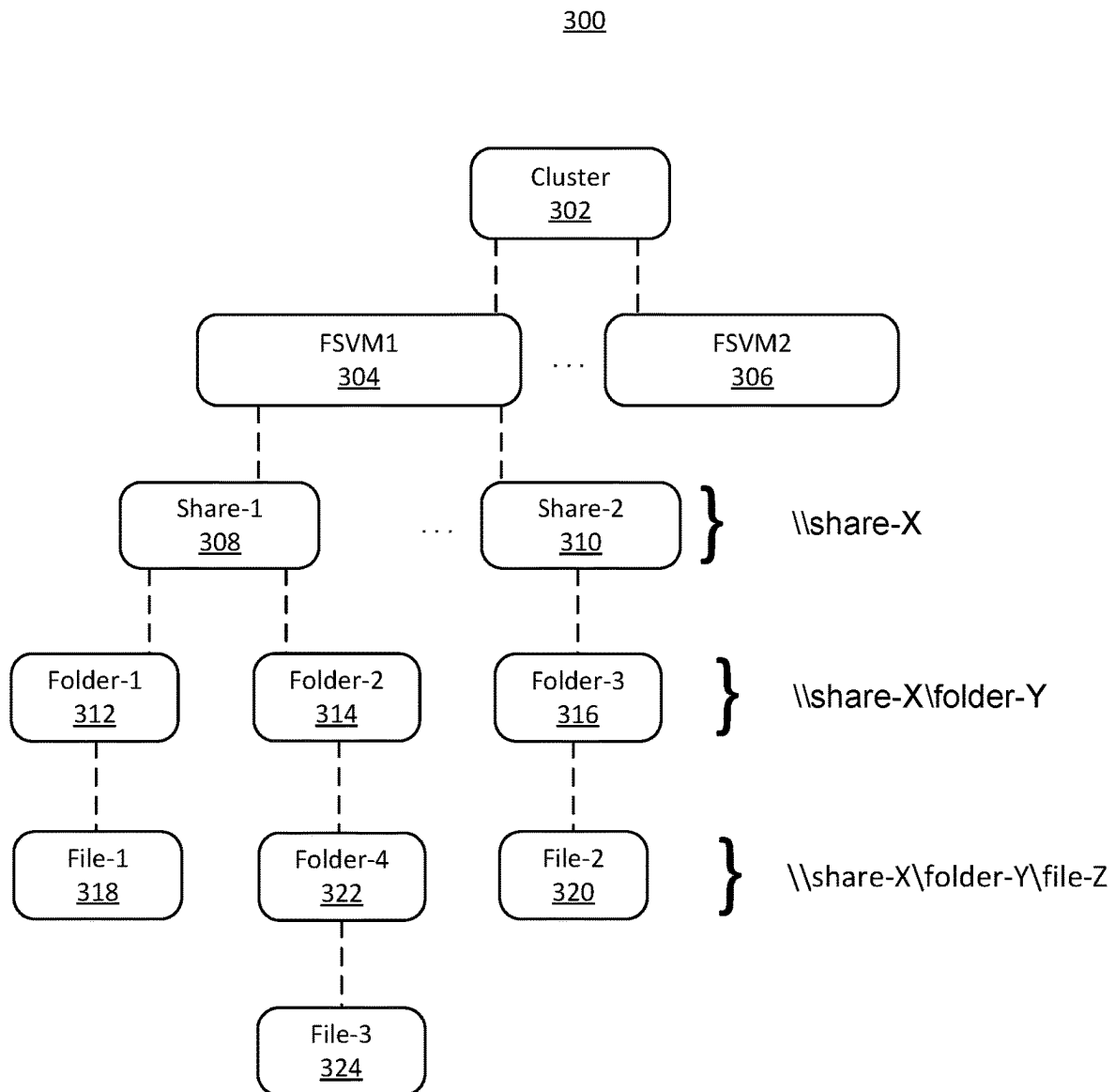


FIG. 2B





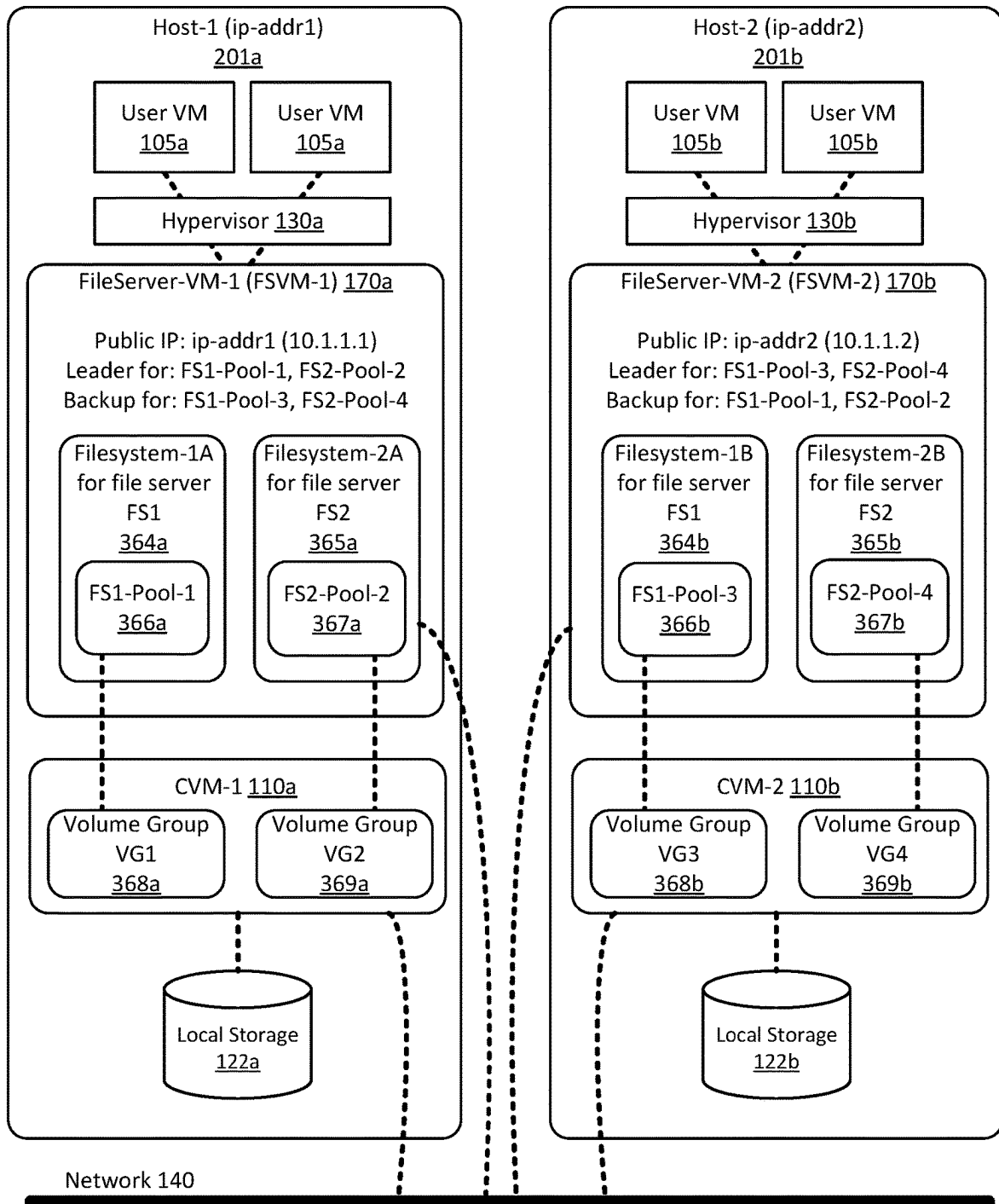


FIG. 3B

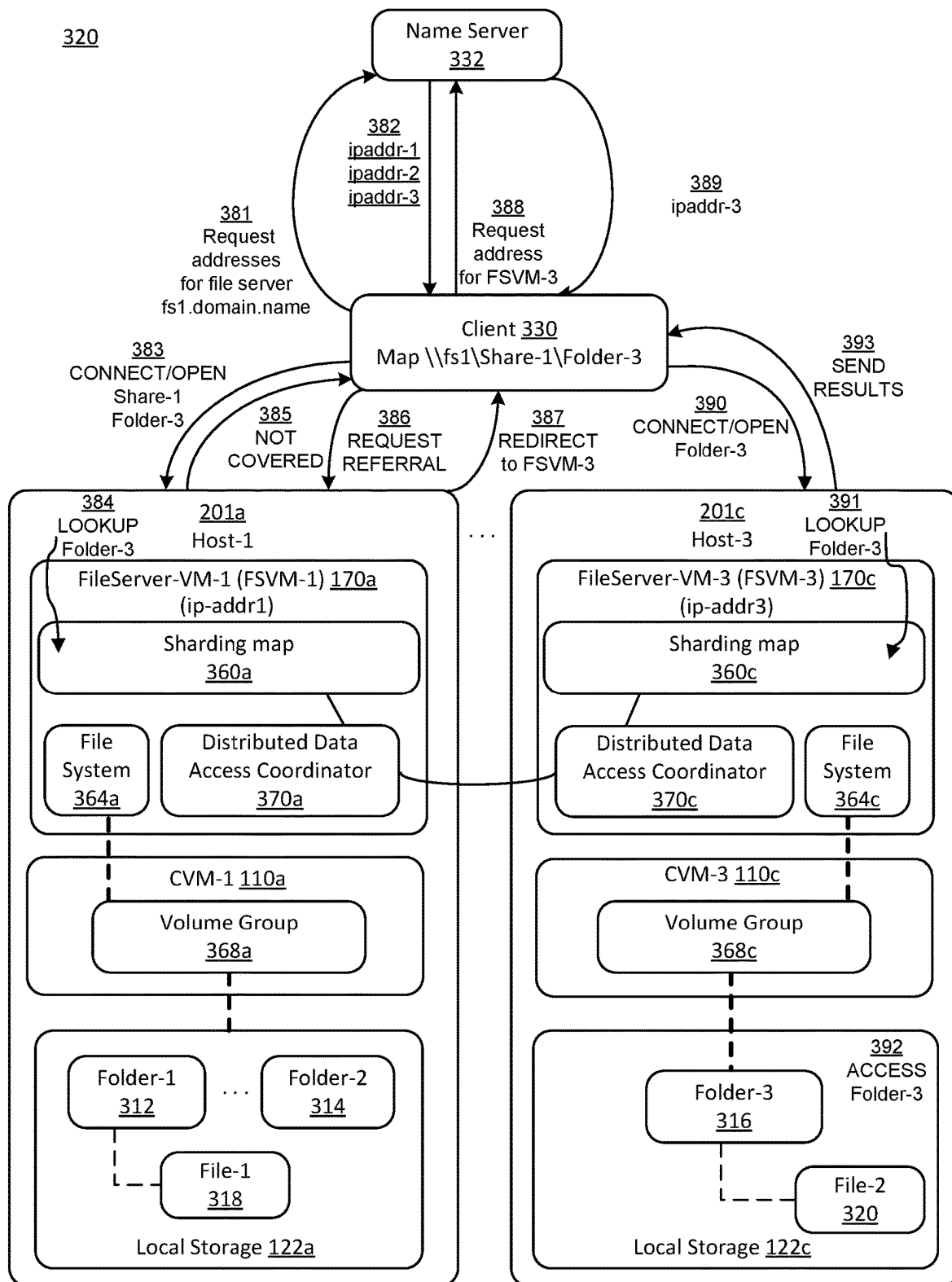
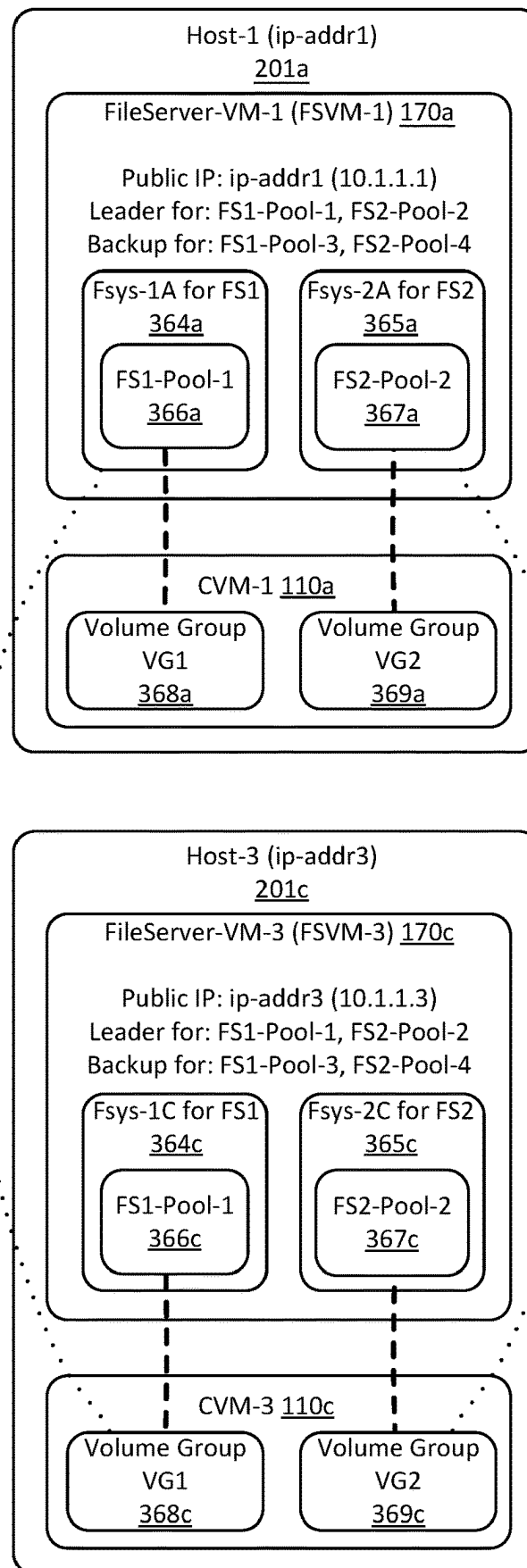


FIG. 3C

FIG. 3D



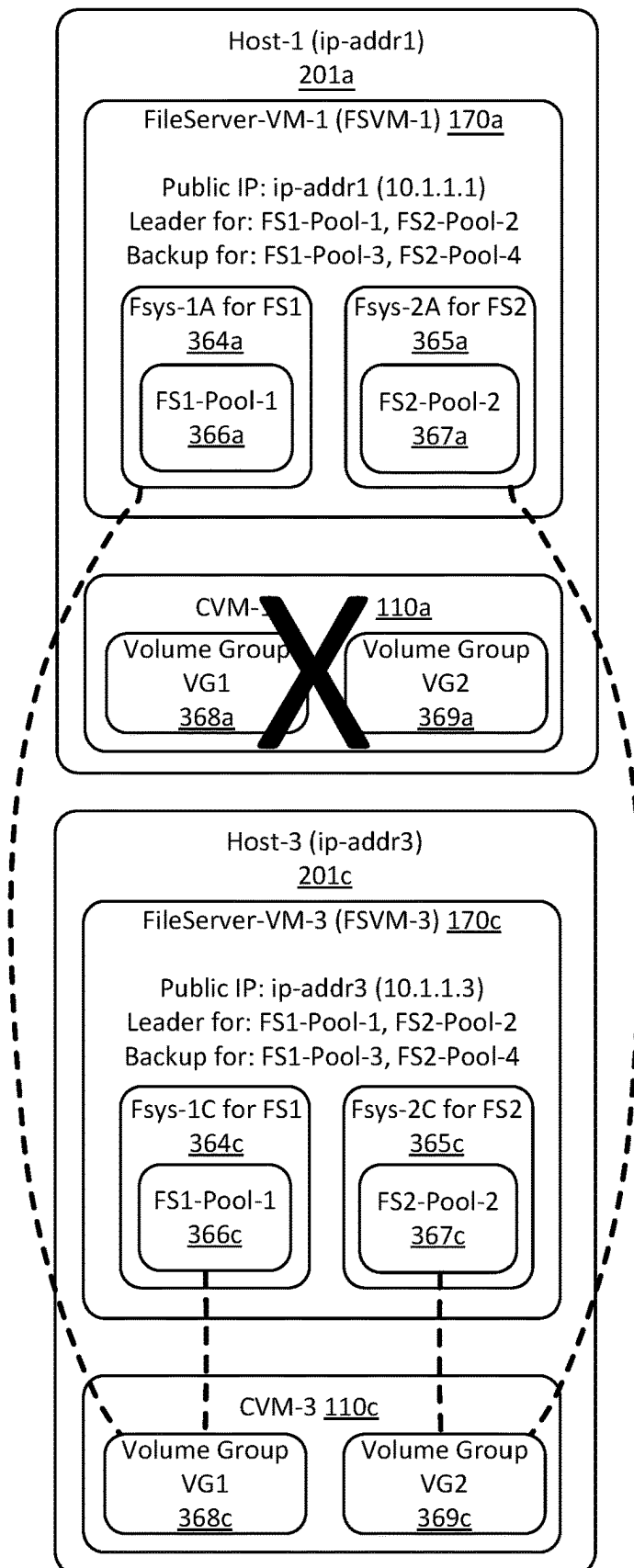
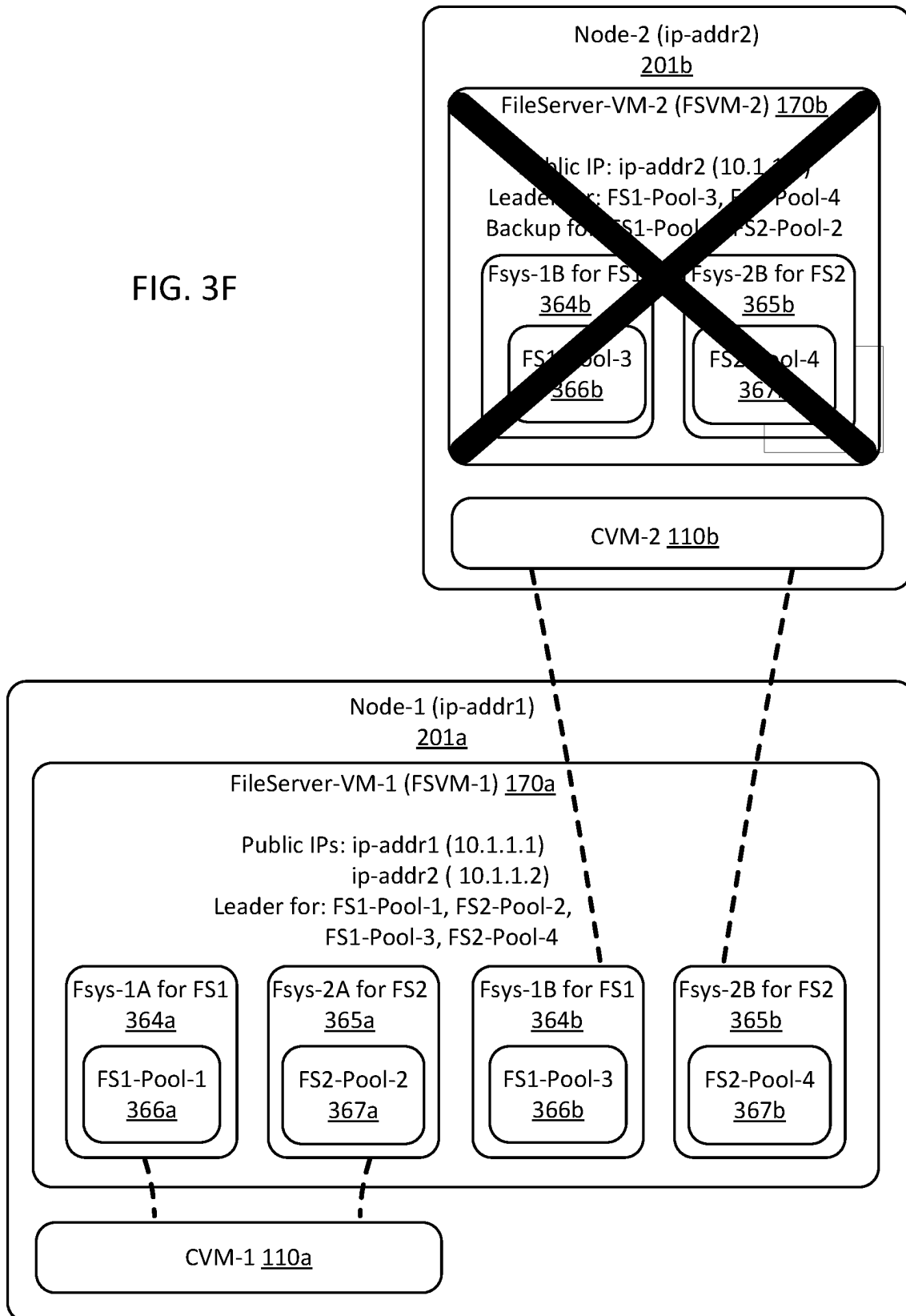


FIG. 3E

FIG. 3F



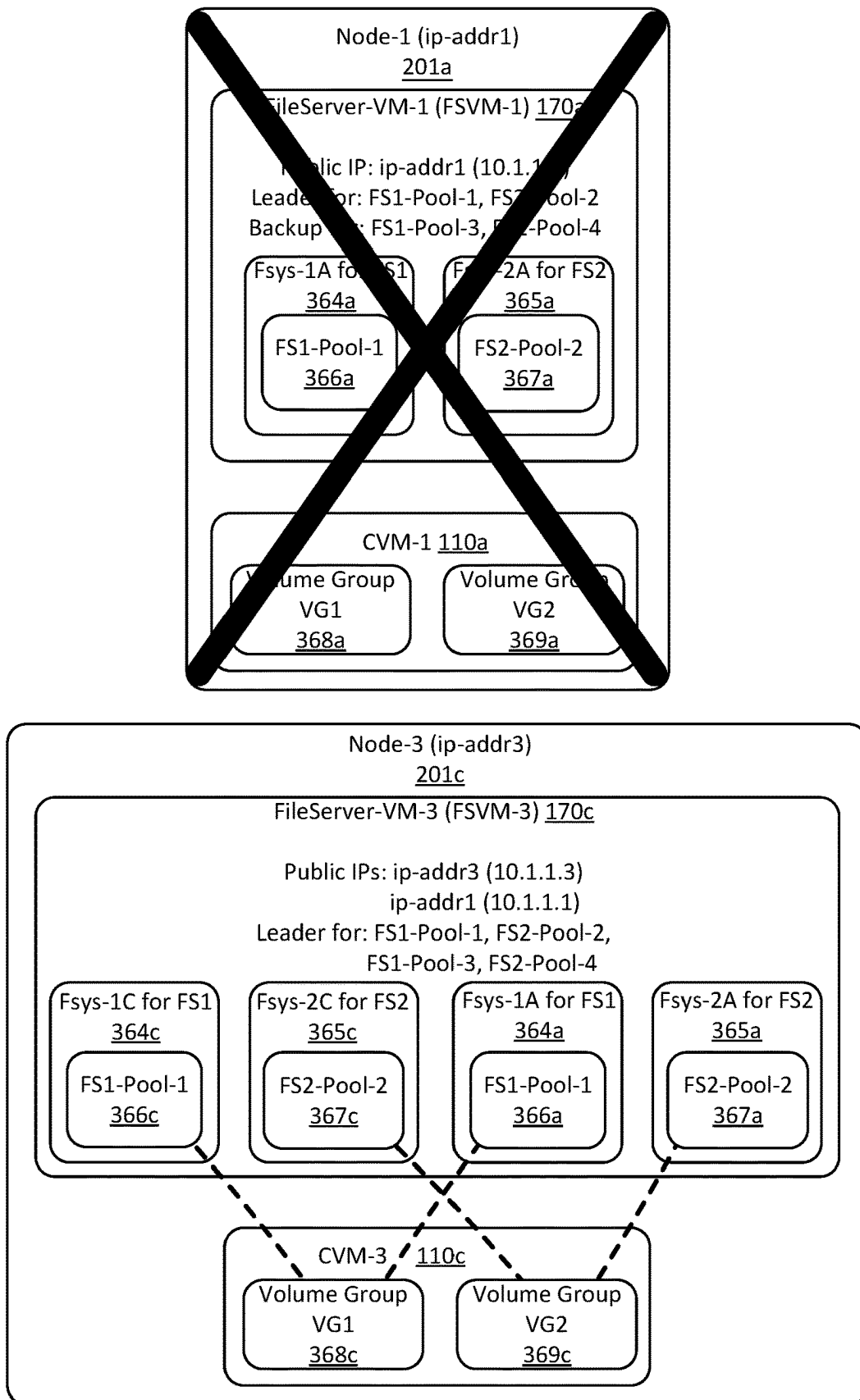


FIG. 3G

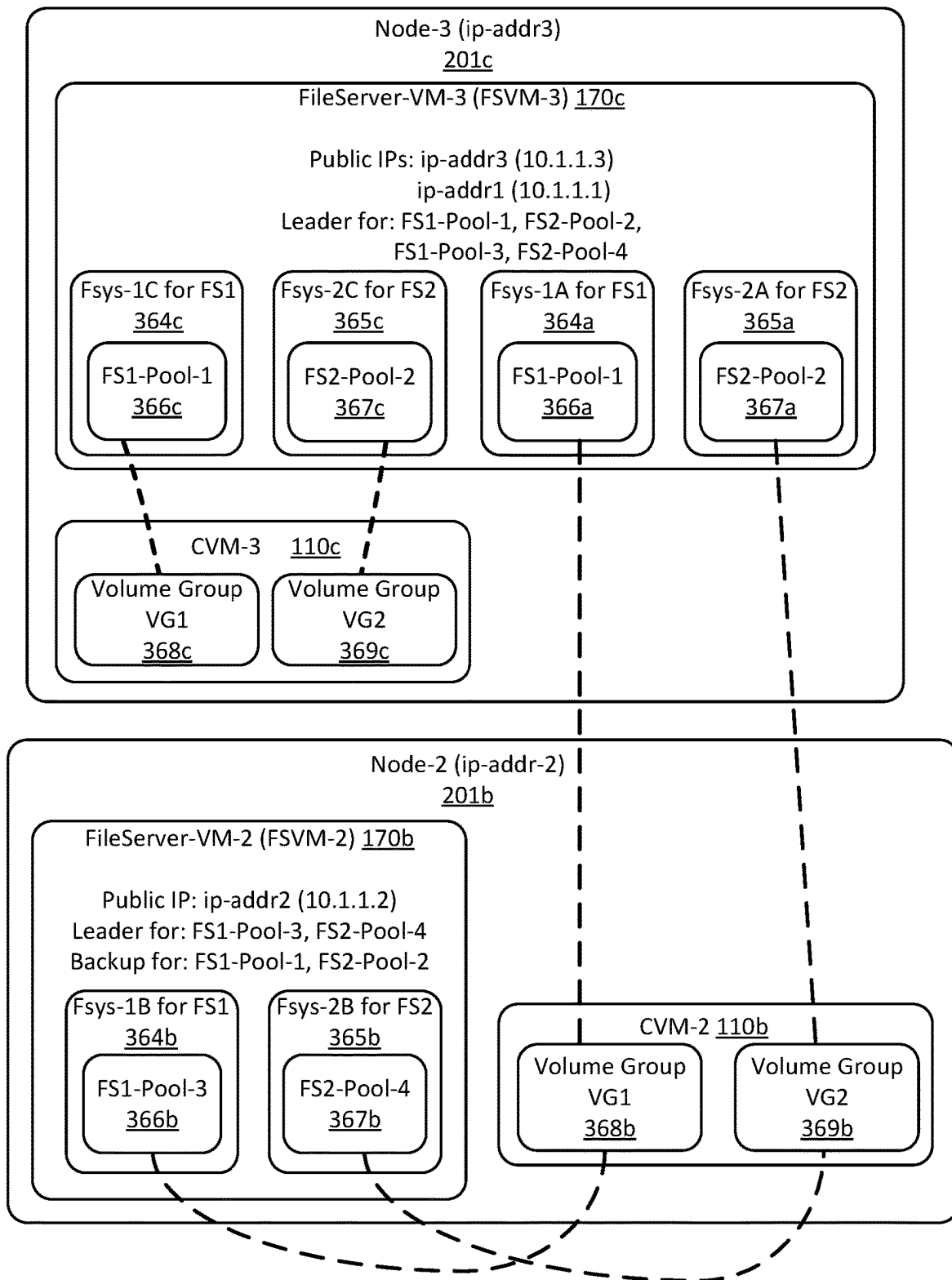


FIG. 3H

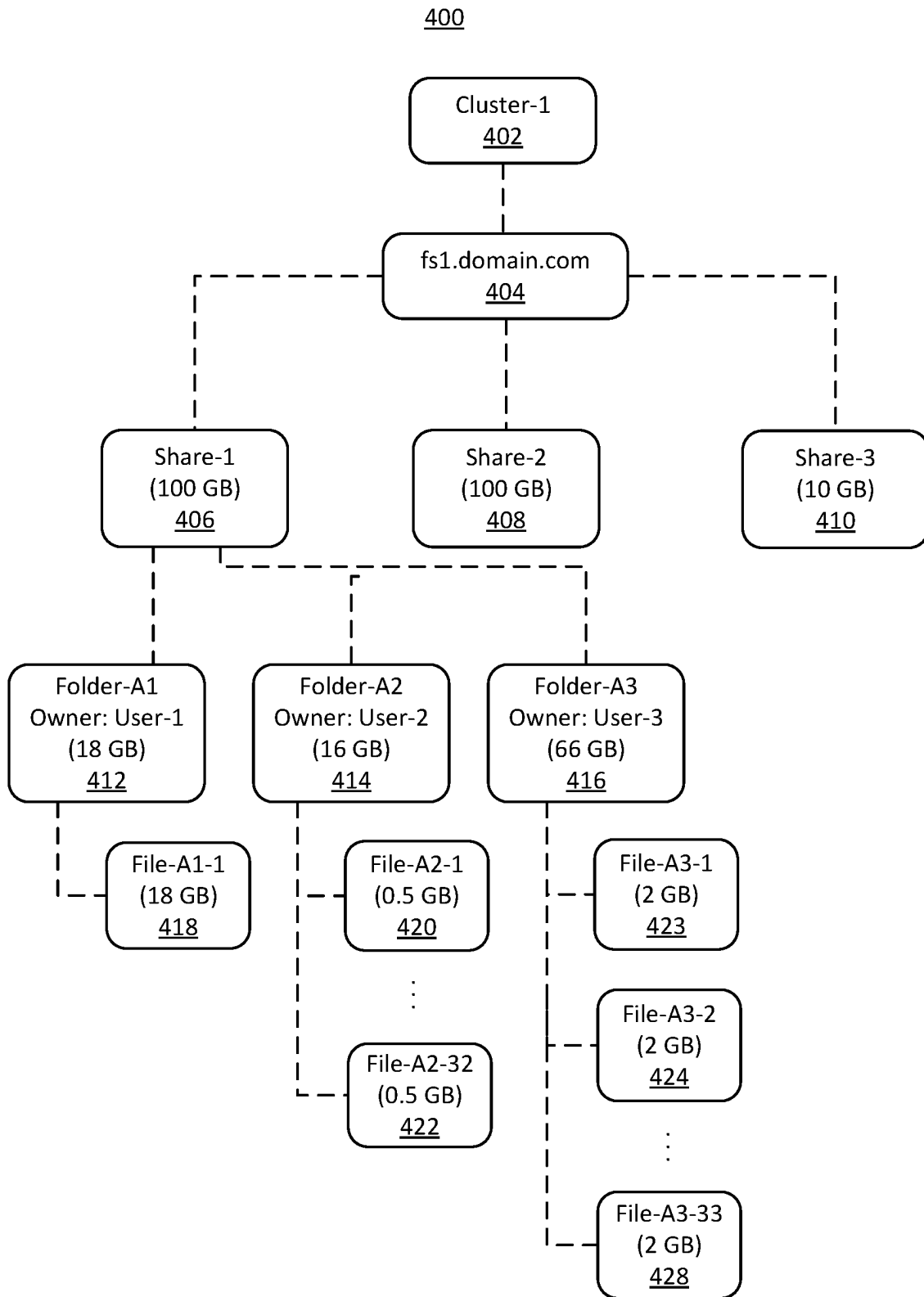


FIG. 4A

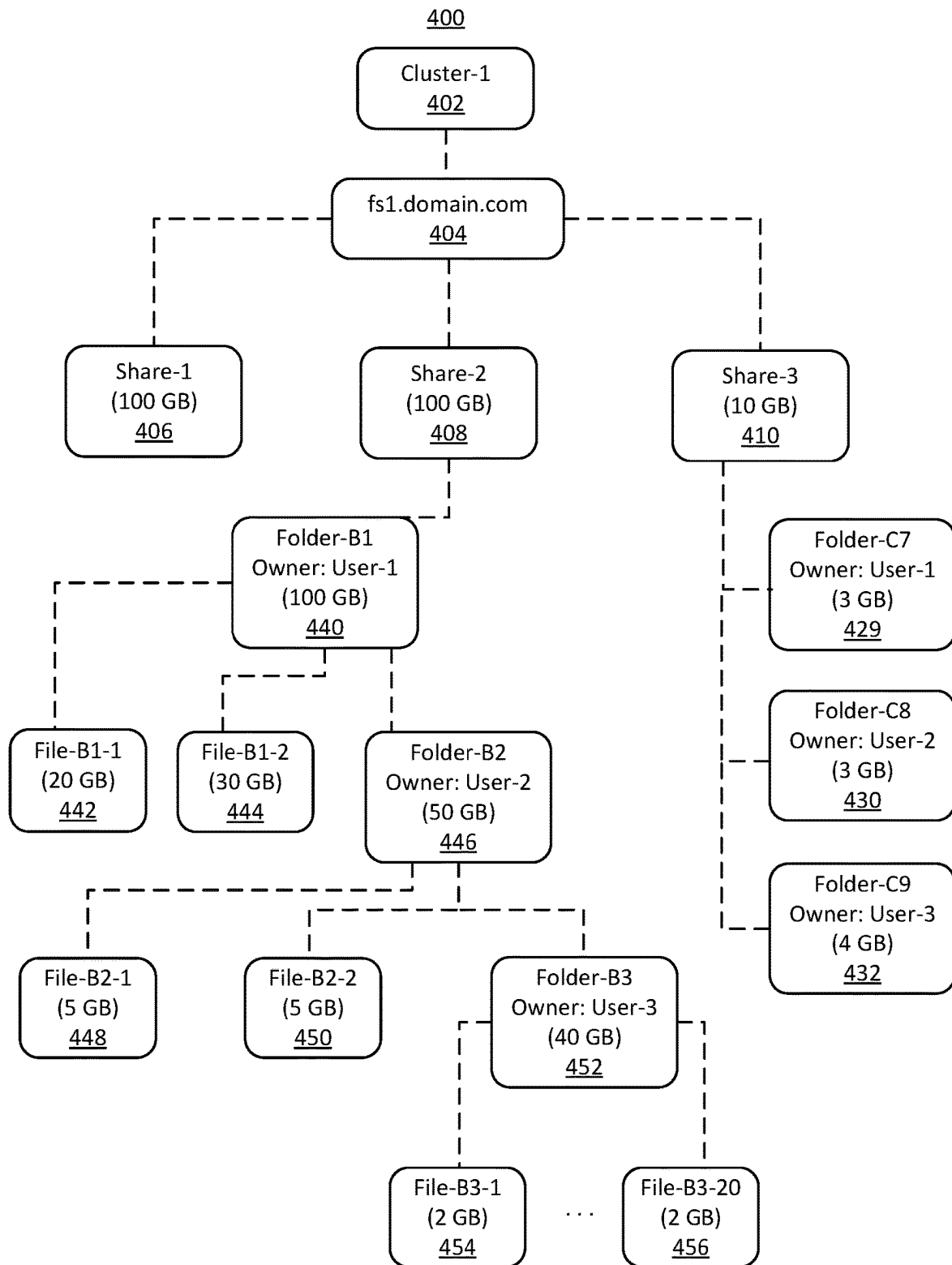


FIG. 4B

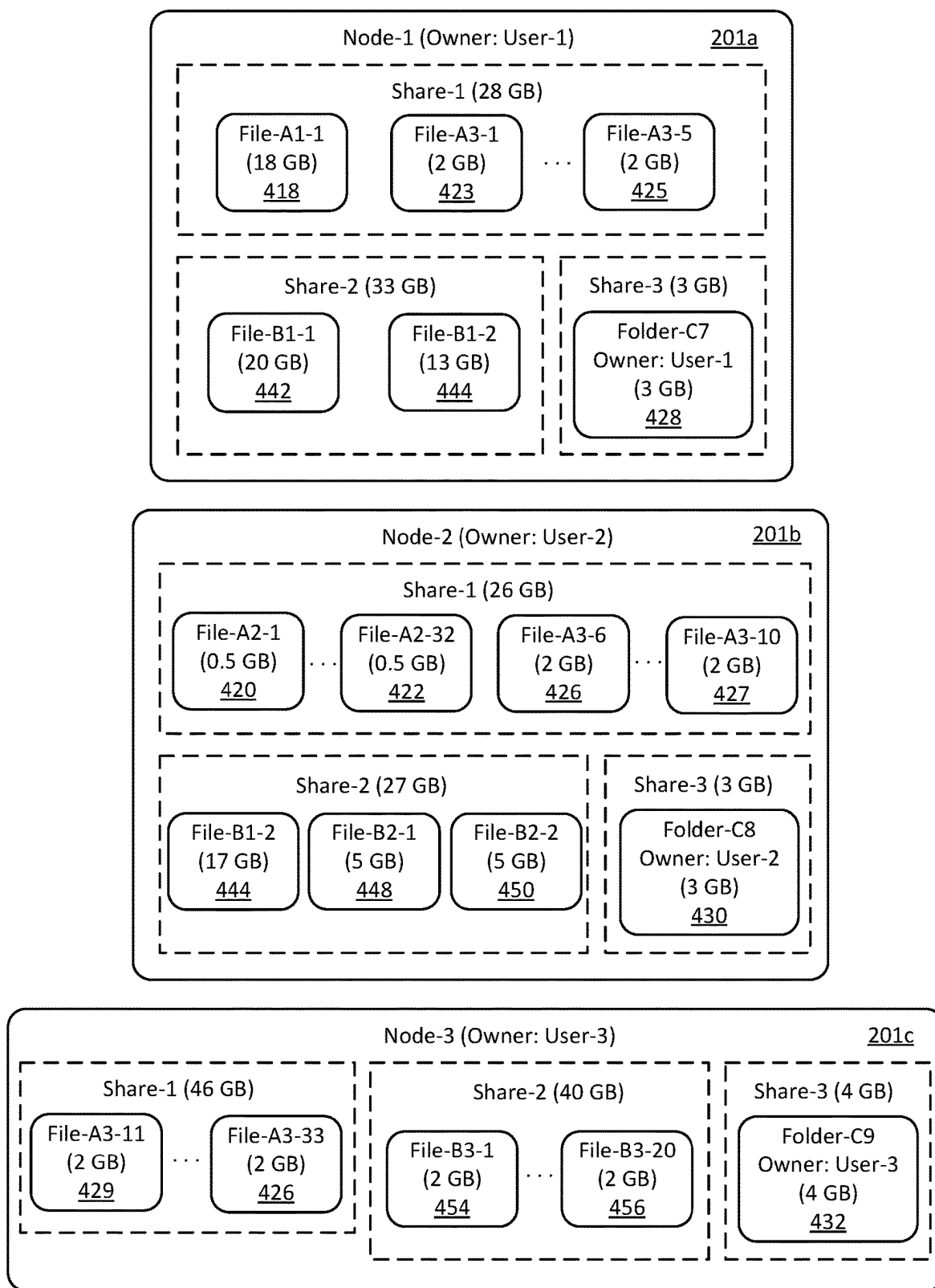


FIG. 4C

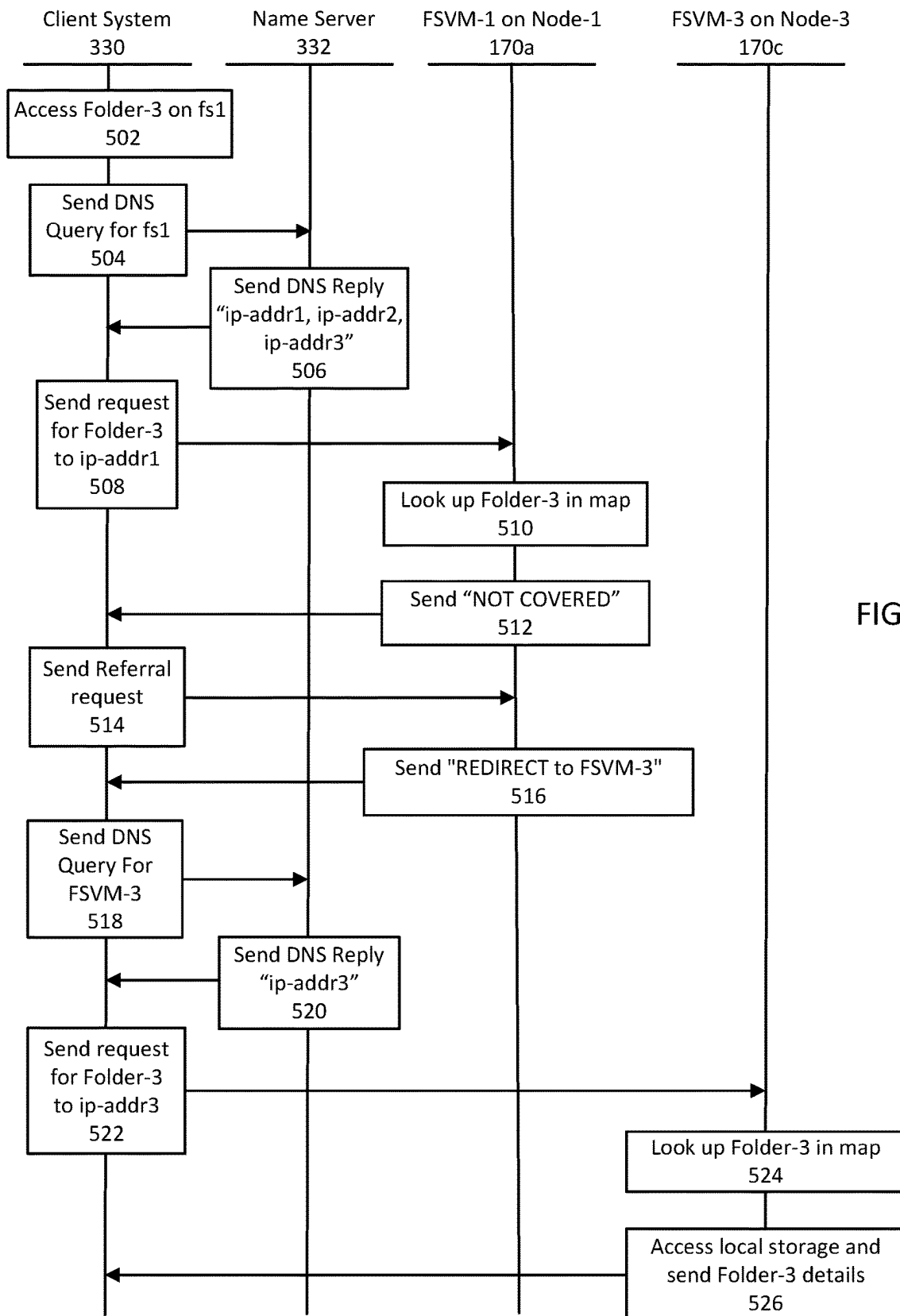


FIG. 5

FIG. 6

Fault Tolerant inter-cluster sharding of a single share 'share1'
AND Cluster/Site/Location aware quotas within the share

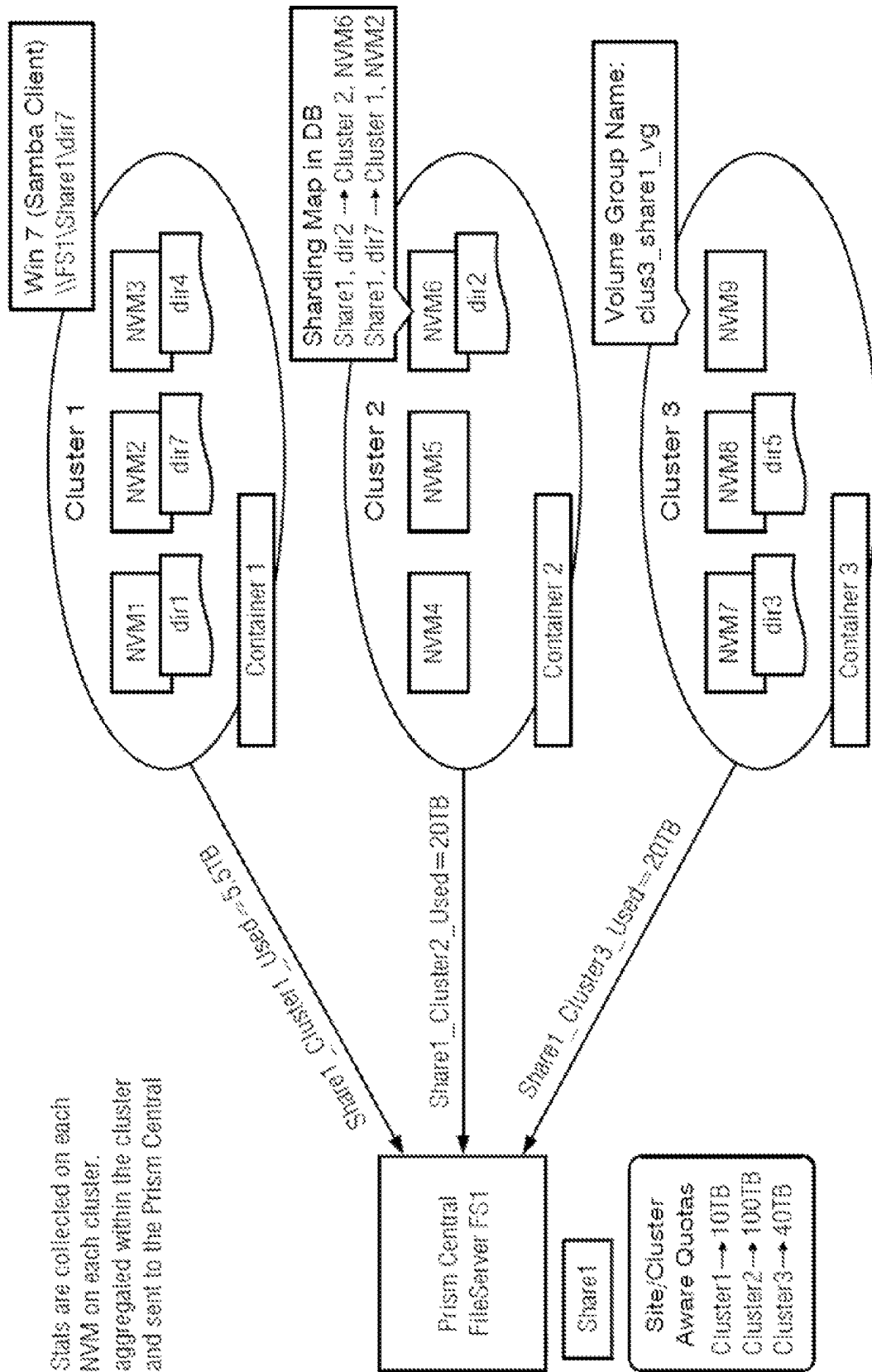
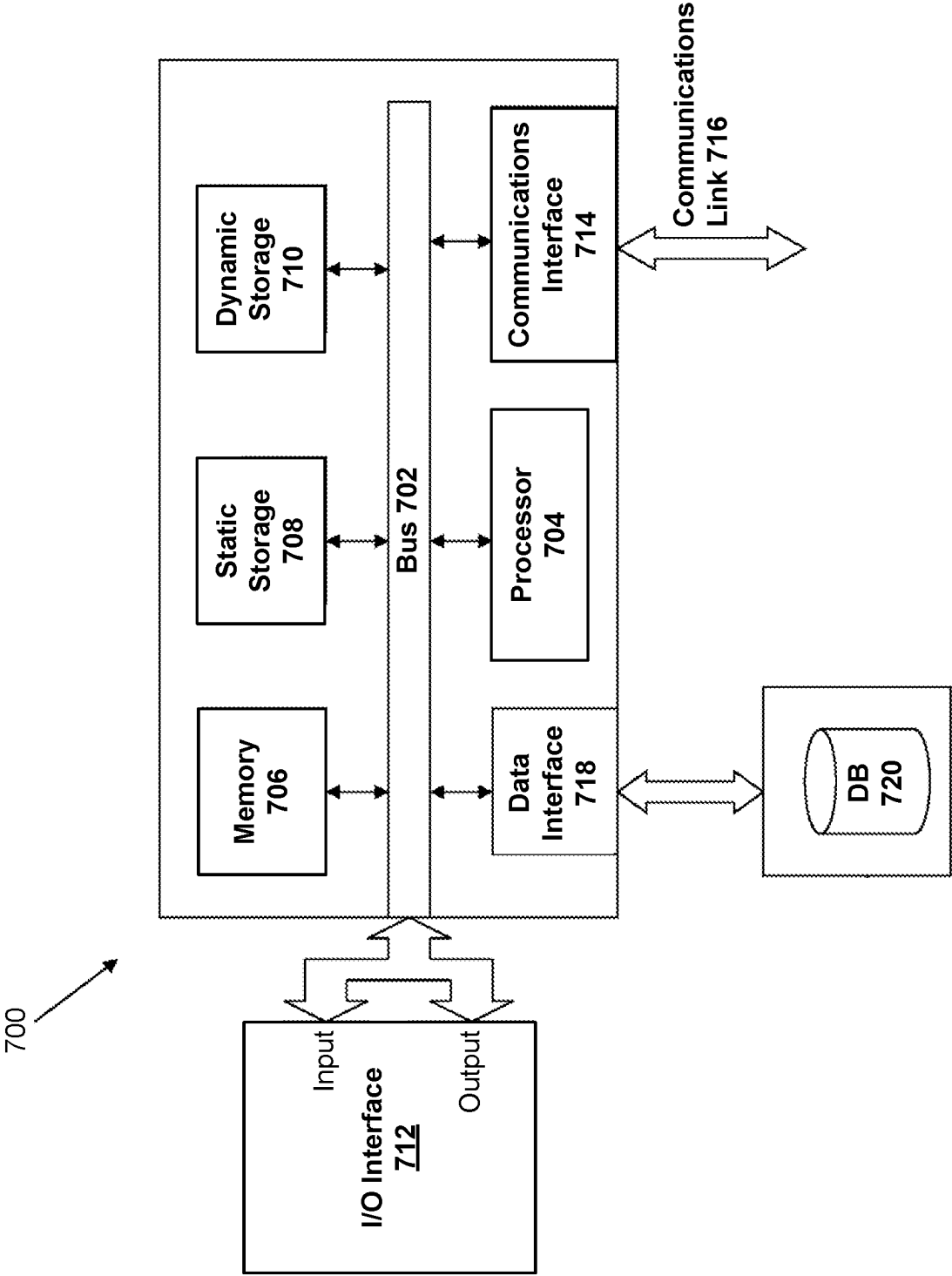


FIG. 7



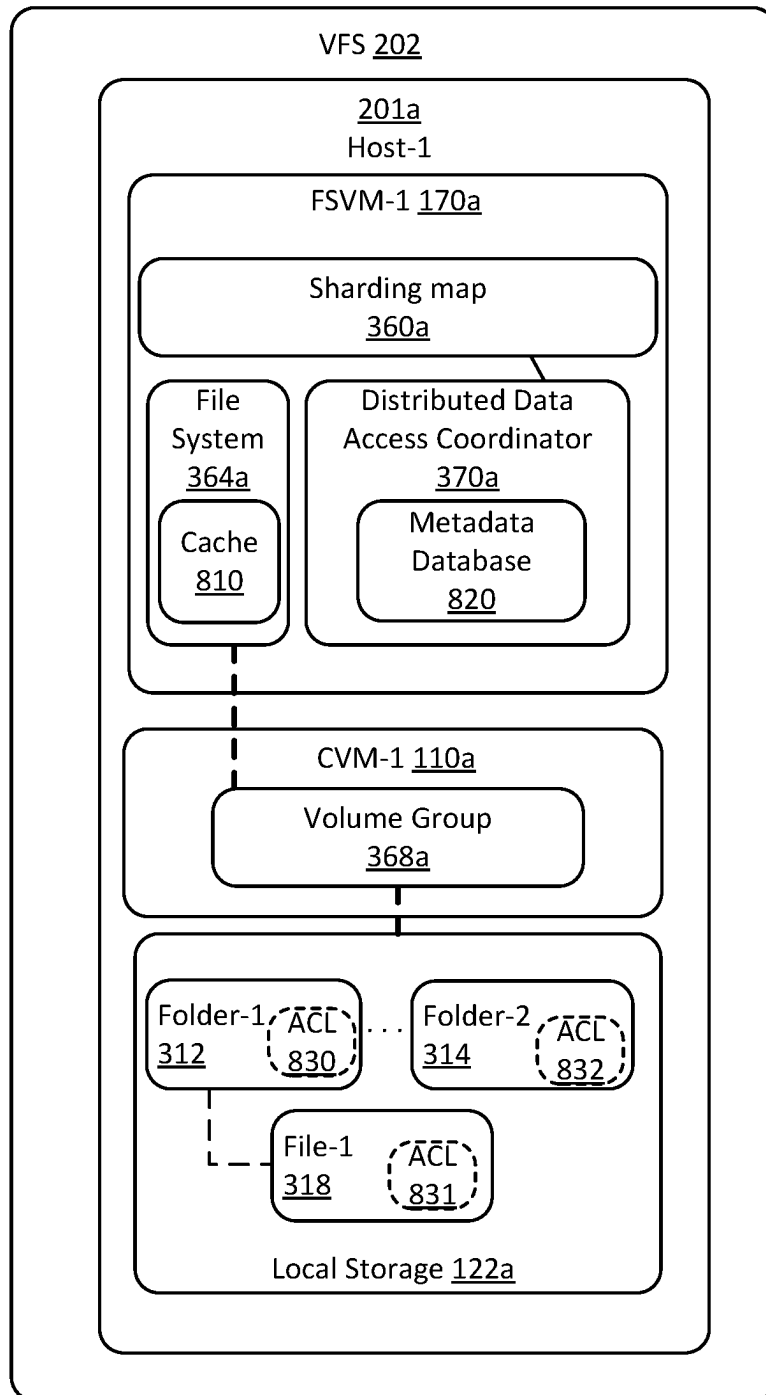
800

FIG. 8

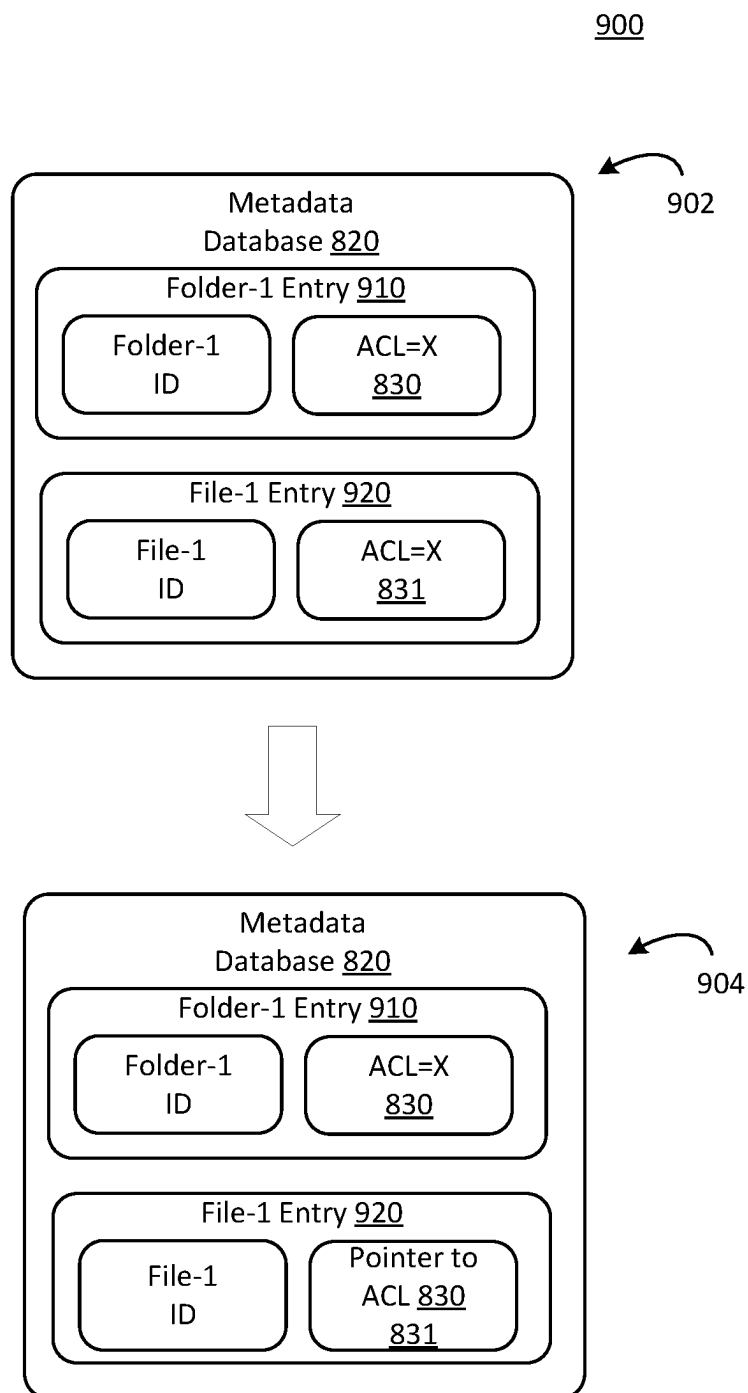


FIG. 9

0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

FIG. 10

0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
Directory_Access_Mask																					

FIG. 11

1

**HANDLING PERMISSIONS FOR
VIRTUALIZED FILE SERVERS****CROSS REFERENCE TO RELATED
APPLICATION(S)**

This application is a continuation of U.S. application Ser. No. 15/829,602 filed Dec. 1, 2017, issued as U.S. Pat. No. 11,568,073 on Jan. 31, 2023, which claims the benefit under 35 U.S.C. 119 of the earlier filing date of U.S. Provisional Application No. 62/429,283 entitled “HANDLING PERMISSIONS FOR VIRTUALIZED FILE SERVERS”, filed Dec. 2, 2016 and claims the benefit under 35 U.S.C. 119 of the earlier filing date of U.S. Provisional Application No. 62/430,264 entitled “EFFICIENT METADATA ACCESS”, filed Dec. 5, 2016. The aforementioned applications are hereby incorporated by reference, in their entirety, for any purpose.

TECHNICAL FIELD

Examples described herein pertain to distributed and cloud computing systems. Examples of distributed virtual file server systems are described.

BACKGROUND

A “virtual machine” or a “VM” refers to a specific software-based implementation of a machine in a virtualization environment, in which the hardware resources of a real computer (e.g., CPU, memory, etc.) are virtualized or transformed into the underlying support for the fully functional virtual machine that can run its own operating system and applications on the underlying physical resources just like a real computer.

Virtualization generally works by inserting a thin layer of software directly on the computer hardware or on a host operating system. This layer of software contains a virtual machine monitor or “hypervisor” that allocates hardware resources dynamically and transparently. Multiple operating systems run concurrently on a single physical computer and share hardware resources with each other. By encapsulating an entire machine, including CPU, memory, operating system, and network devices, a virtual machine is completely compatible with most standard operating systems, applications, and device drivers. Most modern implementations allow several operating systems and applications to safely run at the same time on a single computer, with each having access to the resources it needs when it needs them.

Virtualization allows one to run multiple virtual machines on a single physical machine, with each virtual machine sharing the resources of that one physical computer across multiple environments. Different virtual machines can run different operating systems and multiple applications on the same physical computer.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1A illustrates a clustered virtualization environment according to some particular embodiments.

FIG. 1B illustrates data flow within an example clustered virtualization environment according to particular embodiments.

FIG. 2A illustrates a clustered virtualization environment implementing a virtualized file server (VFS) 202 according to particular embodiments.

2

FIG. 2B illustrates data flow within a clustered virtualization environment implementing a VFS instance 202 in which stored items such as files and folders used by user VMs are stored locally on the same host machines as the user VMs according to particular embodiments.

FIG. 3A illustrates an example hierarchical structure of a VFS instance in a cluster according to particular embodiments.

FIG. 3B illustrates two example host machines, each providing file storage services for portions of two VFS instances FS1 and FS2 according to particular embodiments.

FIG. 3C illustrates example interactions between a client and host machines on which different portions of a VFS instance are stored according to particular embodiments.

FIG. 3D illustrates an example virtualized file server having a failover capability according to particular embodiments.

FIG. 3E illustrates an example virtualized file server that has recovered from a failure of Controller/Service VM CVM by switching to an alternate Controller/Service VM CVM according to particular embodiments.

FIG. 3F illustrates an example virtualized file server that has recovered from failure of a FSVM by electing a new leader FSVM according to particular embodiments.

FIGS. 3G and 3H illustrate example virtualized file servers that have recovered from failure of a host machine by switching to another Controller/Service VM and another FSVM according to particular embodiments.

FIGS. 4A and 4B illustrate an example hierarchical namespace of a file server according to particular embodiments.

FIG. 4C illustrates distribution of stored data amongst host machines in a virtualized file server according to particular embodiments.

FIG. 5 illustrates an example method for accessing data in a virtualized file server according to particular embodiments.

FIG. 6 illustrates an example of how a file server ‘FS1’ may be deployed across multiple clusters according to particular embodiments.

FIG. 7 is metadata database management for a distributed file system according to particular embodiments.

FIG. 8 illustrates a block diagram showing a distributed file system according to particular embodiments.

FIG. 9 depicts an operation of a metadata database according to particular embodiments.

FIG. 10 is a depiction of a bitmask.

FIG. 11 is a depiction of a bitmask.

DETAILED DESCRIPTION

One reason for the broad adoption of virtualization in modern business and computing environments is because of the resource utilization advantages provided by virtual machines. Without virtualization, if a physical machine is limited to a single dedicated operating system, then during periods of inactivity by the dedicated operating system the physical machine is not utilized to perform useful work. This is wasteful and inefficient if there are users on other physical machines which are currently waiting for computing resources. To address this problem, virtualization allows multiple VMs to share the underlying physical resources so that during periods of inactivity by one VM, other VMs can take advantage of the resource availability to process workloads. This can produce great efficiencies for the utilization of physical devices, and can result in reduced redundancies and better resource cost management.

Furthermore, there are now products that can aggregate multiple physical machines, running virtualization environments to not only utilize the processing power of the physical devices to aggregate the storage of the individual physical devices to create a logical storage pool wherein the data may be distributed across the physical devices but appears to the virtual machines to be part of the system that the virtual machine is hosted on. Such systems operate under the covers by using metadata, which may be distributed and replicated any number of times across the system, to locate the indicated data. These systems are commonly referred to as clustered systems, wherein the resources of the group are pooled to provide logically combined, but physically separate systems.

Particular embodiments provide an architecture for implementing virtualized file servers in a virtualization environment. In particular embodiments, a virtualized file server may include a set of File Server Virtual Machines (VMs) that execute on host machines and process storage access operations requested by user VMs executing on the host machines. The file server VMs may communicate with storage controllers provided by Controller/Service VMs executing on the host machines to store and retrieve storage items, such as files and folders, on storage devices associated with the host machines. The storage items may be distributed amongst multiple host machines. The file server VMs may maintain a storage map, such as a sharding map, that maps names or identifiers of storage items, such as folders, files, or portions thereof, to their locations. When a user application executing in a user VM on one of the host machines initiates a storage access operation, such as reading or writing data from or to a storage item or modifying metadata associated with the storage item, the user VM may send the storage access operation in a request to one of the file server VMs on one of the host machines. In particular embodiments, a file server VM executing on a host machine that receives a storage access request may use the storage map to determine whether the requested storage item is located on the host machine (or otherwise associated with the file server VM or Controller/Service VM on the host machine). If so, the file server VM executes the requested operation. Otherwise, the file server VM responds to the request with an indication that the requested storage item is not on the host machine, and may redirect the requesting user VM to the host machine on which the storage map indicates the storage item is located. The client may cache the address of the host machine on which the storage item is located, so that the client may send subsequent requests for the storage item directly to that host machine.

In particular embodiments, the virtualized file server determines the location, e.g., host machine, at which to store a storage item such as a file or folder when the storage item is created. A file server VM may attempt to create a file or folder using a Controller/Service VM on the same host machine as the user VM that requested creation of the file, so that the Controller/Service VM that controls access operations to the storage item is co-located with the requesting user VM. In this way, file access operations between the user VM that is known to be associated with the storage item and is thus likely to access the storage item again (e.g., in the near future and/or on behalf of the same user) may use local communication or short-distance communication to improve performance, e.g., by reducing access times or increasing access throughput. Further, the virtualized file server may also attempt to store the storage item on a storage device that is local to the Controller/Service VM being used to create the storage item, so that storage access operations between

the Controller/Service VM and the storage device may use local or short-distance communication.

Further details of aspects, objects, and advantages of the invention are described below in the detailed description, drawings, and claims. Both the foregoing general description and the following detailed description are exemplary and explanatory, and are not intended to be limiting as to the scope of the invention. Particular embodiments may include all, some, or none of the components, elements, features, functions, operations, or steps of the embodiments disclosed above. The subject matter which can be claimed comprises not only the combinations of features as set out in the attached claims but also any other combination of features in the claims, wherein each feature mentioned in the claims can be combined with any other feature or combination of other features in the claims. Furthermore, any of the embodiments and features described or depicted herein can be claimed in a separate claim and/or in any combination with any embodiment or feature described or depicted herein or with any of the features of the attached claims.

FIG. 1A illustrates a clustered virtualization environment according to some particular embodiments. The architecture of FIG. 1A can be implemented for a distributed platform that contains multiple host machines **100a-c** that manage multiple tiers of storage. The multiple tiers of storage may include network-attached storage (NAS) that is accessible through network **140**, such as, by way of example and not limitation, cloud storage **126**, which may be accessible through the Internet, or local network-accessible storage **128** (e.g., a storage area network (SAN)). Unlike the prior art, the present embodiment also permits local storage **122** that is within or directly attached to the server and/or appliance to be managed as part of storage pool **160**. Examples of such storage include Solid State Drives **125** (henceforth “SSDs”), Hard Disk Drives **127** (henceforth “HDDs” or “spindle drives”), optical disk drives, external drives (e.g., a storage device connected to a host machine via a native drive interface or a direct attach serial interface), or any other directly attached storage. These collected storage devices, both local and networked, form storage pool **160**. Virtual disks (or “vDisks”) can be structured from the storage devices in storage pool **160**, as described in more detail below. As used herein, the term vDisk refers to the storage abstraction that is exposed by a Controller/Service VM (CVM) to be used by a user VM. In some embodiments, the vDisk is exposed via iSCSI (“internet small computer system interface”) or NFS (“network file system”) and is mounted as a virtual disk on the user VM.

Each host machine **100a-c** runs virtualization software, such as VMWARE ESX(I), MICROSOFT HYPER-V, or REDHAT KVM. The virtualization software includes hypervisor **130a-c** to manage the interactions between the underlying hardware and the one or more user VMs **101a, 102a, 101b, 102b, 101c, and 102c** that run client software. Though not depicted in FIG. 1A, a hypervisor may connect to network **140**. In particular embodiments, a host machine **100** may be a physical hardware computing device; in particular embodiments, a host machine **100** may be a virtual machine.

CVMs **110a-c** are used to manage storage and input/output (“I/O”) activities according to particular embodiments. These special VMs act as the storage controller in the currently described architecture. Multiple such storage controllers may coordinate within a cluster to form a unified storage controller system. CVMs **110** may run as virtual machines on the various host machines **100**, and work together to form a distributed system **110** that manages all the storage resources, including local storage **122**, net-

worked storage **128**, and cloud storage **126**. The CVMs may connect to network **140** directly, or via a hypervisor. Since the CVMs run independent of hypervisors **130a-c**, this means that the current approach can be used and implemented within any virtual machine architecture, since the CVMs can be used in conjunction with any hypervisor from any virtualization vendor.

A host machine may be designated as a leader node within a cluster of host machines. For example, host machine **100b**, as indicated by the asterisks, may be a leader node. A leader node may have a software component designated to perform operations of the leader. For example, CVM **110b** on host machine **100b** may be designated to perform such operations. A leader may be responsible for monitoring or handling requests from other host machines or software components on other host machines throughout the virtualized environment. If a leader fails, a new leader may be designated. In particular embodiments, a management module (e.g., in the form of an agent) may be running on the leader node.

Each CVM **110a-c** exports one or more block devices or NFS server targets that appear as disks to user VMs **101** and **102**. These disks are virtual, since they are implemented by the software running inside CVMs **110a-c**. Thus, to user VMs **101** and **102**, CVMs **110a-c** appear to be exporting a clustered storage appliance that contains some disks. All user data (including the operating system) in the user VMs **101** and **102** reside on these virtual disks.

Significant performance advantages can be gained by allowing the virtualization system to access and utilize local storage **122** as disclosed herein. This is because I/O performance is typically much faster when performing access to local storage **122** as compared to performing access to networked storage **128** across a network **140**. This faster performance for locally attached storage **122** can be increased even further by using certain types of optimized local storage devices, such as SSDs. Further details regarding methods and mechanisms for implementing the virtualization environment illustrated in FIG. 1A are described in U.S. Pat. No. 8,601,473, which is hereby incorporated by reference in its entirety.

FIG. 1B illustrates data flow within an example clustered virtualization environment according to particular embodiments. As described above, one or more user VMs and a CVM may run on each host machine **100** along with a hypervisor. As a user VM performs I/O operations (e.g., a read operation or a write operation), the I/O commands of the user VM may be sent to the hypervisor that shares the same server as the user VM. For example, the hypervisor may present to the virtual machines an emulated storage controller, receive an I/O command and facilitate the performance of the I/O command (e.g., via interfacing with storage that is the object of the command, or passing the command to a service that will perform the I/O command). An emulated storage controller may facilitate I/O operations between a user VM and a vDisk. A vDisk may present to a user VM as one or more discrete storage drives, but each vDisk may correspond to any part of one or more drives within storage pool **160**. Additionally or alternatively, Controller/Service VM **110a-c** may present an emulated storage controller either to the hypervisor or to user VMs to facilitate I/O operations. CVMs **110a-c** may be connected to storage within storage pool **160**. CVM **110a** may have the ability to perform I/O operations using local storage **122a** within the same host machine **100a**, by connecting via network **140** to cloud storage **126** or networked storage **128**, or by connecting via network **140** to local storage **122b-c** within another

host machine **100b-c** (e.g., via connecting to another CVM **110b** or **110c**). In particular embodiments, any suitable computing system **700** may be used to implement a host machine **100**.

5 File System Architecture

FIG. 2A illustrates a clustered virtualization environment implementing a virtualized file server (VFS) **202** according to particular embodiments. In particular embodiments, the VFS **202** provides file services to user virtual machines (user VMs) **101** and **102**. The file services may include storing and retrieving data persistently, reliably, and efficiently. The user virtual machines **101** and **102** may execute user processes, such as office applications or the like, on host machines **200a-c**. The stored data may be represented as a set of storage items, such as files organized in a hierarchical structure of folders (also known as directories), which can contain files and other folders.

In particular embodiments, the VFS **202** may include a set of File Server Virtual Machines (FSVMs) **170a-c** that execute on host machines **200a-c** and process storage item access operations requested by user VMs **200a-c** executing on the host machines **200a-c**. The FSVMs **170a-c** may communicate with storage controllers provided by CVMs **110a-c** executing on the host machines **200a-c** to store and retrieve files, folders, or other storage items on local storage **122a-c** associated with, e.g., local to, the host machines **200a-c**. The network protocol used for communication between user VMs **101** and **102**, FSVMs **170a-c**, and CVMs **110a-c** via the network **140** may be Internet Small Computer Systems Interface (iSCSI), Server Message Block (SMB), Network File System (NFS), pNFS (Parallel NFS), or another appropriate protocol.

For the purposes of VFS **202**, host machine **200c** may be designated as a leader node within a cluster of host machines. In this case, FSVM **170c** on host machine **100c** may be designated to perform such operations. A leader may be responsible for monitoring or handling requests from FSVMs on other host machines throughout the virtualized environment. If FSVM **170c** fails, a new leader may be designated for VFS **202**.

In particular embodiments, the user VMs **101** and **102** may send data to the VFS **202** using write requests, and may receive data from it using read requests. The read and write requests, and their associated parameters, data, and results, may be sent between a user VM **101a** and one or more file server VMs (FSVMs) **170a-c** located on the same host machine as the user VM **101a** or on different host machines from the user VM **101a**. The read and write requests may be sent between host machines **200a-c** via network **140**, e.g., using a network communication protocol such as iSCSI, CIFS, SMB, TCP, IP, or the like. When a read or write request is sent between two VMs located on the same one of the host machines **200a-c** (e.g., between the user VM **101a** and the FSVM **170a** located on the host machine **200a**), the request may be sent using local communication within the host machine **200a** instead of via the network **140**. As described above, such local communication may be substantially faster than communication via the network **140**. The local communication may be performed by, e.g., writing to and reading from shared memory accessible by the user VM **101a** and the FSVM **170a**, sending and receiving data via a local "loopback" network interface, local stream communication, or the like.

In particular embodiments, the storage items stored by the VFS **202**, such as files and folders, may be distributed amongst multiple host machines **200a-c**. In particular embodiments, when storage access requests are received

from the user VMs **101** and **102**, the VFS **202** identifies host machines **200a-c** at which requested storage items, e.g., folders, files, or portions thereof, are stored, and directs the user VMs **101** and **102** to the locations of the storage items. The FSVMs **170** may maintain a storage map, such as a sharding map **360** (shown in FIG. 3C), that maps names or identifiers of storage items to their corresponding locations. The storage map may be a distributed data structure of which copies are maintained at each FSVM **170a-c** and accessed using distributed locks or other storage item access operations. Alternatively, the storage map may be maintained by a leader node such as the host machine **200c**, and the other host machines **200a** and **200b** may send requests to query and update the storage map to the leader host machine **200c**. Other implementations of the storage map are possible using appropriate techniques to provide asynchronous data access to a shared resource by multiple readers and writers. The storage map may map names or identifiers of storage items in the form of text strings or numeric identifiers, such as folder names, files names, and/or identifiers of portions of folders or files (e.g., numeric start offset positions and counts in bytes or other units) to locations of the files, folders, or portions thereof. Locations may be represented as names of FSVMs **170a-c**, e.g., “FSVM-1”, as network addresses of host machines **200a-c** on which FSVMs **170a-c** are located (e.g., “ip-addr1” or 128.1.1.10), or as other types of location identifiers.

When a user application executing in a user VM **101a** on one of the host machines **200a** initiates a storage access operation, such as reading or writing data, the user VM **101a** may send the storage access operation in a request to one of the FSVMs **170a-c** on one of the host machines **200a-c**. A FSVM executing on a host machine **200b** that receives a storage access request may use the storage map to determine whether the requested file or folder is located on the host machine **200b** (or otherwise associated with the FSVM **170b** or Controller/Service VM **110b** on the host machine **200b**). If the requested file or folder is located on the host machine **200b** (or otherwise associated with a VM on it), the FSVM **170b** executes the requested storage access operation. Otherwise, the FSVM **170b** responds to the request with an indication that the data is not on the host machine **200b**, and may redirect the requesting user VM **101a** to the host machine **200c** on which the storage map indicates the file or folder is located. The client may cache the address of the host machine **200c** on which the file or folder is located, so that it may send subsequent requests for the file or folder directly to the host machine **200c**.

As an example and not by way of limitation, the location of a file or a folder may be pinned to a particular host machine **200a** by sending a file service operation that creates the file or folder to a CVM **110a** located on the particular host machine **200a**. The CVM **110a** subsequently processes file service commands for that file and sends corresponding storage access operations to storage devices associated with the file. The CVM **110a** may associate local storage **122a** with the file if there is sufficient free space on local storage **122a**. Alternatively, the CVM **110a** may associate a storage device located on another host machine **200b**, e.g., in local storage **122b**, with the file under certain conditions, e.g., if there is insufficient free space on the local storage **122a**, or if storage access operations between the CVM **110a** and the file are expected to be infrequent. Files and folders, or portions thereof, may also be stored on other storage devices, such as the network-attached storage (NAS) **128** or the cloud storage **126** of the storage pool **160**.

In particular embodiments, a name service **220**, such as that specified by the Domain Name System (DNS) Internet protocol, may communicate with the host machines **200a-c** via the network **140** and may store a database of domain name (e.g., host name) to IP address mappings. The name service **220** may be queried by the User VMs **101** to determine the IP address of a particular host machine **200a-c** given a name of the host machine, e.g., to determine the IP address of the host name ip-addr1 for the host machine **200a**. The name service **220** may be located on a separate server computer system or on one or more of the host machines **200**. The names and IP addresses of the host machines of the VFS instance **202**, e.g., the host machines **200**, may be stored in the name service **220** so that the user VMs **101** may determine the IP address of each of the host machines **200**. The name of each VFS instance **202**, e.g., FS1, FS2, or the like, may be stored in the name service **220** in association with a set of one or more names that contains the name(s) of the host machines **200** of the VFS instance **202**. For example, the file server instance name FS1.domain.com may be associated with the host names ip-addr1, ip-addr2, and ip-addr3 in the name service **220**, so that a query of the name service **220** for the server instance name “FS1” or “FS1.domain.com” returns the names ip-addr1, ip-addr2, and ip-addr3. Further, the name service **220** may return the names in a different order for each name lookup request, e.g., using round-robin ordering, so that the sequence of names (or addresses) returned by the name service for a file server instance name is a different permutation for each query until all the permutations have been returned in response to requests, at which point the permutation cycle starts again, e.g., with the first permutation. In this way, storage access requests from user VMs **101** may be balanced across the host machines **200**, since the user VMs **101** submit requests to the name service **220** for the address of the VFS instance **202** for storage items for which the user VMs **101** do not have a record or cache entry, as described below.

In particular embodiments, each FSVM **170** may have two IP addresses: an external IP address and an internal IP address. The external IP addresses may be used by SMB/CIFS clients, such as user VMs **101**, to connect to the FSVMs **170**. The external IP addresses may be stored in the name service **220**. The IP addresses ip-addr1, ip-addr2, and ip-addr3 described above are examples of external IP addresses. The internal IP addresses may be used for iSCSI communication to CVMs **110**, e.g., between the FSVMs **170** and the CVMs **110**, and for communication between the CVMs **110** and storage devices in the storage pool **160**. Other internal communications may be sent via the internal IP addresses as well, e.g., file server configuration information may be sent from the CVMs **110** to the FSVMs **170** using the internal IP addresses, and the CVMs **110** may get file server statistics from the FSVMs **170** via internal communication as needed.

Since the VFS **202** is provided by a distributed set of FSVMs **170a-c**, the user VMs **101** and **102** that access particular requested storage items, such as files or folders, do not necessarily know the locations of the requested storage items when the request is received. A distributed file system protocol, e.g., MICROSOFT DFS or the like, is therefore used, in which a user VM **101a** may request the addresses of FSVMs **170a-c** from a name service **220** (e.g., DNS). The name service may send one or more network addresses of FSVMs **170a-c** to the user VM **101a**, in an order that changes for each subsequent request. These network addresses are not necessarily the addresses of the FSVM

170b on which the storage item requested by the user VM **101a** is located, since the name service **220** does not necessarily have information about the mapping between storage items and FSVMs **170a-c**. Next, the user VM **170a** may send an access request to one of the network addresses provided by the name service, e.g., the address of FSVM **170b**. The FSVM **170b** may receive the access request and determine whether the storage item identified by the request is located on the FSVM **170b**. If so, the FSVM **170b** may process the request and send the results to the requesting user VM **101a**. However, if the identified storage item is located on a different FSVM **170c**, then the FSVM **170b** may redirect the user VM **101a** to the FSVM **170c** on which the requested storage item is located by sending a “redirect” response referencing FSVM **170c** to the user VM **101a**. The user VM **101a** may then send the access request to FSVM **170c**, which may perform the requested operation for the identified storage item.

A particular VFS **202**, including the items it stores, e.g., files and folders, may be referred to herein as a VFS “instance” **202** and may have an associated name, e.g., FS1, as described above. Although a VFS instance **202** may have multiple FSVMs distributed across different host machines **200**, with different files being stored on different host machines **200**, the VFS instance **202** may present a single name space to its clients such as the user VMs **101**. The single name space may include, for example, a set of named “shares” and each share may have an associated folder hierarchy in which files are stored. Storage items such as files and folders may have associated names and metadata such as permissions, access control information, size quota limits, file types, files sizes, and so on. As another example, the name space may be a single folder hierarchy, e.g., a single root directory that contains files and other folders. User VMs **101** may access the data stored on a distributed VFS instance **202** via storage access operations, such as operations to list folders and files in a specified folder, create a new file or folder, open an existing file for reading or writing, and read data from or write data to a file, as well as storage item manipulation operations to rename, delete, copy, or get details, such as metadata, of files or folders. Note that folders may also be referred to herein as “directories.”

In particular embodiments, storage items such as files and folders in a file server namespace may be accessed by clients such as user VMs **101** by name, e.g., “Folder-1\File-1” and “Folder-2\File-2” for two different files named File-1 and File-2 in the folders Folder-1 and Folder-2, respectively (where Folder-1 and Folder-2 are sub-folders of the root folder). Names that identify files in the namespace using folder names and file names may be referred to as “path names.” Client systems may access the storage items stored on the VFS instance **202** by specifying the file names or path names, e.g., the path name “Folder-1\File-1”, in storage access operations. If the storage items are stored on a share (e.g., a shared drive), then the share name may be used to access the storage items, e.g., via the path name “\Share-1\Folder-1\File-1” to access File-1 in folder Folder-1 on a share named Share-1.

In particular embodiments, although the VFS instance **202** may store different folders, files, or portions thereof at different locations, e.g., on different host machines **200**, the use of different host machines or other elements of storage pool **160** to store the folders and files may be hidden from the accessing clients. The share name is not necessarily a name of a location such as a host machine **200**. For example, the name Share-1 does not identify a particular host machine

200 on which storage items of the share are located. The share Share-1 may have portions of storage items stored on three host machines **200a-c**, but a user may simply access Share-1, e.g., by mapping Share-1 to a client computer, to gain access to the storage items on Share-1 as if they were located on the client computer. Names of storage items, such as file names and folder names, are similarly location-independent. Thus, although storage items, such as files and their containing folders and shares, may be stored at different locations, such as different host machines **200a-c**, the files may be accessed in a location-transparent manner by clients (such as the user VMs **101** and **102**). Thus, users at client systems need not specify or know the locations of each storage item being accessed. The VFS **202** may automatically map the file names, folder names, or full path names to the locations at which the storage items are stored. As an example and not by way of limitation, a storage item’s physical location may be specified by the name or address of the host machine **200a-c** on which the storage item is located, the name, address, or identity of the FSVM **170a-c** that provides access to the storage item on the host machine **200a-c** on which the storage item is located, the particular device (e.g., SSD or HDD) of the local storage **122a** (or other type of storage in storage pool **160**) on which the storage item is located, and the address on the device, e.g., disk block numbers. A storage item such as a file may be divided into multiple parts that may be located on different host machines **200a-c**, in which case access requests for a particular portion of the file may be automatically mapped to the location of the portion of the file based on the portion of the file being accessed (e.g., the offset from the beginning of the file and the number of bytes being accessed).

In particular embodiments, VFS **202** determines the location, e.g., particular host machine **200a-c**, at which to store a storage item when the storage item is created. For example, a FSVM **170a** may attempt to create a file or folder using a Controller/Service VM **110a** on the same host machine **200a** as the user VM **101a** that requested creation of the file, so that the Controller/Service VM **110a** that controls access operations to the file folder is co-located with the user VM **101a**. In this way, since the user VM **101a** is known to be associated with the file or folder and is thus likely to access the file again, e.g., in the near future or on behalf of the same user, access operations may use local communication or short-distance communication to improve performance, e.g., by reducing access times or increasing access throughput. If there is a local CVM **110a** on the same host machine as the FSVM **170a**, the FSVM **170a** may identify it and use it by default. If there is no local CVM **110a** on the same host machine as the FSVM **170a**, a delay may be incurred for communication between the FSVM **170a** and a CVM **110b** on a different host machine **200b**. Further, the VFS **202** may also attempt to store the file on a storage device that is local to the CVM **110a** being used to create the file, such as local storage **122a**, so that storage access operations between the CVM **110a** and local storage **122a** may use local or short-distance communication.

In particular embodiments, if a CVM **110a** is unable to store the storage item in local storage **122a**, e.g., because local storage **122a** does not have sufficient available free space, then the file may be stored in local storage **122b** of a different host machine **200b**. In this case, the stored file is not physically local to the host machine **200a**, but storage access operations for the file are performed by the locally-associated CVM **110a** and FSVM **170a**, and the CVM **110a** may communicate with local storage **122b** on the remote

11

host machine **200b** using a network file sharing protocol, e.g., iSCSI, SAMBA or the like.

In particular embodiments, if a virtual machine, such as a user VM **101a**, CVM **110a**, or FSVM **170a**, moves from a host machine **200a** to a destination host machine **200b**, e.g., because of resource availability changes, and data items such as files or folders associated with the VM are not locally accessible on the destination host machine **200b**, then data migration may be performed for the data items associated with the moved VM to migrate them to the new host machine **200b**, so that they are local to the moved VM on the new host machine **200b**. FSVMs **170** may detect removal and addition of CVMs **110** (as may occur, for example, when a CVM **110** fails or is shut down) via the iSCSI protocol or other technique, such as heartbeat messages. As another example, a FSVM **170** may determine that a particular file's location is to be changed, e.g., because a disk on which the file is stored is becoming full, because changing the file's location is likely to reduce network communication delays and therefore improve performance, or for other reasons. Upon determining that a file is to be moved, VFS **202** may change the location of the file by, for example, copying the file from its existing location(s), such as local storage **122a** of a host machine **200a**, to its new location(s), such as local storage **122b** of host machine **200b** (and to or from other host machines, such as local storage **122c** of host machine **200c** if appropriate), and deleting the file from its existing location(s). Write operations on the file may be blocked or queued while the file is being copied, so that the copy is consistent. The VFS **202** may also redirect storage access requests for the file from an FSVM **170a** at the file's existing location to a FSVM **170b** at the file's new location.

In particular embodiments, VFS **202** includes at least three File Server Virtual Machines (FSVMs) **170a-c** located on three respective host machines **200a-c**. To provide high-availability, there may be a maximum of one FSVM **170a** for a particular VFS instance **202** per host machine **200** in a cluster. If two FSVMs **170** are detected on a single host machine **200**, then one of the FSVMs **170** may be moved to another host machine automatically, or the user (e.g., system administrator) may be notified to move the FSVM **170** to another host machine. The user may move a FSVM **170** to another host machine using an administrative interface that provides commands for starting, stopping, and moving FSVMs **170** between host machines **200**.

In particular embodiments, two FSVMs **170** of different VFS instances **202** may reside on the same host machine **200a**. If the host machine **200a** fails, the FSVMs **170** on the host machine **200a** become unavailable, at least until the host machine **200a** recovers. Thus, if there is at most one FSVM **170** for each VFS instance **202** on each host machine **200a**, then at most one of the FSVMs **170** may be lost per VFS **202** per failed host machine **200**. As an example, if more than one FSVM **170** for a particular VFS instance **202** were to reside on a host machine **200a**, and the VFS instance **202** includes three host machines **200a-c** and three FSVMs **170**, then loss of one host machine would result in loss of two-thirds of the FSVMs **170** for the VFS instance **202**, which would be more disruptive and more difficult to recover from than loss of one-third of the FSVMs **170** for the VFS instance **202**.

In particular embodiments, users, such as system administrators or other users of the user VMs **101**, **102**, may expand the cluster of FSVMs **170** by adding additional FSVMs **170**. Each FSVM **170a** may be associated with at least one network address, such as an IP (Internet Protocol)

12

address of the host machine **200a** on which the FSVM **170a** resides. There may be multiple clusters, and all FSVMs of a particular VFS instance are ordinarily in the same cluster. The VFS instance **202** may be a member of a MICROSOFT ACTIVE DIRECTORY domain, which may provide authentication and other services such as name service **220**.

FIG. 2B illustrates data flow within a clustered virtualization environment implementing a VFS instance **202** in which stored items such as files and folders used by user VMs **101** are stored locally on the same host machines **200** as the user VMs **101** according to particular embodiments. As described above, one or more user VMs **101** and a Controller/Service VM **110** may run on each host machine **200** along with a hypervisor **130**. As a user VM **101** processes I/O commands (e.g., a read or write operation), the I/O commands may be sent to the hypervisor **130** on the same server or host machine **200** as the user VM **101**. For example, the hypervisor **130** may present to the user VMs **101** a VFS instance **202**, receive an I/O command, and facilitate the performance of the I/O command by passing the command to a FSVM **170** that performs the operation specified by the command. The VFS **202** may facilitate I/O operations between a user VM **101** and a virtualized file system. The virtualized file system may appear to the user VM **101** as a namespace of mappable shared drives or mountable network file systems of files and directories. The namespace of the virtualized file system may be implemented using storage devices in the local storage **122**, such as disks **204**, onto which the shared drives or network file systems, files, and folders, or portions thereof, may be distributed as determined by the FSVMs **170**. The VFS **202** may thus provide features disclosed herein, such as efficient use of the disks **204**, high availability, scalability, and others. The implementation of these features may be transparent to the user VMs **101**, **102**. The FSVMs **170** may present the storage capacity of the disks **204** of the host machines **200** as an efficient, highly-available, and scalable namespace in which the user VMs **101**, **102** may create and access shares, files, folders, and the like.

As an example, a network share may be presented to a user VM **101** as one or more discrete virtual disks, but each virtual disk may correspond to any part of one or more virtual or physical disks **204** within storage pool **160**. Additionally or alternatively, the FSVMs **170** may present a VFS **202** either to the hypervisor **130** or to user VMs **101** of a host machine **200** to facilitate I/O operations. The FSVMs **170** may access the local storage **122** via Controller/Service VMs **110**. As described above with reference to FIG. 1B, a Controller/Service VM **110a** may have the ability to perform I/O operations using local storage **122a** within the same host machine **200a** by connecting via the network **140** to cloud storage **126** or networked storage **128**, or by connecting via the network **140** to local storage **122b-c** within another host machine **200b-c** (e.g., by connecting to another Controller/Service VM **110b-c**).

In particular embodiments, each user VM **101** may access one or more virtual disk images **206** stored on one or more disks **204** of the local storage **122**, the cloud storage **126**, and/or the networked storage **128**. The virtual disk images **206** may contain data used by the user VMs **101**, such as operating system images, application software, and user data, e.g., user home folders and user profile folders. For example, FIG. 2B illustrates three virtual machine images **206a-c**. The virtual machine image **206a** may be a file named UserVM101a.vmdisk (or the like) stored on disk **204a** of local storage **122a** of host machine **200a**. The virtual machine image **206a** may store the contents of the user VM

13

101a's hard drive. The disk 204a on which the virtual machine image 206a is "local to" the user VM 101a on host machine 200a because the disk 204a is in local storage 122a of the host machine 200a on which the user VM 101a is located. Thus, the user VM 101a may use local (intra-host machine) communication to access the virtual machine image 206a more efficiently, e.g., with less latency and higher throughput, than would be the case if the virtual machine image 206a were stored on disk 204b of local storage 122b of a different host machine 200b, because inter-host machine communication across the network 140 would be used in the latter case. Local communication within a host machine 200a is described in further detail with reference to FIG. 4C. Similarly, a virtual machine image 206b, which may be a file named UserVM101b.vmdisk (or the like), is stored on disk 204b of local storage 122b of host machine 200b, and the image 206b is local to the user VM 101b located on host machine 200b. Thus, the user VM 101a may access the virtual machine image 206b more efficiently than the virtual machine image 206a on host machine 200a, for example. In another example, the CVM 110c may be located on the same host machine 200c as the user VM 101c that accesses a virtual machine image 206c (UserVM101c.vmdisk) of the user VM 101c, with the virtual machine image file 206c being stored on a different host machine 200b than the user VM 101c and the CVM 110c. In this example, communication between the user VM 101c and the CVM 110c may still be local, e.g., more efficient than communication between the user VM 101c and a CVM 110b on a different host machine 200b, but communication between the CVM 110c and the disk 204b on which the virtual machine image 206c is stored is via the network 140, as shown by the dashed lines between CVM 110c and the network 140 and between the network 140 and local storage 122b. The communication between CVM 110c and the disk 204b is not local, and thus may be less efficient than local communication such as may occur between the CVM 110c and a disk 204c in local storage 122c of host machine 200c. Further, a user VM 101c on host machine 200c may access data such as the virtual disk image 206c stored on a remote (e.g., non-local) disk 204b via network communication with a CVM 110b located on the remote host machine 200b. This case may occur if CVM 110c is not present on host machine 200c, e.g., because CVM 110c has failed, or if the FSV 170c has been configured to communicate with local storage 122b on host machine 200b via the CVM 110b on host machine 200b, e.g., to reduce computational load on host machine 200c.

In particular embodiments, since local communication is expected to be more efficient than remote communication, the FSVs 170 may store storage items, such as files or folders, e.g., the virtual disk images 206, on local storage 122 of the host machine 200 on which the user VM 101 that is expected to access the files is located. A user VM 101 may

14

be expected to access particular storage items if, for example, the storage items are associated with the user VM 101, such as by configuration information. For example, the virtual disk image 206a may be associated with the user VM 101a by configuration information of the user VM 101a. Storage items may also be associated with a user VM 101 via the identity of a user of the user VM 101. For example, files and folders owned by the same user ID as the user who is logged into the user VM 101a may be associated with the user VM 101a. If the storage items expected to be accessed by a user VM 101a are not stored on the same host machine 200a as the user VM 101a, e.g., because of insufficient available storage capacity in local storage 122a of the host machine 200a, or because the storage items are expected to be accessed to a greater degree (e.g., more frequently or by more users) by a user VM 101b on a different host machine 200b, then the user VM 101a may still communicate with a local CVM 110a to access the storage items located on the remote host machine 200b, and the local CVM 110a may communicate with local storage 122b on the remote host machine 200b to access the storage items located on the remote host machine 200b. If the user VM 101a on a host machine 200a does not or cannot use a local CVM 110a to access the storage items located on the remote host machine 200b, e.g., because the local CVM 110a has crashed or the user VM 101a has been configured to use a remote CVM 110b, then communication between the user VM 101a and local storage 122b on which the storage items are stored may be via a remote CVM 110b using the network 140, and the remote CVM 110b may access local storage 122b using local communication on host machine 200b. As another example, a user VM 101a on a host machine 200a may access storage items located on a disk 204c of local storage 122c on another host machine 200c via a CVM 110b on an intermediary host machine 200b using network communication between the host machines 200a and 200b and between the host machines 200b and 200c.

In particular embodiments, VFS 202 may provide access controls for accessing storage items. The access controls may be stored for each of the storage item may be specified in access control lists (ACLs) including permissions information for individual users and groups. An ACL may comprise a list of access control entries (ACE). Each ACE in an ACL identifies a user and specifies the access rights allowed, denied, or audited for that user. For example, in a Windows environment, each ACL may be on average 4 KB in size, but may grow up to 64 KB in size. By way of example and not limitation, as described in the specification for MICROSOFT SMB protocol versions 2 and 3 (v20160714, released Jul. 14, 2016), permissions information for accessing a file, pipe, or printer may be stored in the form of a 4-byte bitmask (a small set of Boolean values) as shown in FIG. 10.

Such a bitmask may enable specification of the following types of permissions:

FILE_READ_DATA 0x00000001	Indicates the right to read data from the file or named pipe
FILE_WRITE_DATA 0x00000002	Indicates the right to write data into the file or named pipe beyond the end of the file
FILE_APPEND_DATA 0x00000004	Indicates the right to append data into the file or named pipe
FILE_READ_EA 0x00000008	Indicates the right to read the extended attributes of the file or named pipe
FILE_WRITE_EA 0x00000010	Indicates the right to write or change the extended attributes of the file or named pipe

-continued

FILE_DELETE_CHILD 0x00000040	Indicates the right to delete entries within a directory
FILE_EXECUTE 0x00000020	Indicates the right to execute the file
FILE_READ_ATTRIBUTES 0x00000080	Indicates the right to read the attributes of the file
FILE_WRITE_ATTRIBUTES 0x00000100	Indicates the right to change attributes of the file
DELETE 0x00010000	Indicates the right to delete the file
READ_CONTROL 0x00020000	Indicates the right to read the security descriptor for the file or named pipe
WRITE_DAC 0x00040000	Indicates the right to change the discretionary access control list (DACL) in the security descriptor for the file or named pipe
WRITE_OWNER 0x00080000	Indicates the right to change the owner in the security descriptor for the file or named pipe
SYNCHRONIZE 0x00100000	SMB2 clients set this flag to any value. SMB2 servers should ignore this flag.
ACCESS_SYSTEM_SECURITY 0x01000000	Indicates the right to read or change the system access control list (SACL) in the security descriptor for the file or named pipe
MAXIMUM_ALLOWED 0x02000000	Indicates that the client is requesting an open to the file with the highest level of access the client has on this file
GENERIC_ALL 0x10000000	Indicates a request for all the access flags that are previously listed except MAXIMUM_ALLOWED and ACCESS_SYSTEM_SECURITY
GENERIC_EXECUTE 0x20000000	Indicates a request for the following combination of access flags listed above: FILE_READ_ATTRIBUTES FILE_EXECUTE SYNCHRONIZE READ_CONTROL.
GENERIC_WRITE 0x40000000	Indicates a request for the following combination of access flags listed above: FILE_WRITE_DATA FILE_APPEND_DATA FILE_WRITE_ATTRIBUTES FILE_WRITE_EA SYNCHRONIZE READ_CONTROL.
GENERIC_READ 0x80000000	Indicates a request for the following combination of access flags listed above: FILE_READ_DATA FILE_READ_ATTRIBUTES FILE_READ_EA SYNCHRONIZE READ_CONTROL.

Further by way of example and not limitation, as described in the specification for MICROSOFT SMB protocol versions 2 and 3 (v20160714, released Jul. 14, 2016), permissions information for accessing a directory may be stored in the form of a 4-byte bitmask as shown in FIG. 11. Such a bitmask may enable specification of the following types of permissions:

FILE_LIST_DIRECTORY 0x00000001	Indicates the right to enumerate contents of the directory
FILE_ADD_FILE 0x00000002	Indicates the right to create a file under the directory
FILE_ADD_SUBDIRECTORY 0x00000004	Indicates the right to add a sub-directory under the directory
FILE_READ_EA 0x00000008	Indicates the right to read the extended attributes of the directory
FILE_WRITE_EA 0x00000010	Indicates the right to write or change the extended attributes of the directory
FILE_TRAVERSE 0x00000020	Indicates the right to traverse this directory if the server enforces traversal checking
FILE_DELETE_CHILD 0x00000040	Indicates the right to delete the files and directories within this directory
FILE_READ_ATTRIBUTES 0x00000080	Indicates the right to read the attributes of the directory
FILE_WRITE_ATTRIBUTES 0x00000100	Indicates the right to change attributes of the directory
DELETE 0x00010000	Indicates the right to delete the directory
READ_CONTROL 0x00020000	Indicates the right to read the security descriptor for the directory
WRITE_DAC 0x00040000	Indicates the right to change the discretionary access control list (DACL) in the security descriptor for the directory

-continued

WRITE_OWNER 0x00080000	Indicates the right to change the owner in the security descriptor for the directory
SYNCHRONIZE 0x00100000	SMB2 clients set this flag to any value. SMB2 servers should ignore this flag.
ACCESS_SYSTEM_SECURITY 0x01000000	Indicates the right to read or change the system access control list (SACL) in the security descriptor for the directory
MAXIMUM_ALLOWED 0x02000000	Indicates that the client is requesting an open to the directory with the highest level of access the client has on this directory
GENERIC_ALL 0x10000000	Indicates a request for all the access flags that are previously listed except MAXIMUM_ALLOWED and ACCESS_SYSTEM_SECURITY
GENERIC_EXECUTE 0x20000000	Indicates a request for the following combination of access flags listed above: FILE_READ_ATTRIBUTES FILE_TRAVERSE SYNCHRONIZE READ_CONTROL.
GENERIC_WRITE 0x40000000	Indicates a request for the following combination of access flags listed above: FILE_ADD_FILE FILE_ADD_SUBDIRECTORY FILE_WRITE_ATTRIBUTES FILE_WRITE_EA SYNCHRONIZE READ_CONTROL.
GENERIC_READ 0x80000000	Indicates a request for the following combination of access flags listed above: FILE_LIST_DIRECTORY FILE_READ_ATTRIBUTES FILE_READ_EA SYNCHRONIZE READ_CONTROL.

When the ACL for the storage item is large in size (e.g., as may be the case where there are many entries listed for different users/groups and/or many different types of permissions that may be specified), retrieving the permissions information may be an expensive operation. If the permissions information must be accessed with each and every storage access operation, the cost may become considerable. However, by performing a one-time extraction of all the permissions information available for a given user with respect to the storage item, and then caching such permissions information in the form of a bitmask, upon which standard bitmask operations may be performed, the cost of checking permissions for subsequent requests by the same user for storage access operations upon the storage item may be substantially reduced. For example, if a user were to execute a file containing a script (and, upon discovering an apparent bug in the script), retrieve the Last-Modified time-stamp for the file, display contents of the file, edit the file, and then execute the script again, such a series of storage access operations may involve at least five steps for which the user's permissions must be checked.

Particular embodiments may handle checking permissions information in two phases. In a first phase, upon receiving a request for a storage item from a particular user, VFS 202 may fetch all of the permissions information, walk through the permissions information to discover user and group entries relevant to the particular user, and cache a permissions profile containing all relevant permissions granted to the particular user for the requested file (e.g., cache the 4-byte bitmask for the user). In a second phase, as the VFS 202 continues to field requests from the particular user with respect to the requested storage item, VFS 202 may simply check the cached information (e.g., by performing a bitmask comparison between the permissions required to perform the storage access operation and all permissions granted to the user) during a specified period of time (e.g., two seconds), rather than retrieving and reading through all of the permissions information for the storage item with each subsequent request from the particular user. Latency for storage access operations may thereby be reduced with each request for a storage access operation as received from a user, by (1) avoiding the task of fetching the entire ACL each

time, and (2) replacing the task of reading the entire ACL each time with the smaller task of simply reading the cached permissions profile for the user.

In particular embodiments, a system (VFS 202) for managing data access using a virtualized file server may comprise (1) a plurality of host machines implementing a virtualization environment, wherein each of the host machines comprises a hypervisor and at least one user virtual machine (user VM); and (2) a virtualized file server comprising a plurality of file server virtual machines (FSVMs) and a storage pool, wherein each of the FSVMs is running on one of the host machines, wherein the FSVMs conduct I/O transactions with the storage pool. One of the user VMs may send, to one of the FSVMs, a request to perform a storage access operation on a storage item for an identified user, wherein the I/O request complies with a protocol for a distributed file server. The FSVM may then retrieve, from a cache, a user permissions profile for the storage item, wherein the user permissions profile consists of permissions information for the user with respect to the storage item. The FSVM may then determine whether the storage access operation is permissible based on the user permissions profile. Finally, the FSVM may send a response to the user VM with respect to the request.

Prior to the FSVM retrieving the user permissions profile for the storage item from the cache, the FSVM may determine that no cache entry exists for the user permissions profile for the storage item. The FSVM may retrieve access control information for the storage item, wherein the access control information comprises all permissions information for all users having any access rights to the storage item. The FSVM may then create a user permissions profile for the identified user based on all permissions information extracted from the access control information that is relevant to the identified user. Finally, the FSVM may create a cache entry for the user permissions profile.

In particular embodiments, wherein the user permissions profile is stored as a first bitmask representing all permissions information extracted from the access control information for the storage item and relevant to the identified user, the operation to determine whether the storage access operation is permissible based on the user permissions

profile comprises performing one or more bitmask operations to compare the first bitmask to one or more second bitmasks representing the permissions required to perform the storage access operation upon the storage item.

In particular embodiments, the cache entry may comprise a key-value pair, wherein (1) a key of the key-value pair comprises one or more pieces of identifying information comprising: a session ID associated with the identified user, an identifier associated with the identified user, a share name associated with the storage item, or a name of the storage item; and (2) a value of the key-value pair comprises the user permissions profile for the storage item.

In particular embodiments, rather than expiring after a specified period of time, the permissions profile may be cached on a per-session basis or on a persistent global basis (e.g., when permissions are not anticipated to change, or to change very rarely/infrequently).

In such embodiments, any changes to the access control information in relation to the user (e.g., if the access control information is updated to add or remove write permissions for the storage item) may trigger an event whereby the cache is immediately invalidated for that user.

In particular embodiments, the system may detect access patterns and cache clusters of permission information accordingly. For example, if a significant percentage of users having write access to a file are detected as attempting to access the file within the past 10 minutes, the system may retrieve and cache user permissions profiles for all users having write access to the file. In another example, if the system detects that a user has listing all files of a particular type within a directory, the system may retrieve and cache user permissions profiles for the user for all files of that type within the directory.

In particular embodiments, the specified period of time after which the cache expires may be tuned based on an anticipated frequency of changes to the permissions information.

In particular embodiments, metadata associated with storage items (e.g., inode or NTACL) may be stored on disk and then cached when a storage access operation referencing the storage item is received. Such metadata may include, by way of example and not limitation: file type, permissions, owner, group, file size, file access time, file modification time, file deletion time, number of (soft/hard) links to the file/directory, extended attributes, or an access control list (ACL). An ACL is a list of access control entries (ACE). Each ACE in an ACL identifies a user and specifies the access rights allowed, denied, or audited for that user. For example, in a Windows environment, each ACL may be on average 4 KB in size, but may grow up to 64 KB in size.

When a new file is created in a directory, it may inherit the ACL of the directory. In such cases, oftentimes, many or most of the files in a directory have the same ACL information. During a storage access operation where the ACLs of all files in a directory must be read (e.g., a request to list all files in a directory), a conventional file server may need to scan the ACLs of all files in the directory in order to determine whether or not to list each of the files. When many of the ACLs contain duplicate information, such an operation may cause far more latency than necessary.

For files whose ACLs are duplicates of their parent directory, particular embodiments may, at the time of file creation, store pointers to the block containing the ACL for the directory (rather than creating a separate ACL for the file). By creating such pointers, the cache may operate much more efficiently, since it reads the ACL for the parent directory first, and then registers a cache hit for such files.

For files having access permissions different than their parent directory, particular embodiments may, at the time of file creation, may create a separate ACL for such files.

FIG. 3A illustrates an example hierarchical structure 300 of a VFS instance in a cluster according to particular embodiments. A Cluster 302 contains two VFS instances, FS1 304 and FS2 306. Each VFS instance may be identified by a name such as "instance", e.g., "\FS1" for WINDOWS filesystems, or a name such as "instance", e.g., "FS1" for UNIX-type filesystems. The VFS instance FS1 304 contains shares, including Share-1 308 and Share-2 310. Shares may have names such as "Users" for a share that stores user home directories, or the like. Each share may have a path name such as \FS1\Share-1 or \FS1\Users. As an example and not by way of limitation, a share may correspond to a disk partition or a pool of filesystem blocks on WINDOWS and UNIX-type filesystems. As another example and not by way of limitation, a share may correspond to a folder or directory on a VFS instance 304. Shares may appear in the filesystem instance 202 as folders or directories to users of user VMs 101a. Share-1 308 includes two folders, Folder-1 312, and Folder-2 314, and may also include one or more files (e.g., files not in folders). Each folder 312, 314 may include one or more files 318. Share-2 310 includes a folder Folder-3 316, which includes a file File-2 320. Each folder has a folder name such as "Folder-1", "Users", or "Sam" and a path name such as "\FS1\Share-1\Folder-1" (WINDOWS) or "share-1/fs1/Users/Sam" (UNIX). Similarly, each file has a file name such as "File-1" or "Forecast.xls" and a path name such as "\FS1\Share-1\Folder-1\File-1" or "share-1:/fs1/Users/Sam/Forecast.xls".

FIG. 3A illustrates an example hierarchical structure 300 of a VFS instance in a cluster according to particular embodiments. A Cluster 302 contains two VFS instances, FS1 304 and FS2 306. Each VFS instance may be identified by a name such as "instance", e.g., "\FS1" for WINDOWS file systems, or a name such as "instance", e.g., "FS1" for UNIX-type file systems. The VFS instance FS1 304 contains shares, including Share-1 308 and Share-2 310. Shares may have names such as "Users" for a share that stores user home directories, or the like. Each share may have a path name such as \FS1\Share-1 or \FS1\Users. As an example and not by way of limitation, a share may correspond to a disk partition or a pool of file system blocks on WINDOWS and UNIX-type file systems. As another example and not by way of limitation, a share may correspond to a folder or directory on a VFS instance 304. Shares may appear in the file system instance 202 as folders or directories to users of user VMs 101a. Share-1 308 includes two folders, Folder-1 312, and Folder-2 314, and may also include one or more files (e.g., files not in folders). Each folder 312, 314 may include one or more files 318. Share-2 310 includes a folder Folder-3 316, which includes a file File-2 320. Each folder has a folder name such as "Folder-1", "Users", or "Sam" and a path name such as "\FS1\Share-1\Folder-1" (WINDOWS) or "share-1/fs1/Users/Sam" (UNIX). Similarly, each file has a file name such as "File-1" or "Forecast.xls" and a path name such as "\FS1\Share-1\Folder-1\File-1" or "share-1:/fs1/Users/Sam/Forecast.xls".

FIG. 3B illustrates two example host machines 200a and 200b, each providing file storage services for portions of two VFS instances FS1 and FS2 according to particular embodiments. The first host machine, Host-1 200a, includes two user VMs 101a, 102a, a Hypervisor 130a, a FSVM named FileServer-VM-1 (abbreviated FSVM-1) 170a, a Controller/Service VM named CVM-1 110a, and local storage 122a. Host-1's FileServer-VM-1 170a has an IP (Internet Proto-

col) network address of 10.1.1.1, which is an address of a network interface on Host-1 **200a**. Host-1 has a hostname ip-adrl, which may correspond to Host-1's IP address 10.1.1.1. The second host machine, Host-2 **200b**, includes two user VMs **101b**, **102b**, a Hypervisor **130b**, a File Server VM named FileServer-VM-2 (abbreviated FSVM-2) **170b**, a Controller/Service VM named CVM-2 **110b**, and local storage **122b**. Host-2's FileServer-VM-1 **170b** has an IP network address of 10.1.1.2, which is an address of a network interface on Host-2 **200b**.

In particular embodiments, file systems FileSystem-1A **364a** and FileSystem-2A **365a** implement the structure of files and folders for portions of the FS1 and FS2 file server instances, respectively, that are located on (e.g., served by) FileServer-VM-1 **170a** on Host-1 **200a**. Other file systems on other host machines may implement other portions of the FS1 and FS2 file server instances. The file systems **364a** and **365a** may implement the structure of at least a portion of a file server instance by translating file system operations, such as opening a file, writing data to or reading data from the file, deleting a file, and so on, to disk I/O operations such as seeking to a portion of the disk, reading or writing an index of file information, writing data to or reading data from blocks of the disk, allocating or de-allocating the blocks, and so on. The file systems **364a**, **365a** may thus store their file system data, including the structure of the folder and file hierarchy, the names of the storage items (e.g., folders and files), and the contents of the storage items on one or more storage devices, such as local storage **122a**. The particular storage device or devices on which the file system data for each file system are stored may be specified by an associated file system pool (e.g., **366a-c** and **367a-c**). For example, the storage device(s) on which data for FileSystem-1A **364a** and FileSystem-2A, **365a** are stored may be specified by respective file system pools FS1-Pool-1 **366a** and FS2-Pool-2 **367a**. The storage devices for the pool **366a** may be selected from volume groups provided by CVM-1 **110a**, such as volume group VG1 **368a** and volume group VG2 **369a**. Each volume group **368a**, **369a** may include a group of one or more available storage devices that are present in local storage **122a** associated with (e.g., by iSCSI communication) the CVM-1 **110a**. The CVM-1 **110a** may be associated with a local storage **122a** on the same host machine **200a** as the CVM-1 **110a**, or with a local storage **122b** on a different host machine **200b**. The CVM-1 **110a** may also be associated with other types of storage, such as cloud storage **126**, networked storage **128** or the like. Although the examples described herein include particular host machines, virtual machines, file servers, file server instances, file server pools, CVMs, volume groups, and associations there between, any number of host machines, virtual machines, file servers, file server instances, file server pools, CVMs, volume groups, and any associations there between are possible and contemplated.

In particular embodiments, the file system pool **366a** may associate any storage device in one of the volume groups **368a**, **369a** of storage devices that are available in local storage **122a** with the file system FileSystem-1A **364a**. For example, the file system pool FS1-Pool-1 **366a** may specify that a disk device named hd1 in the volume group VG1 **368a** of local storage **122a** is a storage device for FileSystem-1A **364a** for file server FS1 on FSVM-1 **170a**. A file system pool FS2-Pool-2 **367a** may specify a storage device FileSystem-2A **365a** for file server FS2 on FSVM-1 **170a**. The storage device for FileSystem-2A **365a** may be, e.g., the disk device hd1, or a different device in one of the volume groups **368a**, **369a**, such as a disk device named hd2 in volume group

VG2 **369a**. Each of the file systems FileSystem-1A **364a**, FileSystem-2A **365a** may be, e.g., an instance of the NTFS file system used by the WINDOWS operating system, of the UFS Unix file system, or the like. The term "file system" may also be used herein to refer to an instance of a type of file system, e.g., a particular structure of folders and files with particular names and content.

In one example, referring to FIG. 3A, an FS1 hierarchy rooted at File Server FS1 **304** may be located on FileServer-VM-1 **170a** and stored in file system instance FileSystem-1A **364a**. That is, the file system instance FileSystem-1A **364a** may store the names of the shares and storage items (such as folders and files), as well as the contents of the storage items, shown in the hierarchy at and below File Server FS1 **304**. A portion of the FS1 hierarchy shown in FIG. 3A, such the portion rooted at Folder-2 **314**, may be located on FileServer-VM-2-**170b** on Host-2 **200b** instead of FileServer-VM-1-**170a**, in which case the file system instance FileSystem-1B **364b** may store the portion of the FS1 hierarchy rooted at Folder-2 **314**, including Folder-3 **314**, Folder-4 **322** and File-3 **324**. Similarly, an FS2 hierarchy rooted at File Server FS2 **306** in FIG. 3A may be located on FileServer-VM-1 **170a** and stored in file system instance FileSystem-2A **365a**. The FS2 hierarchy may be split into multiple portions (not shown), such that one portion is located on FileServer-VM-1 **170a** on Host-1 **200a**, and another portion is located on FileServer-VM-2 **170b** on Host-2 **200b** and stored in file system instance FileSystem-2B **365c**.

In particular embodiments, FileServer-VM-1 (abbreviated FSVM-1) **170a** on Host-1 **200a** is a leader for a portion of file server instance FS1 and a portion of FS2, and is a backup for another portion of FS1 and another portion of FS2. The portion of FS1 for which FileServer-VM-1 **170a** is a leader corresponds to a storage pool labeled FS1-Pool-1 **366a**. FileServer-VM-1 is also a leader for FS2-Pool-2 **367a**, and is a backup (e.g., is prepared to become a leader upon request, such as in response to a failure of another FSVM) for FS1-Pool-3 **366b** and FS2-Pool-4 **367b** on Host-2. In particular embodiments, FileServer-VM-2 (abbreviated FSVM-2) **170b** is a leader for a portion of file server instance FS1 and a portion of FS2, and is a backup for another portion of FS1 and another portion of FS2. The portion of FS1 for which FSVM-2 **170b** is a leader corresponds to a storage pool labeled FS1-Pool-3 **366b**. FSVM-2 **170b** is also a leader for FS2-Pool-4 **367b**, and is a backup for FS1-Pool-1 **366a** and FS2-Pool-2 **367a** on Host-1.

In particular embodiments, the file server instances FS1, FS2 provided by the FSVMs **170a** and **170b** may be accessed by user VMs **101a** and **101b** via a network file system protocol such as SMB, CIFS, NFS, or the like. Each FSVM **170a** and **170b** may provide what appears to client applications on user VMs **101a** and **101b** to be a single file system instance, e.g., a single namespace of shares, files and folders, for each file server instance **202**. However, the shares, files, and folders in a file server instance such as FS1 may actually be distributed across multiple FSVMs **170a** and **170b**. For example, different folders in the same file server instance may be associated with different corresponding FSVMs **170a** and **170b** and CVMs **110a** and **110b** on different host machines **200a** and **200b**.

The example file server instance FS1 **304** shown in FIG. 3A has two shares, Share-1 **308** and Share-2 **310**. Share-1 **308** may be located on FSVM-1 **170a**, CVM-1 **110a**, and local storage **122a**. Network file system protocol requests from user VMs **101** and **102** to read or write data on file server instance FS1 **304** and any share, folder, or file in the

instance may be sent to FSVM-1 170a. FSVM-1 170a may determine whether the requested data, e.g., the share, folder, file, or a portion thereof, referenced in the request, is located on FSVM-1, and FSVM-1 is a leader for the requested data. If not, FSVM-1 may respond to the requesting User-VM with an indication that the requested data is not covered by (e.g., is not located on or served by) FSVM-1. Otherwise, the requested data is covered by (e.g., is located on or served by) FSVM-1, so FSVM-1 may send iSCSI protocol requests to a CVM that is associated with the requested data. Note that the CVM associated with the requested data may be the CVM-1 110a on the same host machine 200a as the FSVM-1, or a different CVM on a different host machine 200b, depending on the configuration of the VFS 202. In this example, the requested Share-1 is located on FSVM-1, so FSVM-1 processes the request. To provide for path availability, multipath I/O (MPIO) may be used for communication with the FSVM, e.g., for communication between FSVM-1 and CVM-1. The active path may be set to the CVM that is local to the FSVM (e.g., on the same host machine) by default. The active path may be set to a remote CVM instead of the local CVM, e.g., when a failover occurs.

Continuing with the data request example, the associated CVM is CVM 110a, which may in turn access the storage device associated with the requested data as specified in the request, e.g., to write specified data to the storage device or read requested data from a specified location on the storage device. In this example, the associated storage device is in local storage 122a, and may be an HDD or SSD. CVM-1 110a may access the HDD or SSD via an appropriate protocol, e.g., iSCSI, SCSI, SATA, or the like. CVM 110a may send the results of accessing local storage 122a, e.g., data that has been read, or the status of a data write operation, to CVM 110a via, e.g., SATA, which may in turn send the results to FSVM-1 170a via, e.g., iSCSI. FSVM-1 170a may then send the results to user VM 101a via SMB through the Hypervisor 130a.

Share-2 310 may be located on FSVM-2 170b, on Host-2. Network file service protocol requests from user VMs 101a and 101b to read or write data on Share-2 may be directed to FSVM-2 170b on Host-2 by other FSVMs 170a. Alternatively, user VMs 101a and 101b may send such requests directly to FSVM-2 170b on Host-2, which may process the requests using CVM-2 110b and local storage 122b on Host-2 as described above for FSVM-1 170a on Host-1.

A file server instance 202 such as FS1 304 in FIG. 3A may appear as a single file system instance (e.g., a single namespace of folders and files that are accessible by their names or pathnames without regard for their physical locations), even though portions of the file system are stored on different host machines 200a-c. Since each FSVM 170 may provide a portion of a file server instance 202, each FSVM 170 may have one or more "local" file systems 364a, 365a that provide the portion of the file server instance 202 (e.g., the portion of the namespace of files and folders) associated with the FSVM 170.

FIG. 3C illustrates example interactions between a client 330 and host machines 200a and 200c on which different portions of a VFS instance are stored according to particular embodiments. A client 330, e.g., an application program executing in one of the user VMs 101 and 102 on the host machines 200a-c of FIGS. 2A-2B (e.g. user VM 101b on host machine 200b) requests access to a folder \\FS1.domain.name\Share-1\Folder-3. The request may be in response to an attempt to map \\FS1.domain.name\Share-1 to a network drive in the operating system executing in the user VM 101c followed by an

attempt to access the contents of Share-1 or to access the contents of Folder-3, such as listing the files in Folder-3.

FIG. 3C shows interactions that occur between the client 330, FSVMs 170a and 170b on host machines 200a and 200b, and a name server 332 when a storage item is mapped or otherwise accessed. The name server 332 may be provided by a server computer system, such as one or more of the host machines 200, or a server computer system separate from the host machines 200. In one example, the name server 332 may be provided by an ACTIVE DIRECTORY service executing on one or more computer systems and accessible via the network 140. The interactions are shown as arrows that represent communications, e.g., messages sent via the network 140. Note that the client 330 may be executing in a user VM 101, which may be co-located with one of the FSVMs 170a and 170b. In such a co-located case, the arrows between the client 330 and the host machine 200 on which the FSVM 170 is located may represent communication within the host machine 200, and such intra-host machine communication may be performed using a mechanism different from communication over the network 140, e.g., shared memory or inter process communication.

In particular embodiments, when the client 330 requests access to Folder-3, a VFS client component executing in the user VM 101b may use a distributed file system protocol such as MICROSOFT DFS, or the like, to send the storage access request to one or more of the FSVMs 170a-c of FIGS. 2A-2B. To access the requested file or folder, the client determines the location of the requested file or folder, e.g., the identity and/or network address of the FSVM 170 on which the file or folder is located. The client may query a domain cache of FSVM 170a-c network addresses that the client has previously identified (e.g., looked up). If the domain cache contains the network address of an FSVM 170 associated with the requested folder name \\FS1.domain.name\Share-1\Folder-3, then the client retrieves the associated network address from the domain cache and sends the access request to the network address, starting at step 393 as described below.

In particular embodiments, at step 381, the client may send a request for a list of addresses of FSVMs 170a-170c to a name server 332. The name server 332 may be, e.g., a DNS server or other type of server, such as a MICROSOFT domain controller (not shown), that has a database of FSVM addresses. At step 382, the name server 332 may send a reply that contains a list of FSVM 170 network addresses, e.g., ip-addr1, ip-addr2, and ip-addr3, which correspond to the FSVMs 170a-c in this example. At step 383, the client 330 may send an access request to one of the network addresses, e.g., the first network address in the list (ip-addr1 in this example), requesting the contents of Folder-3 of Share-1. By selecting the first network address in the list, the particular FSVM 170 to which the access request is sent may be varied, e.g., in a round-robin manner by enabling round-robin DNS (or the like) on the name server 332. The access request may be, e.g., an SMB connect request, an NFS open request, and/or appropriate request(s) to traverse the hierarchy of Share-1 to reach the desired folder or file, e.g., Folder-3 in this example.

At step 384, FileServer-VM-1 170a may process the request received at step 383 by searching a mapping or lookup table, such as a sharding map 360a, for the desired folder or file. The map 360 maps stored objects, such as shares, folders, or files, to their corresponding locations, e.g., the names or addresses of FSVMs 170. The map 360 may have the same contents on each host machine 200, with the contents on different host machines being synchronized

using a distributed data store as described below. For example, the map **360a** may contain entries that map Share-1 and Folder-1 to the File Server FSVM-1 **170a**, and Folder-3 to the File Server FSVM-3 **170c**. An example map **360** is shown in Table 1 below.

TABLE 1

Stored Object	Location
Folder-1	FSVM-1
Folder-2	FSVM-1
File-1	FSVM-1
Folder-3	FSVM-3
File-2	FSVM-3

In particular embodiments, the map **360** may be accessible on each of the host machines **200**. As described with reference to FIGS. 2A-2B, the maps **360a** and **360c** may be copies of a distributed data structure that are maintained and accessed at each FSVM **170a-c** using a distributed data access coordinator **370a** and **370c**. The distributed data access coordinator **370a** and **370c** may be implemented based on distributed locks or other storage item access operations. Alternatively, the distributed data access coordinator **370a** and **370c** may be implemented by maintaining a master copy of the maps **360a** and **360c** at a leader node such as the host machine **200c**, and using distributed locks to access the master copy from each FSVM **170a** and **170b**. The distributed data access coordinator **370a** and **370c** may be implemented using distributed locking, leader election, or related features provided by a centralized coordination service for maintaining configuration information, naming, providing distributed synchronization, and/or providing group services (e.g., APACHE ZOOKEEPER or other distributed coordination software). Since the map **360a** indicates that Folder-3 is located at FSVM-3 **170c** on Host-3 **200c**, the lookup operation at step **384** determines that Folder-3 is not located at FSVM-1 on Host-1 **200a**. Thus, at step **385** the FSVM-1 **170a** sends a response, e.g., a “Not Covered” DFS response, to the client **330** indicating that the requested folder is not located at FSVM-1. At step **386**, the client **330** sends a request to FSVM-1 for a referral to the FSVM on which Folder-3 is located. FSVM-1 uses the map **360a** to determine that Folder-3 is located at FSVM-3 on Host-3 **200c**, and at step **387** returns a response, e.g., a “Redirect” DFS response, redirecting the client **330** to FSVM-3. The client **330** may then determine the network address for FSVM-3, which is ip-addr3 (e.g., a host name “ip-addr3.domain.name” or an IP address, 10.1.1.3). The client **330** may determine the network address for FSVM-3 by searching a cache stored in memory of the client **330**, which may contain a mapping from FSVM-3 to ip-addr3 cached in a previous operation. If the cache does not contain a network address for FSVM-3, then at step **388** the client **330** may send a request to the name server **332** to resolve the name FSVM-3. The name server may respond with the resolved address, ip-addr3, at step **389**. The client **330** may then store the association between FSVM-3 and ip-addr3 in the client’s cache.

In particular embodiments, failure of FSVMs **170** may be detected using the centralized coordination service. For example, using the centralized coordination service, each FSVM **170a** may create a lock on the host machine **200a** on which the FSVM **170a** is located using ephemeral nodes of the centralized coordination service (which are different from host machines **200** but may correspond to host machines **200**). Other FSVMs **170b** and **170c** may volunteer

for leadership of resources of remote FSVMs **170** on other host machines **200**, e.g., by requesting a lock on the other host machines **200**. The locks requested by the other nodes are not granted unless communication to the leader host machine **200c** is lost, in which case the centralized coordination service deletes the ephemeral node and grants the lock to one of the volunteer host machines **200a** and **200b**, which becomes the new leader. For example, the volunteer host machines **200a** and **200b** may be ordered by the time at which the centralized coordination service received their requests, and the lock may be granted to the first host machine **200** on the ordered list. The first host machine **200** (e.g., host machine **200b**) on the list may thus be selected as the new leader. The FSVM **170b** on the new leader has ownership of the resources that were associated with the failed leader FSVM **170a** until the failed leader FSVM **170c** is restored, at which point the restored FSVM **170a** may reclaim the local resources of the host machine **200c** on which it is located.

At step **390**, the client **330** may send an access request to FSVM-3 **170c** at ip-addr3 on Host-3 **200c** requesting the contents of Folder-3 of Share-1. At step **391**, FSVM-3 **170c** queries FSVM-3’s copy of the map **360** using FSVM-3’s instance of the distributed data access coordinator **370c**. The map **360** indicates that Folder-3 is located on FSVM-3, so at step **392** FSVM-3 accesses the file system **364c** to retrieve information about Folder-3 **316** and its contents (e.g., a list of files in the folder, which includes File-2 **320**) that are stored on the local storage **122c**. FSVM-3 may access local storage **122c** via CVM-3 **110c**, which provides access to local storage **122c** via a volume group **368c** that contains one or more volumes stored on one or more storage devices in local storage **122c**. At step **393**, FSVM-3 may then send the information about Folder-3 and its contents to the client **330**. Optionally, FSVM-3 may retrieve the contents of File-2 and send them to the client **330**, or the client **330** may send a subsequent request to retrieve File-2 as needed.

FIG. 3D illustrates an example virtualized file server having a failover capability according to particular embodiments. To provide high availability, e.g., so that the file server continues to operate after failure of components such as a CVM, FSVM, or both, as may occur if a host machine fails, components on other host machines may take over the functions of failed components. When a CVM fails, a CVM on another host machine may take over input/output operations for the failed CVM. Further, when an FSVM fails, an FSVM on another host machine may take over the network address and CVM or volume group that were being used by the failed FSVM. If both an FSVM and an associated CVM on a host machine fail, as may occur when the host machine fails, then the FSVM and CVM on another host machine may take over for the failed FSVM and CVM. When the failed FSVM and/or CVM are restored and operational, the restored FSVM and/or CVM may take over the operations that were being performed by the other FSVM and/or CVM. In FIG. 3D, FSVM-1 **170a** communicates with CVM-1 **110a** to use the data storage in volume groups VG1 **368a** and VG2 **369a**. For example, FSVM-1 is using disks in VG1 and VG2, which are iSCSI targets. FSVM-1 has iSCSI initiators that communicate with the VG1 and VG2 targets using MPIO (e.g., DM-MPIO on the LINUX operating system). FSVM-1 may access the volume groups VG1 and VG2 via in-guest iSCSI. Thus, any FSVM may connect to any iSCSI target if an FSVM failure occurs.

In particular embodiments, during failure-free operation, there are active iSCSI paths between FSVM-1 and CVM-1, as shown in FIG. 3D by the dashed lines from the FSVM-1

file systems for FS1 **364a** and FS2 **365a** to CVM-1's volume group VG1 **368a** and VG2 **369a**, respectively. Further, during failure-free operation there are inactive failover (e.g., standby) paths between FSVM-1 and CVM-3 **110c**, which is located on Host-3. The failover paths may be, e.g., paths that are ready to be activated in response to the local CVM CVM-1 becoming unavailable. There may be additional failover paths that are not shown in FIG. 3D. For example, there may be failover paths between FSVM-1 and a CVM on another host machine, such as CVM-2 **110b** on Host-2 **200b**. The local CVM CVM-1 **110a** may become unavailable if, for example, CVM-1 crashes, or the host machine on which the CVM-1 is located crashes, loses power, loses network communication between FSVM-1 **170a** and CVM-1 **110a**. As an example and not by way of limitation, the failover paths do not perform I/O operations during failure-free operation. Optionally, metadata associated with a failed CVM **110a**, e.g., metadata related to volume groups **368a**, **369a** associated with the failed CVM **110a**, may be transferred to an operational CVM, e.g., CVM **110c**, so that the specific configuration and/or state of the failed CVM **110a** may be re-created on the operational CVM **110c**.

FIG. 3E illustrates an example virtualized file server that has recovered from a failure of Controller/Service VM CVM-1 **110a** by switching to an alternate Controller/Service VM CVM-3 **110c** according to particular embodiments. When CVM-1 **110a** fails or otherwise becomes unavailable, then the FSVM associated with CVM-1, FSVM-1 **170a**, may detect a PATH DOWN status on one or both of the iSCSI targets for the volume groups VG1 **368a** and VG2 **369a**, and initiate failover to a remote CVM that can provide access to those volume groups VG1 and VG2. For example, when CVM-1 **110a** fails, the iSCSI MPIO may activate failover (e.g., standby) paths to the remote iSCSI target volume group(s) associated with the remote CVM-3 **110c** on Host-3 **200c**. CVM-3 provides access to volume groups VG1 and VG2 as VG1 **368c** and VG2 **369c**, which are on storage device(s) of local storage **122c**. The activated failover path may take over I/O operations from failed CVM-1 **110a**. Optionally, metadata associated with the failed CVM-1 **110a**, e.g., metadata related to volume groups **368a**, **369a**, may be transferred to CVM-3 so that the specific configuration and/or state of CVM-1 may be re-created on CVM-3. When the failed CVM-1 again becomes available, e.g., after it has been re-started and has resumed operation, the path between FSVM-1 and CVM-1 may be reactivated or marked as the active path, so that local I/O between CVM-1 and FSVM-1 may resume, and the path between CVM-3 and FSVM-1 may again become a failover (e.g., standby) path.

FIG. 3F illustrates an example virtualized file server that has recovered from failure of a FSVM by electing a new leader FSVM according to particular embodiments. When an FSVM-2 **170b** fails, e.g., because it has been brought down for maintenance, has crashed, the host machine on which it was executing has been powered off or crashed, network communication between the FSVM and other FSVMs has become inoperative, or other causes, then the CVM that was being used by the failed FSVM, the CVM's associated volume group(s), and the network address of the host machine on which the failed FSVM was executing may be taken over by another FSVM to provide continued availability of the file services that were being provided by the failed FSVM. In the example shown in FIG. 3F, FSVM-2 **170b** on Host-2 **200b** has failed. One or more other FSVMs, e.g., FSVM-1 **170a** or FSVM-3 **170c**, or other components located on one or more other host machines, may detect the

failure of FSVM-2, e.g., by detecting a communication timeout or lack of response to a periodic status check message. When FSVM-2's failure is detected, an election may be held, e.g., using a distributed leader election process such as that provided by the centralized coordination service. The host machine that wins the election may become the new leader for the file system pools **366b**, **367b** for which the failed FSVM-2 was the leader. In this example, FSVM-1 **170a** wins the election and becomes the new leader for the pools **366b**, **367b**. FSVM-1 **170a** thus attaches to CVM-2 **110b** by creating file system **364b**, **365c** instances for the file server instances FS1 and FS2 using FS1-Pool-3 **366b** and FS2-Pool-4 **367b**, respectively. In this way, FSVM-1 takes over the file systems and pools for CVM-2's volume groups, e.g., volume groups VG1 **366b** and VG2 **367b** of local storage **122b**. Further, FSVM-1 takes over the IP address associated with FSVM-2, 10.1.1.2, so that storage access requests sent to FSVM-2 are received and processed by FSVM-1. Optionally, metadata used by FSVM-1, e.g., metadata associated with the file systems, may be transferred to FSVM-3 so that the specific configuration and/or state of the file systems may be re-created on FSVM-3. Host-2 **200b** may continue to operate, in which case CVM-2 **110b** may continue to execute on Host-2. When FSVM-2 again becomes available, e.g., after it has been re-started and has resumed operation, FSVM-2 may assert leadership and take back its IP address (10.1.1.2) and storage (FS1-Pool-3 **366b** and FS2-Pool-4 **367b**) from FSVM-1.

FIGS. 3G and 3H illustrate example virtualized file servers that have recovered from failure of a host machine **200a** by switching to another Controller/Service VM and another FSVM according to particular embodiments. The other Controller/Service VM and FSVM are located on a single host machine **200c** in FIG. 3G, and on two different host machines **200b**, **200c** in FIG. 3H. In both FIGS. 3G and 3H, Host-1 **200a** has failed, e.g., crashed or otherwise become inoperative or unresponsive to network communication. Both FSVM-1 **170a** and CVM-1 **110a** located on the failed Host-1 **200a** have thus failed. Note that the CVM **110a** and FSVM **170a** on a particular host machine **200a** may both fail even if the host machine **200a** itself does not fail. Recovery from failure of a CVM **110a** and an FSVM **170a** located on the same host machine **200a**, regardless of whether the host machine **200a** itself failed, may be performed as follows. The failure of FSVM-1 and CVM-1 may be detected by one or more other FSVMs, e.g., FSVM-2 **170b**, FSVM-3 **170c**, or by other components located on one or more other host machines. FSVM-1's failure may be detected when a communication timeout occurs or there is no response to a periodic status check message within a timeout period, for example. CVM-1's failure may be detected when a PATH DOWN condition occurs on one or more of CVM-1's volume groups' targets (e.g., iSCSI targets).

When FSVM-1's failure is detected, an election may be held as described above with reference to FIG. 3F to elect an active FSVM to take over leadership of the portions of the file server instance for which the failed FSVM was the leader. These portions are FileSystem-1A **364a** for the portion of file server FS1 located on FSVM-1, and FileSystem-2A **365a** for the portion of file server FS2 located on FSVM-1. FileSystem-1A **364a** uses the pool FS-Pool-1 **366a** and FileSystem-2A **365a** uses the pool FS2-Pool-2 **367a**. Thus, the FileSystem-1A **364a** and FileSystem-2A may be re-created on the new leader FSVM-3 **170c** on Host-3 **200c**. Further, FSVM-3 **170c** may take over the IP

address associated with failed FSVM-1 **170a**, 10.1.1.1, so that storage access requests sent to FSVM-1 are received and processed by FSVM-3.

One or more failover paths from an FSVM to volume groups on one or more CVMs may be defined for use when a CVM fails. When CVM-1's failure is detected, the MPIO may activate one of the failover (e.g., standby) paths to remote iSCSI target volume group(s) associated with a remote CVM. For example, there may be a first predefined failover path from FSVM-1 to the volume groups VG1 **368c**, **369c** in CVM-3 (which are on the same host as FSVM-1 when FSVM-1 is restored on Host-3 in examples of FIGS. **3G** and **3H**), and a second predefined failover path to the volume groups VG1 **368b**, VG2 **369b** in CVM-2. The first failover path, to CVM-3, is shown in FIG. **3G**, and the second failover path, to CVM-2 is shown in FIG. **3H**. An FSVM or MPIO may choose the first or second failover path according to the predetermined MPIO failover configuration that has been specified by a system administrator or user. The failover configuration may indicate that the path is selected (a) by reverting to the previous primary path, (b) in order of most preferred path, (c) in a round-robin order, (d) to the path with the least number of outstanding requests, (e) to the path with the least weight, or (f) to the path with the least number of pending requests. When failure of CVM-1 **110a** is detected, e.g., by FSVM-1 or MPIO detecting a PATH DOWN condition on one of CVM-1's volume groups VG1 **368a** or VG2 **369a**, the alternate CVM on the selected failover path may take over I/O operations from the failed CVM-1. As shown in FIG. **3G**, if the first failover path is chosen, CVM-3 **110c** on Host-3 **200c** is the alternate CVM, and the pools FS1-Pool-1 **366a** and FS2-Pool-2 **367a**, used by the file systems FileSystem-1A **364a** and FileSystem-2A **365a**, respectively, which have been restored on FSVM-3 on Host-3, may use volume groups VG1 **368c** and VG2 **369c** of CVM-3 **110c** on Host-3 when the first failover path is chosen. Alternatively, as shown in FIG. **3H**, if the second failover path is chosen, CVM-2 on Host-2 is the alternate CVM, and the pools FS1-Pool-1 **366a** and FS2-Pool-2 **367a** used by the respective file systems FileSystem-1A **364a** and FileSystem-2A **365a**, which have been restored on FSVM-3, may use volume groups VG1 **368b** and VG2 **369b** on Host-2, respectively.

Optionally, metadata used by FSVM-1 **170a**, e.g., metadata associated with the file systems, may be transferred to FSVM-3 as part of the recovery process so that the specific configuration and/or state of the file systems may be re-created on FSVM-3. Further, metadata associated with the failed CVM-1 **110a**, e.g., metadata related to volume groups **368a**, **369a**, may be transferred to the alternate CVM (e.g., CVM-2 or CVM-3) that the specific configuration and/or state of CVM-1 may be re-created on the alternative CVM. When FSVM-1 again becomes available, e.g., after it has been re-started and has resumed operation on Host-1 **200a** or another host machine, FSVM-1 may assert leadership and take back its IP address (10.1.1.1) and storage assignments (FileSystem-1A and FS1-Pool-1 **366a**, and FileSystem-2A and FS2-Pool-2 **366b**) from FSVM-3. When CVM-1 again becomes available, MPIO or FSVM-1 may switch the FSVM to CVM communication paths (iSCSI paths) for FileSystem-1A **364a** and FileSystem-2A **365a** back to the pre-failure paths, e.g., the paths to volume groups VG1 **368a** and **369a** in CVM-1 **110a**, or the selected alternate path may remain in use. For example, the MPIO configuration may specify that fail back to FSVM-1 is to occur when the primary path is restored, since communication between FSVM-1 and CVM-1 is local and may be faster than

communication between FSVM-1 and CVM-2 or CVM-3. In this case, the paths between CVM-2 and/or CVM-3 and FSVM-1 may again become failover (e.g., standby) paths.

FIGS. **4A** and **4B** illustrate an example hierarchical namespace **400** of a file server according to particular embodiments. Cluster-1 **402** is a cluster, which may contain one or more file server instances, such as an instance named FS1.domain.com **404**. Although one cluster is shown in FIGS. **4A** and **4B**, there may be multiple clusters, and each cluster may include one or more file server instances. The file server FS1.domain.com **404** contains three shares: Share-1 **406**, Share-2 **408**, and Share-3 **410**. Share-1 may be a home directory share on which user directories are stored, and Share-2 and Share-3 may be departmental shares for two different departments of a business organization, for example. Each share has an associated size in gigabytes, e.g., 100 GB (gigabytes) for Share-1, 100 GB for Share-2, and 10 GB for Share-3. The sizes may indicate a total capacity, including used and free space, or may indicate used space or free space. Share-1 includes three folders, Folder-A1 **412**, Folder-A2 **414**, and Folder-A3 **416**. The capacity of Folder-A1 is 18 GB, Folder-A2 is 16 GB, and Folder-A3 is 66 GB. Further, each folder is associated with a user, referred to as an owner. Folder-A1 is owned by User-1, Folder-A2 by User-2, and Folder-A3 by User-3. Folder-A1 contains a file named File-A1-1 **418**, of size 18 Gb. Folder-A2 contains 32 files, each of size 0.5 GB, named File-A2-1 **420** through File-A2-32 **422**. Folder-A3 contains 33 files, each of size 2 GB, named File-A3-1 **423** and File-A3-2 **424** through File-A3-33 **426**.

FIG. **4B** shows the contents of Share-2 **408** and Share-3 **410** of FS1.domain.com **404**. Share-2 contains a folder named Folder-B1 **440**, owned by User-1 and having a size of 100 Gb. Folder-B1 contains File-B1-1 **442** of size 20 Gb, File-B1-2 **444** of size 30 Gb, and Folder-B2 **446**, owned by User-2 and having size 50 Gb. Folder-B2 contains File-B2-1 **448** of size 5 Gb, File-B2-2 **450** of size 5 Gb, and Folder-B3 **452**, owned by User-3 and having size 40 Gb. Folder-B3 **452** contains 20 files of size 2 Gb each, named File-B3-1 **454** through File-B3-20 **456**. Share-3 contains three folders: Folder-C7 **429** owned by User-1 of size 3 GB, Folder-C8 **430** owned by User-2 of size 3 GB, and Folder-C9 **432** owned by User-3 of size 4 GB.

FIG. **4C** illustrates distribution of stored data amongst host machines in a virtualized file server according to particular embodiments. In the example of FIG. **4C**, the three shares are spread across three host machines **200a-c**. Approximately one-third of each share is located on each of the three FSVMs **170a-c**. For example, approximately one-third of Share-3's files are located on each of the three FSVMs **170a-c**. Note that from a user's point of a view, a share looks like a directory. Although the files in the shares (and in directories) are distributed across the three host machines **200a-c**, the VFS **202** provides a directory structure having a single namespace in which client executing on user VMs **101** and **102** may access the files in a location-transparent way, e.g., without knowing which host machines store which files (or which blocks of files).

In the example of FIG. **4C**, Host-1 stores (e.g., is assigned to) 28 Gb of Share-1, including 18 Gb for File-A1-1 **418** and 2 Gb each for File-A3-1 **423** through File-A3-5 **425**, 33 Gb of Share-2, including 20 Gb for File-B1-1 and 13 Gb for File-B1-2, and 3 Gb of Share-3, including 3 Gb of Folder-C7. Host-2 stores 26 Gb of Share-1, including 0.5 Gb each of File-A2-1 **420** through File-A2-32 **422** (16 Gb total) and 2 Gb each of File-A3-6 **426** through File-A3-10 **427** (10 Gb total), 27 Gb of Share-2, including 17 Gb of File-B1-2, 5 Gb

31

of File-B2-1, and 5 Gb of File-B2-2, and 3 Gb of Share-3, including 3 Gb of Folder-C8. Host-3 stores 46 GB of Share-1, including 2 GB each of File-A3-11 429 through File-A3-33 428 (66 GB total), 40 GB of Share-2, including 2 GB each of File-B3-1 454 through File-B3-20 456, and Share-3 stores 4 GB of Share-3, including 4 GB of Folder-C9 432.

In particular embodiments, a system for managing communication connections in a virtualization environment includes a plurality of host machines implementing a virtualization environment. Each of the host machines includes a hypervisor and at least one user virtual machine (user VM) 101. The system may also include a connection agent, an I/O controller, and/or a virtual disk comprising a plurality of storage devices. The virtual disk may be accessible by all of the I/O controllers, and the I/O controllers may conduct I/O transactions with the virtual disk based on I/O requests received from the user VMs 101. The I/O requests may be, for example, requests to perform particular storage access operations such as list folders and files in a specified folder, create a new file or folder, open an existing file for reading or writing, read data from or write data to a file, as well as file manipulation operations to rename, delete, copy, or get details, such as metadata, of files or folders. Each I/O request may reference, e.g., identify by name or numeric identifier, a file or folder on which the associated storage access operation is to be performed. The system further includes a virtualized file server, which includes a plurality of FSVMs 170 and associated local storage 122. Each FSVM 170 and associated local storage device 122 is local to a corresponding one of the host machines 200. The FSVMs 170 conduct I/O transactions with their associated local storage 122 based on I/O requests received from the user VMs 101. For each one of the host machines 200, each of the user VMs 101 on the one of the host machines 200 sends each of its respective I/O requests 383 to a selected one of the FSVMs 170, which may be selected based on a lookup table 360, e.g., a sharding map, that maps a file 318, folder 312, or other storage resource referenced by the I/O request to the selected one of the FSVMs 170).

In particular embodiments, the initial FSVM to receive the request from the user VM may be determined by selecting any of the FSVMs 170 on the network 140, e.g., at random, by round robin selection, or by a load-balancing algorithm, and sending an I/O request 383 to the selected FSVM 170 via the network 140 or via local communication within the host machine 200. Local communication may be used if the file 318 or folder 412 referenced by the I/O request is local to the selected FSVM, e.g., the referenced file or folder is located on the same host machine 200 as the selected FSVM 170. In this local case, the I/O request 383 need not be sent via the network 140. Instead, the I/O request 383 may be sent to the selected FSVM 170 using local communication, e.g., a local communication protocol such as UNIX domain sockets, a loopback communication interface, inter-process communication on the host machine 200, or the like. The selected FSVM 170 may perform the I/O transaction specified in the I/O request and return the result of the transaction via local communication. If the referenced file or folder is not local to the selected FSVM, then the selected FSVM may return a result indicating that the I/O request cannot be performed because the file or folder is not local to the FSVM. The user VM may then submit a REFERRAL request or the like to the selected FSVM, which may determine which FSVM the referenced file or folder is local to (e.g., by looking up the FSVM in a distributed mapping table), and return the identity of that FSVM to the

32

user VM in a REDIRECT response or the like. Alternatively, the selected FSVM may determine which FSVM the referenced file or folder is local to, and return the identity of that FSVM to the user VM in the first response without the REFERRAL and REDIRECT messages. Other ways of redirecting the user VM to the FSVM of the referenced file are contemplated. For example, the FSVM that is on the same host as the requesting user VM (e.g., local to the requesting user VM) may determine which FSVM the file or folder is local to, and inform the requesting user VM of the identity of that FSVM without communicating with a different host.

In particular embodiments, the file or folder referenced by the I/O request includes a file server name that identifies a virtualized file server on which the file or folder is stored. The file server name may also include or be associated with a share name that identifies a share, file system, partition, or volume on which the file or folder is stored. Each of the user VMs on the host machine may send a host name lookup request, e.g., to a domain name service, that includes the file server name, and may receive one or more network addresses of one or more host machines on which the file or folder is stored.

In particular embodiments, as described above, the FSVM may send the I/O request to a selected one of the FSVMs. The selected one of the FSVMs may be identified by one of the host machine network addresses received above. In one aspect, the file or folder is stored in the local storage of one of the host machines, and the identity of the host machines may be determined as described below.

In particular embodiments, when the file or folder is not located on storage local to the selected FSVM, e.g., when the selected FSVM is not local to the identified host machine, the selected FSVM responds to the I/O request with an indication that the file or folder is not located on the identified host machine. Alternatively, the FSVM may look up the identity of the host machine on which the file or folder is located, and return the identity of the host machine in a response.

In particular embodiments, when the host machine receives a response indicating that the file or folder is not located in the local storage of the selected FSVM, the host machine may send a referral request (referencing the I/O request or the file or folder from the I/O request) to the selected FSVM. When the selected FSVM receives the referral request, the selected FSVM identifies one of the host machines that is associated with a file or folder referenced in the referral request based on an association that maps files to host machines, such as a sharding table (which may be stored by the centralized coordination service). When the selected FSVM is not local to the host machine, then the selected FSVM sends a redirect response that redirects the user VM on the host machine to the machine on which the selected FSVM is located. That is, the redirect response may reference the identified host machine (and by association the selected second one of the FSVMs). In particular embodiments, the user VM on the host machine receives the redirect response and may cache an association between the file or folder referenced in the I/O request and the host machine referenced in the redirect response.

In particular embodiments, the user VM on the host machine may send a host name lookup request that includes the name of the identified host machine to a name service, and may receive the network address of the identified host machine from the name service. The user VM on the host machine may then send the I/O request to the network address received from the name service. The FSVM on the

host machine may receive the I/O request and performs the I/O transaction specified therein. That is, when the FSVM is local to the identified host machine, the FSVM performs the I/O transaction based on the I/O request. After performing or requesting the I/O transaction, the FSVM may send a response that includes a result of the I/O transaction back to the requesting host machine. I/O requests from the user VM may be generated by a client library that implements file I/O and is used by client program code (such as an application program).

Particular embodiments may provide dynamic referral type detection and customization of the file share path. Referring to FIG. 3C, when a user VM (e.g., client 330 or one of the user VMs 105a-c) sends a request for a storage access operation specifying a file share to a FSVM node (e.g., FSVM-1 170a) in the VFS cluster of FSVM nodes, the user VM may be sent a referral to another FSVM node (e.g., FSVM-3 170c) that is assigned to the relevant file share. Certain types of authentication may use either host-based referrals (e.g., Kerberos) or IP-based referrals (e.g., NTLM). In order to flexibly adapt to any referral type, particular embodiments of the FSVM-1/3 170a and 170c may detect the referral type in an incoming request and construct a referral response that is based on the referral type and provide the referral. For example, if the user VM sends a request to access a storage item at a specified file share using an IP address, particular embodiments may construct and provide an IP address-based referral; if the user VM sends a request to access the storage item at the specified file share using a hostname, then particular embodiments may construct and provide a hostname-based referral, including adding the entire fully qualified domain name.

For example, if a user VM sends a request for File-A2-1 (which resides on Node-2, as illustrated in FIG. 4C) to Node-1 using a hostname-based address \\fs1\share-1\File-A2-1, VFS 202 may determine that File-A2-1 actually resides on Node-2 and send back a referral in the same referral type (hostname) as the initial request: \\fs2.domain.com\share-1\File-A2-1. If a user VM sends a request for File-A2-1 to Node-1 using an IP-based address \\198.82.0.23\share-1\File-A2-1, after determining that File-A2-1 actually resides on Node-2, VFS 202 may send back a referral in the same referral type (IP) as the initial request: \\198.82.0.43\share-1\File-A2-1.

In particular embodiments, the hostname for the referral node may be stored in a distributed cache in order to construct the referral dynamically using hostname, current domain, and share information.

FIG. 5 illustrates an example method for accessing data in a virtualized file server according to particular embodiments. The client system 330 may access the data, such as a specified folder, as follows. At step 502, the client system 330 receives a storage access request from an application executing in a user VM. Each storage access request references a file path (e.g., \\FS1.share.com\share-1\Folder-1), which includes a file or folder name and further includes or can be used to identify a share name (e.g., FS1.share.com\share-1) or an NFS remote file system name (e.g., fs1.share.com:/share-1. The storage access request may also include an operation type (e.g., read, write, delete, rename, etc.), a position in the file (for read/write requests), data to be written (for write requests), quantity of data to be read (for read requests), a new file path (for rename requests), folder name (for folder creation requests) or other information appropriate for the operation type. At step 504, the client system may send a DNS query request for the file server portion of the share name (e.g., \\fs1.domain.com for

the share \\FS1.domain.com\share-1) to a name server 332, which may return the identity of a selected host machine as a result. The name server 332 may be a DNS server. The selected host machine is not necessarily the host machine on which the file or folder itself is located, however, since the share may be distributed amongst multiple host machines, one of which actually stores the file or folder. In particular embodiments, a FSVM each host machine can determine which host machine a file is stored on, and, if a FSVM receives a request for a file stored on a different host machine, the FSVM sends a referral response that includes the identity of the host machine on which the file is stored.

At step 506, the name server 332 may respond to the client with an IP (network) address of one or more host machines on FSVMs for the file or folder may be located. For example, the DNS server entry FS1.domain.com includes entries for FSVM-1, FSVM-2, and FSVM-3, which are, respectively, ip-addr1, ip-addr2, ip-addr3 (or 10.1.1.1, 10.1.1.2, 10.1.1.3). One of these three example IP addresses may be selected by the DNS server and returned in a response. In one example, the DNS server returns the three IP addresses in a different permutation for each request using DNS round robin so that a different server may be selected by the client for each request to balance the request load among the three servers. In this example, ip-addr1 (10.1.1.1) is the first address in the list sent in the reply to the client 330, and so is selected by the client as the address to which the I/O request will, at least initially, be sent. At step 508, the client may send the I/O request to access the folder "Folder-3" to the FSVM located on the host machine having address ip-addr1. The I/O request may be, e.g., a DFS attach or connect request, an NFS open request, or the like.

At step 510, FSVM-1 170a on Host-1 200a receives the I/O request and consults a map or lookup table, such as the sharding map 360a, to determine whether the folder "Folder-3" is stored on a locally-attached storage resource of the host machine on which FSVM 170a is located. If so, FSVM 170a performs executes step 567 to perform the I/O transaction identified by the I/O request. If not, at step 512 FSVM-1 170a responds to the client 330 with an indication that the folder is not located on the FSVM-1 170a's host machine 200a. The indication may be, e.g., a PATH_NOT_COVERED DFS response. At step 514, upon receiving the indication that the file is not located on the FSVM 170a to which the request was sent, the client 330 sends a DFS REFERRAL request to FSVM 170a, requesting a referral to the FSVM on which "Folder-3" is stored. At step 545, FSVM 170a receives the REFERRAL request and sends a DFS "REDIRECT to FSVM-3" response back to the client 330. FSVM 170a looks up the FSVM on which the folder "Folder-3" is stored in the map 360a that associates files or shares with host machines. The result of the lookup, FSVM-3 170c, may have been determined previously by the lookup at step 510 when the initial request for Folder-3 was received, or may be determined at step 516 when the referral request for Folder-3 is received. For example, the map 360a may be stored in a shared data structure provided by the centralized coordination service, and the lookup may be performed by accessing the shared data structure. In this example, the file or folder is "Folder-3" and map indicates that the folder is associated with FSVM 170c, so at step 516 FSVM 170a may send a REDIRECT response to the client indicating that the requested folder is stored on host machine 200c (on which FSVM 170c is located). The REDIRECT response may reference the host machine 200c, the FSVM 170c, the network address of host machine 200c (e.g., ip-addr3, in which case steps 518 and 520 may not be

35

necessary), or other identifier for the location of the requested folder. The client 330 may receive the REDIRECT response and cache the association between Folder-3 and host machine 200c (and/or FSVM 170c) for potential future use.

At step 518, the client 330 may send a DNS query request to the DNS server 332 to determine the IP address of the FSVM specified in the received REDIRECT response, which is FSVM 170c having IP address ip-addr3 in this example. At step 520, the DNS server 332 may send a reply to the client 330 indicating the IP address of the requested host machine. For example, the reply may be ip-addr3 (or 10.1.1.3), which is the IP address of FSVM 170c. At step 522, the client sends the I/O request to access Folder-3 to the IP address received in the DNS reply (e.g., ip-addr3). At step 524, the FSVM 170c on host machine 200c receives the I/O request that references Folder-3 and looks up Folder-3 in the sharding map. At step 526, FSVM 170c performs the requested I/O transaction for Folder-3, e.g., by accessing local storage 122c, and sends the results of the access, e.g., details about Folder-3 in this example, such as a list of files and associated metadata, back to the client 330 in an I/O response. The client 330 receives the I/O response and may pass the results of the I/O transaction to the application or other program code that requested the access. Any subsequent requests for the same data (Folder-3 in this example) by the client 330 may be sent directly to host machine 200c on which the data is stored because the client 330 may use the cached identity of the host machine or FSVM on which the data is stored. Although data contained in a folder is accessed in the example of FIG. 5, other types of data may be accessed similarly, e.g., data contained in files.

In particular embodiments, a VFS 202 consists of multiple compute units, e.g., FSVMs 170. These FSVMs 170 act as a single VFS 202 to the outside world. Clusters with appropriate platforms and licenses use a hypervisor-agnostic code-image for the VFS 202. This image may be stored as part of pre-created, ready to use disk images. When a user with correct privileges decides to create a VFS 202, the image may be cloned N times, where N is the number of FSVMs 170, and the FSVMs 170 are created. The FSVMs 170 form a cluster, which may provide a VFS 202 to the outside world. In this way, the user is abstracted from the complex process of deploying the VFS 202, as the input requested from the user is a small number of parameters that can be provided by the user in a simple user interface. The pre-created fileserver image may reduce the deployment time to be as fast as booting the host machines 200.

In particular embodiments, the VFS comprises multiple FSVMs 170. The host machines 200 may combine to act as VFS 202 to the outside world. Each host machine 200 may have two types of vDisks: code and data. The operating system (OS) code and file server code reside on the code vDisk. The fileserver persistent data and configuration are stored on the data vDisk. In a first technique for upgrading the VFS 202, before the upgrade process is started, the newer version of the code disk is prepared and cloned N times (where N is the number of FSVMs 170). While upgrading the VFS 202 to the latest version, a new code disk is swapped with the existing code disk for each FSVM 170. After rebooting the FSVM 170, it will be running with newer code, and continues serving the data using the newer code.

In particular embodiments, in a second technique for upgrading the VFS 202, before the upgrade process is started and after the newer version of the code disk is prepared and cloned, a first FSVM 170a acquires an upgrade token, swaps

36

the old code disk with the newer disk, and reboots. When the first FSVM 170a comes back up and is running, the upgrade token is passed to the next FSVM 170b, which may perform the swap and reboot, and pass the upgrade token to the next FSVM 170c. These operations are repeated until the last FSVM 170, e.g., FSVM 170c in this example, is upgraded. During the time that each FSVM 170b is being rebooted, one of the peer FSVMs 170a takes over the storage and IP address of the FSVM 170b so that the client does not see any interruption in the file service.

In particular embodiments, users dealing with Virtual Disk Image (“VDI”) files and their corresponding root directories are, by definition, bound to their VMs. This binding provides a user VM 101 to root-level directory mapping. A sharding algorithm may determine a mapping between a user VM 101 and its corresponding host machine 200. This mapping may in turn provide a root-level directory-to-host machine mapping. The sharding algorithm may use this mapping and add metadata to keep storage units and compute units local, e.g., located on, the same host machine. On migration of the user virtual machines 102, metadata and storage will be moved accordingly.

Particular embodiments may provide enhanced performance via adaptive data and compute-unit migration. Particular embodiments may provide the ability to restrict compute units and storage units to a location governed by user policy.

In particular embodiments, data migration from an existing VFS 202 to a new VFS 202 may be bounded by the speed of connection between the existing infrastructure (e.g., host machines 200) and the new system (e.g., other host machines). By using smart data ingestion, data migration speed can be increased with a multiplier of the number of file server host machines.

In previous approaches, data is migrated using a utility to copy data from one source to one target location. Migration speed is limited by the connection speed. In particular embodiments, using the smart data ingestion approach described herein, top-level directories in an existing VFS 202 are preprocessed to acquire the destination host machine 200 on a new (destination) VFS 202. When data migration begins, each host machine 200 in the VFS 202 starts data migration with the share directories assigned, which speeds up data migration with a multiplier of host machine count. By taking advantage of the distributed nature of the VFS 202, data migration is performed in parallel to speed up the migration process. Using the same sharding algorithm as file I/O to decide the migration target ensure the consistency of migrated data placement in, e.g., the new (destination) VFS 202. In particular embodiments, no further processing is needed after data is migrated, and data is ready to be served.

In a first example, when the organization that manages a virtualized file server instance (VFS) 202 decides to, for example, segregate the existing VFS 202 to a departmental level, the VFS 202 may be split into multiple virtualized file server instances (VFSs) 202 without affecting the stored data, with zero to minimal down time, and with zero data copying or migration. In a second example, when an organization that manages multiple VFSs 202 decides to merge them into one manageable VFS 202, the multiple VFSs 202 may be merged together without affecting the stored data, and with zero to minimal down time and zero data copying or migration. When an organization needs to merge multiple VFSs 202, then a system administrator may deploy a new VFS 202 and migrate the stored data from the multiple VFSs 202 to the newly deployed VFS 202, which takes more time and resources. When the organization needs to split the VFS

202, then a system administrator may deploy new VFSs 202 and migrate the data from the old VFS 202 to the newly deployed VFSs 202, which also takes more time and resources.

In particular embodiments, the splitting and merging operations may be performed as follows. To split an existing VFS 202, e.g., upon a system administrator's request, the following operations may be performed:

1. Select the FSVMs 170 to be segregated from VFS 202.
2. The FSVMs 170 are removed one by one.
3. Before removing a FSVM 170 from the VFS 202, first select a lightly loaded FSVM 170 and voluntarily relinquish the storage resources to the selected FSVM 170. The IP address of the FSVM 170 being removed may also be moved to the selected FSVM 170 to retain SMB client connections.
4. After removing all the FSVMs 170 from the VFS 202, a new VFS 202 is constructed.
5. The FSVMs 170 of the new VFS 202 join the domain and start serving the new shares. Old shares may still be served by the old VFS 202. Once the administrator decides to move the old shares to the new VFS 202, trigger a storage transition that relinquishes the storage to the appropriate selected FSVMs 170 and move the IP addresses of FSVMs 170 of the old VFS 202 to FSVMs 170 of the new VFS 202.
6. The same process may be continued to segregate other VFSs 202.

In particular embodiments, to merge multiple VFSs 202 together, e.g., upon a system administrator's request, an election is triggered between the multiple VFSs 202 based on the virtual IP address or based on preference policies. The VFS 202 that wins the election or is selected by an administrator is treated as a master VFS. All other VFSs then join to the master VFS's ACTIVE DIRECTORY domain. FSVMs 170 from all slave VFSs 202 may be added to the master VFS 202, and storage pool metadata of the slave VFSs 202 is modified to serve for the master VFS 202. The following operations may be performed to merge the slave VFSs into the master VFS:

1. Select the VFSs 202 to be merged.
2. Initiate the election to elect the master VFS 202 based on the policy of the VFS 202 that has the higher IP address.
3. Once the master VFS has been selected, clients connect to it.
4. Select a slave VFS to merge.
5. Relinquish the storage to a lightly-loaded FSVM on the master VFS and move the IP address to refer to the lightly-loaded FSVM.
6. Start serving SMB clients for new and old shares.
7. Stop the slave file server, add its FSVM(s) one by one to the new master file server, and take back its resource on the new master file server.
8. Continue these steps for other slave file servers.

In scenarios such as a company splitting into multiple companies, it could be a requirement that a single VFS 202 is split into two VFSs 202. However, there may be certain SAMBA shares in the original VFS 202 that need to be made accessible to both the VFSs 202. As an example, consider two different file servers FS1 and FS2. FS1 originally hosted a share 'share1'. FS2 needs the ability to read/write to the share 'share1'. The SMB requests for 'share1' on FS2 may be forwarded or proxied to FS1, thereby allowing the share 'share1' to be readable/writable from two different VFSs 202. Another approach is to NFS-mount the original share on the new VFS 202, to provide a single namespace. The

ability to selectively choose certain shares to be shared across other VFSs 202 ensures a tight security boundary at the VFS level, along with the collaborative access via two different VFSs.

In particular embodiments, disaster recovery of the VFS 202 may be performed by making backups and replicating delta changes in a storage layer, then recovering the data stored in the VFS 202 at a remote site. The data may be recovered by reconstructing the VFS 202 from a replicated configuration. In a production environment, the data stored on a VFS 202 is securely protected and restored on a remote location without loss of the data and metadata within a supported Recovery Point Objective (which may be the age of files to be recovered from backup storage for normal operations to resume). A custom replication policy may be configured for the VFS 202, and the ability may be provided to map the VFS 202's configuration between sites to provide disaster recovery of virtual file-services across geographical locations. Particular embodiments may provide the ability to protect individual shares or share groups by protecting the volume group(s) used for file-services storage, e.g., by adding them to a protection domain. Users may apply the replication and backup policies on the protection domain to configure the Recovery Point Objective, recovery sites (alternate cluster or cloud), and replication constraints such as bandwidth and schedule. Particular embodiments may take lightweight snapshots and transfer the delta of the snapshots for the given volume groups. Along with file-services share data, particular embodiments may also transfer the VFS configuration e.g. file-server size, compute-unit configuration, and metadata, e.g., share ACLs, quotas, and so on. Particular embodiments may also provide a simplified user interface to configure mapping of network, DNS-servers, active-directory, etc. between remote sites. Potential benefits may include:

1. Granular level of protection (share or group of shares) to configure different Recovery Point Objective.
2. Custom replication policies to utilize the network resources effectively for replication.
3. Fine control on network and ecosystem resource mapping between sites.
4. Light weight snapshot includes share data delta, metadata and file-server configuration leading to less replication traffic across sites.
5. One click restore of file-services on remote site.
6. Distribution of share replication across multiple remote sites.
7. Multiple recovery points on multiple remote sites for multi-site failures.

In particular embodiments, a high volume of snapshots (particularly lightweight snapshots) may be stored, and in some cases, snapshots may be captured and stored using a timestamp format that differs from the naming convention used by the client to access the snapshot. This may cause significant latency when browsing through thousands of snapshots, due to the performance overhead required to read the stat or query the file system to get the snapshot create time and then map it to corresponding client side label. For example, the underlying file system may choose to store snapshots using afs-auto-snapshot-hourly-YYYY-MM-DD-HHMM naming format; if the user VMs are then using SMB requests to retrieve the snapshots, they may be using @GMT tokens to access the snapshots. @GMT tokens are special token that can be present as part of a file path to indicate a request to see a previous version of the file or directory. The format is "@GMT-YYYY.MM.DD-HH.MM.SS".

Particular embodiments may, at the time of capturing and/or storing a snapshot, extract the creation timestamp from the snapshot name, utilize the creation timestamp to determine the expected request format then cache a mapping from the expected request format to the actual name of the corresponding snapshot. Latency may thereby be significantly reduced to a one-time computation that takes place at capture-time, rather than potentially multiple system calls taking place at recovery time.

FIG. 6 illustrates an example of how a file server 'FS1' may be deployed across multiple clusters according to particular embodiments. Particular embodiments may facilitate deploying and managing a VFS 202 whose networking, compute-unit, and storage resources are distributed across multiple clusters from a single management portal. Particular embodiments may create a VFS 202 and distribute compute units, which may be the FSVMs 170. A portal user interface may be used by a user or administrator to create a VFS 202. While creating the VFS 202, a user is given a list of clusters that may be used to distribute the compute units (e.g., FSVMs, or may perform the operations of FSVMs as described herein), networking (IP addresses) and storage (containers). In the example of FIG. 6, the user has chosen three clusters, Cluster 1, Cluster 2, and Cluster 3. Three FSVMs are created on each cluster, for a total of 9 FSVMs across the three clusters. Each cluster for this file server hosts a separate container, which holds a part of the file server data. The containers are labeled Container 1, Container 2, and Container 3. The containers are hidden from the user.

Particular embodiments may create shares and distribute storage units and compute units. The portal user interface may be used to create a share 'share1' within the file server FS1. The data within 'share1' is distributed across all the clusters. A storage pool of multiple vDisks is constructed on all FSVMs across all clusters. Each storage pool on each FSVM is responsible for a subset of the 'share1' data. The share is sharded at the top-level directories across FSVMs residing in different clusters. The sharding strategy is as follows. Assuming that directories dir1, dir2, dir3, dir4, dir5, dir6 have been created:

1. Each FSVM within each cluster hosts a storage pool created from a subset of the container storage. A background process periodically runs on a leader FSVM in each cluster to aggregate the file system space used for each share across all FSVMs in the cluster. This data is published to a cluster manager that stores the data in an entity database, e.g., APACHE CASSANDRA or the like. The cluster manager may be NUTANIX PRISM CENTRAL, which is a multi-cluster manager responsible for managing multiple clusters to provide a single, centralized management interface.
2. User creates a new top-level directory, e.g., 'dir7'.
3. The Samba VFS layer intercepts the directory creation request and consults a database to determine whether the directory is hosted by any FSVM (or FSVM). If it is not, the VFS layer makes an RPC call to a file server service running in PRISM CENTRAL to identify a location (which may be an optimal location) for 'dir7'.
4. The file server service running in PRISM CENTRAL retrieves the per-cluster usage statistics for each share that it received in step 1 above, and chooses the cluster that has the least used space for the share 'share1'. In the example of FIG. 6, Cluster 1 is chosen. The file server service may also provide an option to simply choose the cluster that has the greatest amount of free fileserver container space.

5. Next, the file server service running in PRISM CENTRAL queries Cluster 1 for average CPU utilization for the past 24 hours for all VMs within Cluster 1. The file server service then chooses the least loaded FSVM. The file server service in PRISM CENTRAL returns this <Cluster 1, FSVM2 FQDN> tuple back to the VFS layer.
6. The VFS layer now knows the FSVM2 FQDN, which should host 'dir7' and hence creates this new directory on the Unix file system corresponding to FSVM2. The VFS layer records this mapping <Share1, dir1>→Cluster 1, FSVM2 in a database, and returns a PATH_NOT_COVERED message to the client.
7. Through DFS referral, the SAMBA client requests the path for the directory 'dir1'. The FSVM looks up 'dir1' in the database, and returns FSVM2 IP to the client. The client now accesses 'dir1' on the FSVM2 file system.

The file system on any FSVM may be composed of vDisks. Since vDisks are distributed across the cluster, this arrangement provides uniform sharding of storage within the cluster. The sharding strategy described above causes all clusters' containers and FSVMs to be used, and achieves uniform sharding of storage units and compute units across the clusters.

Particular embodiments may provide cluster-aware sharding and cluster-aware share level quotas. At the time of share creation, user is given the option to co-locate the data for the share within certain clusters. This option may be useful if the user wishes to have one set of shares distributed within a certain geographical boundary, and a different set of shares distributed across a different geographical boundary, in which case the above sharding strategy remains the same. In step 4 above, only those clusters that were selected while creating the share would be made available to be considered for sharding. This technique provides cluster-aware sharding.

Similarly, quotas can be set on a file server service 202. Quotas may set a limit on the amount of data storage to be used for each share within each cluster. Since file server service stores a per-share, per-cluster storage usage, it can detect when a cluster-level share quota is reached. Depending on the quota policy, the user may be alerted when this cluster-level quota is reached, or the file server service may notify the FSVM (or FSVM) leader within the cluster whose quota has been reached via RPC. On receiving this notification, the FSVM leader may make all file systems for that share across FSVMs read-only to respect the storage quota limit.

Particular embodiments may handle sharding by considering geographic quotas, user-based quotas, fault tolerance of clusters, available resources across clusters, etc. Some benefits may include:

1. Provides uniform auto sharding of compute, network, and storage units across multiple clusters, which also leads to a smaller fault domain
2. File server centrally managed from a single interface, although resources are distributed, leading to easy manageability.
3. Provides flexibility of co-locating shares where necessary and distributing them across clusters when necessary.
4. Provides ability to set cluster aware share level quotas, which could be utilized for location-aware sharding. and
5. Fault tolerant within cluster and capable of tolerating entire cluster failure.

Particular embodiments may identify corrupted or infected data and recover a consistent version of the data from a VFS **202**. When user data is infected by a virus or corrupted by a file system or storage system, identifying the corrupted data and the needed recovery level may be difficult. If the appropriate recovery level is not detected and data is recovered at the wrong place, then a valid version of data may be lost. Particular embodiments may provide capabilities to virtual file services to detect problems from file level to storage level. System administrators need not worry about detecting and recovering a consistent version of data when the system administrator detects the corruption and infected data and manually recovers the data from a file system or from storage system. A self-healing mechanism of the VFS **202** frequently takes snapshots of file system and storage pools and monitors the user data at file system and storage system levels. In particular embodiments, a virtualized file server may accomplish the following levels of detection and recovery:

1. File/folder level recovery: File system or anti-virus or other internal modules can detect the file or folder level infection or corruption. Self-Healing mechanism monitors these events and once it detects, it will recover those particular data from the previous file system snapshot by overwriting the infected/corrupted files/folders.
2. File System level recovery: Self-healing mechanism monitors the checksum of the file system and if it finds any discrepancy on that, it will recover the file system with its latest snapshot.
3. Storage level recovery: Self-Healing mechanism monitors storage-pool corruption and alerts generated by the cluster and detect the data loses and corruption. Once it detects the data corruption/data loss, it will recover the storage-pool for the latest snapshot.

Distributed Self-Healing: Since virtualized file server compute and storage units are distributed across multiple host machines **200**, the self-healing mechanism efficiently monitors the corruptions and data loss in parallel and distributed fashion on all the host machines and detects and recovers that particular data without affecting the overall file server **202**.

Some benefits may include: Detection of and recovery from data loss, data corruption and infected files on file/folder level, file system level and storage level without manual intervention. Efficient detection of and recovery from the data loss, data corruption and infected files in parallel and distributed fashion. Recovery from data loss and data corruption without affecting the overall file server **202**.

Particular embodiments may back up cold data stored in a cluster to an object store, which is either in a public cloud (e.g., AMAZON WEB SERVICES), or to a low-cost storage medium within the same cluster. Particular embodiments may then retrieve the backed-up volume groups as needed to restore files for the file server. Particular embodiments may provide a method to backup data on a virtualized file server running on a hyper-converged infrastructure to a low-cost storage medium hosted on the same physical infrastructure. This consists of a virtualized server running on the same hyper-converged infrastructure providing an object store interface (such as AMAZON S3) with storage as low-cost media such as SMR drives. This particular virtual machine can act as a backup server for other VMs running on the same infrastructure.

Particular embodiments of the backup server may be hosted on the same hyper converged infrastructure as the compute and storage. Particular embodiments of the backup

server may be used for low cost storage media like SMR drives attached to the same hyper converged infrastructure. Particular embodiments of the backup server may provide generic object store interfaces such as AMAZON S3. Particular embodiments of the backup server may provide the same level of availability as the other highly available services (such as FSVM) run on the cluster.

Particular embodiments may include a cloud service as a storage tier of a virtualized file server service. Particular embodiments may then retrieve the backed-up volume groups as needed to restore files for the file server.

Particular embodiments may provide block awareness for a virtualized file server service in order to maintain availability of virtual file server services in case of block failure by deploying FSVMs **170** on different host machines **200**. In case of block failure (e.g., due to power loss affecting a block, a.k.a. hardware host machine), the high-availability features attempt to migrate the VMs running on those host machines to available running host machines. If there are not enough resources on the available running host machines, then the file-server HA features are triggered, and online FSVM(s) take ownership of the volume-group of offline FSVM(s). When one FSVM with metadata service is down, the file-server may continue to serve requests to end users without any performance degradation. Potential benefits may include:

1. Virtualized file-server is available even if one block in the cluster goes down.
2. Users or administrators need not to reserve the resources or free up the resources for block failure to get the virtualized file-server working.
3. In a hyper-converged deployment, the user VMs can be prioritized over FSVMs for migration during block failure.

Particular embodiments may recover from multimode file service failures in a scale-out NAS environment with minimal down time. Traditional file server deployments protected against single host machine failures by having a standby host machine. Detection of service failures is not spontaneous and issues can occur with keeping backup host machines synchronized. Further, if the new active host machine is also down, there may be no way to recover the service. These issues not only cause service interruption but also create complications if there are multiple host machine failures.

In a scale out NAS environment, host machine and service failures may be detected and recovered from without interrupting service to clients. Using distributed cluster health service, these service failures may be detected, and the other active host machines may take over the failed host machine services (both storage and network). Each host machine in cluster acts as potential backup host machine, which will help with managing multiple simultaneous host machine failures based on cluster health. So, even if the new active host machine is down, other host machines in the cluster can take over the new active host machine's load and provide continuous availability. In this way, clients using the scale-out NAS services do not see any downtime or service interruptions when multiple host machines in the cluster are down.

Particular embodiments may help to avoid catastrophic failures due to resource exhaustion. In scenarios such as a user's home directory being accessed for read/write operations, the user may not be able to determine how much disk quota is assigned to the user or how much actual space is available to the user to write data.

As an example, consider a scenario when many users have their home directories on the same share. Existing technologies display the user's drive size as being the same as total share size, thereby giving the user the perception that the total share size is available to write data. However, when the user's quota limit has been met or exceeded, any write to the drive fails.

Particular embodiments may address the administrative challenge of creating and managing a large number of home directories for users. Typically, a home directory is created and maintained for each user account. Conventional file systems place all the home directories on a dedicated home share comprising storage residing on a specific node (e.g., a single host machine storing a volume group `\\home` containing home directories `\\home\user-A`, `\\home\user-B`, . . . , `\\home\user-N`)—this may become a challenge to manage when there are many users, requiring a large number of shares. Certain file systems may distribute the home directories across multiple nodes in the cluster and create separate home shares (e.g., a first host machine storing a volume group `\\home1` containing home directories `\\home1\user-A`, `\\home1\user-B`, . . . , `\\home1\user-F`, and a second host machine storing a volume group `\\home2` containing home directories `\\home2\user-G`, `\\home2\user-H`, . . . , `\\home2\user-N`).

In particular embodiments, certain directories may be designated as home shares, and user home directories may be created as top-level directories in a distributed home share. The home directories may be distributed across a plurality of nodes while providing the ability to serve storage access operations addressed to the single distributed home share (e.g., a request to access a file in a user directory that is sent to a first FSVM running on a first host machine may be sent a share-level DFS referral to a second host machine, where the user directory resides). Particular embodiments may create home directories for users on multiple different host machines. Upon successfully authenticating a user, the user's home directory is exported. When the client sends a request to perform a storage access operation to a first FSVM, if the user's home directory resides in local storage of the host machine on which the first FSVM is running, then the FSVM sends a response to the storage access operation. If the user's home directory resides in remote storage, then the FSVM may force a lookup to determine which host machine is storing the user's home directory in its local storage, construct a reference to the actual location of the user's home directory, then send back a share-level DFS referral to redirect future requests to perform storage access operations in the user's home directory to the correct host machine.

In particular embodiments, if a home directory does not yet exist for a new user, the home directory for the user may be created in local storage for the host machine running the FSVM to which the request was sent. If the home directory for the user needs to be created on a different host machine (e.g., due to load balancing or capacity issues), particular embodiments may create the new home directory on another host machine, then send back a share-level DFS referral to that host machine.

In particular embodiments, home directories may be redistributed on the fly. For example, in a situation where each of three host machines A, B, and C stores home directories for 20, 20, and 15 users respectively, home machine A may be experiencing undue strain on its resources (e.g., with respect to consumption of processor cycles, due to a high frequency of I/O transactions, or with respect to disk usage, due to a large number of files that are

being stored). Once a determination is made that the home directories on host machine A that are triggering high resource consumption need to be redistributed, the data from such home directories may be copied over to a new host machine (e.g., host machine C), and once the data is completely or mostly copied over, FSVMs may start redirecting requests relating to the moved home directories to host machine C by way of sending back share-level DFS referrals.

Particular embodiments may expose user-specific data so that when each user accesses their VDI environment they see their home directory as a mounted drive, and see data specific to their disk portion, such as disk capacity, average rate of utilization of space, frequency of disk accesses, file type, etc. On every soft quota limit reached, the user may be alerted through email that they are about to exhaust their disk quota. Less-frequently-accessed files, folders, and other items may be archived automatically to the cloud.

Particular embodiments may provide high availability of storage services in a scale out file-server. In traditional file server deployments, high-availability is supported by configuring host machines as pairs where the storage resources are inter-connected between two host machines. So, if one of the host machines fails, the other host machine in the pair may take over the storage resources along with the IP address. One limitation with this approach is that an even number of host machines is needed in the cluster. In a scale out file-server, minimal to zero disruption occurs in case of any failure. In a virtualized scale-out file server, all host machines in the scale out cluster monitor the health for every other host machine. If one of the host machines experiences down time because of either a planned shutdown or unplanned host machine failures, one of the host machines starts taking over the storage resources of the down host machine. At the same time, the IP address fails over so that clients can continue to contact the takeover host machine without any disruptions. To increase the load balancing, the failover storage resources may be distributed to multiple host machines, so that the down host machine resources may be distributed across different host machines.

FIG. 7 is a block diagram of an illustrative computing system 700 suitable for implementing particular embodiments. In particular embodiments, one or more computer systems 700 perform one or more steps of one or more methods described or illustrated herein. In particular embodiments, one or more computer systems 700 provide functionality described or illustrated herein. In particular embodiments, software running on one or more computer systems 700 performs one or more steps of one or more methods described or illustrated herein or provides functionality described or illustrated herein. Particular embodiments include one or more portions of one or more computer systems 700. Herein, reference to a computer system may encompass a computing device, and vice versa, where appropriate. Moreover, reference to a computer system may encompass one or more computer systems, where appropriate.

This disclosure contemplates any suitable number of computer systems 700. This disclosure contemplates computer system 700 taking any suitable physical form. As example and not by way of limitation, computer system 700 may be an embedded computer system, a system-on-chip (SOC), a single-board computer system (SBC) (such as, for example, a computer-on-module (COM) or system-on-module (SOM)), a desktop computer system, a mainframe, a mesh of computer systems, a server, a laptop or notebook computer system, a tablet computer system, or a combina-

tion of two or more of these. Where appropriate, computer system 700 may include one or more computer systems 700; be unitary or distributed; span multiple locations; span multiple machines; span multiple data centers; or reside in a cloud, which may include one or more cloud components in one or more networks. Where appropriate, one or more computer systems 700 may perform without substantial spatial or temporal limitation one or more steps of one or more methods described or illustrated herein. As an example and not by way of limitation, one or more computer systems 700 may perform in real time or in batch mode one or more steps of one or more methods described or illustrated herein. One or more computer systems 700 may perform at different times or at different locations one or more steps of one or more methods described or illustrated herein, where appropriate.

Computer system 700 includes a bus 702 (e.g., an address bus and a data bus) or other communication mechanism for communicating information, which interconnects subsystems and devices, such as processor 704, memory 706 (e.g., RAM), static storage 708 (e.g., ROM), dynamic storage 710 (e.g., magnetic or optical), communication interface 714 (e.g., modem, Ethernet card, a network interface controller (NIC) or network adapter for communicating with an Ethernet or other wire-based network, a wireless NIC (WNIC) or wireless adapter for communicating with a wireless network, such as a WI-FI network), input/output (I/O) interface 712 (e.g., keyboard, keypad, mouse, microphone). In particular embodiments, computer system 700 may include one or more of any such components.

In particular embodiments, processor 704 includes hardware for executing instructions, such as those making up a computer program. As an example and not by way of limitation, to execute instructions, processor 704 may retrieve (or fetch) the instructions from an internal register, an internal cache, memory 706, static storage 708, or dynamic storage 710; decode and execute them; and then write one or more results to an internal register, an internal cache, memory 706, static storage 708, or dynamic storage 710. In particular embodiments, processor 704 may include one or more internal caches for data, instructions, or addresses. This disclosure contemplates processor 704 including any suitable number of any suitable internal caches, where appropriate. As an example and not by way of limitation, processor 704 may include one or more instruction caches, one or more data caches, and one or more translation lookaside buffers (TLBs). Instructions in the instruction caches may be copies of instructions in memory 706, static storage 708, or dynamic storage 710, and the instruction caches may speed up retrieval of those instructions by processor 704. Data in the data caches may be copies of data in memory 706, static storage 708, or dynamic storage 710 for instructions executing at processor 704 to operate on; the results of previous instructions executed at processor 704 for access by subsequent instructions executing at processor 704 or for writing to memory 706, static storage 708, or dynamic storage 710; or other suitable data. The data caches may speed up read or write operations by processor 704. The TLBs may speed up virtual-address translation for processor 704. In particular embodiments, processor 704 may include one or more internal registers for data, instructions, or addresses. This disclosure contemplates processor 704 including any suitable number of any suitable internal registers, where appropriate. Where appropriate, processor 704 may include one or more arithmetic logic units (ALUs); be a multi-core processor; or include one or more processors 702. Although this disclosure

describes and illustrates a particular processor, this disclosure contemplates any suitable processor.

In particular embodiments, I/O interface 712 includes hardware, software, or both, providing one or more interfaces for communication between computer system 700 and one or more I/O devices. Computer system 700 may include one or more of these I/O devices, where appropriate. One or more of these I/O devices may enable communication between a person and computer system 700. As an example and not by way of limitation, an I/O device may include a keyboard, keypad, microphone, monitor, mouse, printer, scanner, speaker, still camera, stylus, tablet, touch screen, trackball, video camera, another suitable I/O device or a combination of two or more of these. An I/O device may include one or more sensors. This disclosure contemplates any suitable I/O devices and any suitable I/O interfaces 712 for them. Where appropriate, I/O interface 712 may include one or more device or software drivers enabling processor 704 to drive one or more of these I/O devices. I/O interface 712 may include one or more I/O interfaces 712, where appropriate. Although this disclosure describes and illustrates a particular I/O interface, this disclosure contemplates any suitable I/O interface.

In particular embodiments, communication interface 714 includes hardware, software, or both providing one or more interfaces for communication (such as, for example, packet-based communication) between computer system 700 and one or more other computer systems 700 or one or more networks. As an example and not by way of limitation, communication interface 714 may include a network interface controller (NIC) or network adapter for communicating with an Ethernet or other wire-based network or a wireless NIC (WNIC) or wireless adapter for communicating with a wireless network, such as a WI-FI network. This disclosure contemplates any suitable network and any suitable communication interface 714 for it. As an example and not by way of limitation, computer system 700 may communicate with an ad hoc network, a personal area network (PAN), a local area network (LAN), a wide area network (WAN), a metropolitan area network (MAN), or one or more portions of the Internet or a combination of two or more of these. One or more portions of one or more of these networks may be wired or wireless. As an example, computer system 700 may communicate with a wireless PAN (WPAN) (such as, for example, a BLUETOOTH WPAN), a WI-FI network, a WI-MAX network, a cellular telephone network (such as, for example, a Global System for Mobile Communications (GSM) network), or other suitable wireless network or a combination of two or more of these. Computer system 700 may include any suitable communication interface 714 for any of these networks, where appropriate. Communication interface 714 may include one or more communication interfaces 714, where appropriate. Although this disclosure describes and illustrates a particular communication interface, this disclosure contemplates any suitable communication interface.

One or more memory buses (which may each include an address bus and a data bus) may couple processor 704 to memory 706. Bus 702 may include one or more memory buses, as described below. In particular embodiments, one or more memory management units (MMUs) reside between processor 704 and memory 706 and facilitate accesses to memory 706 requested by processor 704. In particular embodiments, memory 706 includes random access memory (RAM). This RAM may be volatile memory, where appropriate. Where appropriate, this RAM may be dynamic RAM (DRAM) or static RAM (SRAM). Moreover, where appro-

priate, this RAM may be single-ported or multi-ported RAM. This disclosure contemplates any suitable RAM. Memory **706** may include one or more memories **706**, where appropriate. Although this disclosure describes and illustrates particular memory, this disclosure contemplates any suitable memory.

Where appropriate, the ROM may be mask-programmed ROM, programmable ROM (PROM), erasable PROM (EPROM), electrically erasable PROM (EEPROM), electrically alterable ROM (EAROM), or flash memory or a combination of two or more of these. In particular embodiments, dynamic storage **710** may include a hard disk drive (HDD), a floppy disk drive, flash memory, an optical disc, a magneto-optical disc, magnetic tape, or a Universal Serial Bus (USB) drive or a combination of two or more of these. Dynamic storage **710** may include removable or non-removable (or fixed) media, where appropriate. Dynamic storage **710** may be internal or external to computer system **700**, where appropriate. This disclosure contemplates mass dynamic storage **710** taking any suitable physical form. Dynamic storage **710** may include one or more storage control units facilitating communication between processor **704** and dynamic storage **710**, where appropriate.

In particular embodiments, bus **702** includes hardware, software, or both coupling components of computer system **700** to each other. As an example and not by way of limitation, bus **702** may include an Accelerated Graphics Port (AGP) or other graphics bus, an Enhanced Industry Standard Architecture (EISA) bus, a front-side bus (FSB), a HYPERTRANSPORT (HT) interconnect, an Industry Standard Architecture (ISA) bus, an INFINIBAND interconnect, a low-pin-count (LPC) bus, a memory bus, a Micro Channel Architecture (MCA) bus, a Peripheral Component Interconnect (PCI) bus, a PCI-Express (PCIe) bus, a serial advanced technology attachment (SATA) bus, a Video Electronics Standards Association local (VLB) bus, or another suitable bus or a combination of two or more of these. Bus **702** may include one or more buses **706**, where appropriate. Although this disclosure describes and illustrates a particular bus, this disclosure contemplates any suitable bus or interconnect.

According to particular embodiments, computer system **700** performs specific operations by processor **704** executing one or more sequences of one or more instructions contained in memory **706**. Such instructions may be read into memory **706** from another computer readable/usable medium, such as static storage **708** or dynamic storage **710**. In alternative embodiments, hard-wired circuitry may be used in place of or in combination with software instructions. Thus, particular embodiments are not limited to any specific combination of hardware circuitry and/or software. In one embodiment, the term “logic” shall mean any combination of software or hardware that is used to implement all or part of particular embodiments disclosed herein.

The term “computer readable medium” or “computer usable medium” as used herein refers to any medium that participates in providing instructions to processor **704** for execution. Such a medium may take many forms, including but not limited to, nonvolatile media and volatile media. Non-volatile media includes, for example, optical or magnetic disks, such as static storage **708** or dynamic storage **710**. Volatile media includes dynamic memory, such as memory **706**.

Common forms of computer readable media include, for example, floppy disk, flexible disk, hard disk, magnetic tape, any other magnetic medium, CD-ROM, any other optical medium, punch cards, paper tape, any other physical medium with patterns of holes, RAM, PROM, EPROM,

FLASH-EPROM, any other memory chip or cartridge, or any other medium from which a computer can read.

In particular embodiments, execution of the sequences of instructions is performed by a single computer system **700**. According to other particular embodiments, two or more computer systems **700** coupled by communication link **716** (e.g., LAN, PTSN, or wireless network) may perform the sequence of instructions in coordination with one another.

Computer system **700** may transmit and receive messages, data, and instructions, including program, i.e., application code, through communication link **716** and communication interface **714**. Received program code may be executed by processor **704** as it is received, and/or stored in static storage **708** or dynamic storage **710**, or other non-volatile storage for later execution. A database **720** may be used to store data accessible by the system **700** by way of data interface **718**.

FIG. **8** illustrates a block diagram showing a distributed file system **800** according to particular embodiments. The distributed file system **800** may use many of the same components that are illustrated in FIGS. **2A**, **2B**, and **3A-3H**, and they may operate in a substantially similar manner. Therefore, in the interests of brevity and clarity, an explanation of the function and operation of these common components will not be repeated.

In particular embodiments, the VFS **202** may provide access controls for accessing storage items. The access controls may be stored for each of the storage item may be specified in access control lists (ACLs) including permissions information for individual users and groups. For example, an ACL **830** may be associated with the Folder-1 **312**, an ACL **831** may be associated with the File-1 **318**, and an ACL **832** may be associated with the Folder-2 **314**. Additional ACLs may be associated with others of the folders and files described in FIGS. **3A-3H**. While the ACLs **830**, **831**, and **832** are depicted as being stored with the Folder-1 **312**, File-1 **318**, and Folder-2 **314**, respectively, it is appreciated that the ACLs may be stored separately, such as in the metadata database **820**. Each ACL **830**, **831**, and **832** may include a list of access control entries (ACE). Each ACE may identify a user (and/or group) and specify the access rights allowed, denied, or audited for that user. In some examples, each ACL may be on average 4 KB in size, but may grow up to 64 KB in size.

When the ACL for the storage item is large in size (e.g., as may be the case where there are many entries listed for different users/groups and/or many different types of permissions that may be specified), retrieving the permissions information may be an expensive operation. If the permissions information must be accessed with each and every storage access operation, the cost may become considerable. To mitigate this cost, in response to an access request of a storage item (e.g., one of the Folder-1 **312**, File-1 **318**, and Folder-2 **314**) associated with a user, the FileSystem-1A **364a** may perform a one-time extraction of all the permissions information associated with the user with respect to the storage item, and then may cache the extracted permissions information in a cache **810**. For subsequent access request associated with the storage item and the user, the permissions information may be retrieved from the cache **810**. In some examples, the permissions information may be stored in the form of a bitmask, such as a 4-byte bitmask (e.g., where individual bits are associated with a respective permission grant). In some examples, the use of a bitmask to store the permissions information in the cache **810** may simplify operations to evaluate permissions for a user by using standard bitmask operations, which may reduce the

cost of checking permissions for subsequent requests by the same user for access requests associated with the storage item. For example, if a user were to execute a file containing a script (and, upon discovering an apparent bug in the script), retrieve the Last-Modified timestamp for the file, display contents of the file, edit the file, and then execute the script again, such a series of storage access operations may involve at least five steps for which the user's permissions must be checked.

Particular embodiments may handle checking permissions information in two phases. In a first phase, upon receiving a request for a storage item from a particular user, the FileSystem-1A 364a may fetch all of the permissions information associated with the storage item, iterate through the permissions information to identify user and/or group entries relevant to the particular user, and create a cache entry having the permissions profile pertaining to the user and the storage item in the cache 810. (e.g., cache the 4-byte bitmask for the user). In some examples, the permissions information is retrieved from the storage item. In other examples, the permissions information is retrieved from the metadata database 820. In a second phase, as the FileSystem-1A 364a continues to field access requests from the particular user with respect to the requested storage item, the FileSystem-1A 364a may simply check the cache entry for the user and storage item (e.g., by performing a bitmask comparison between the permissions required to perform the access operation and the permissions profile for the user) to determine whether the access is permitted.

In some examples, the FileSystem-1A 364a may maintain the cache entry in the cache 810 on a time period basis (e.g., two seconds or some other specified time), a session basis (e.g., for an entire session associated with the particular user) a global basis (e.g., when permissions are not anticipated to change, or to change very rarely/infrequently), or for another duration specified by an administrator. The specified period of time after which the cache expires may be tuned based on an anticipated frequency of changes to the permissions information. In some embodiments, changes to the access control information in relation to the user (e.g., if the access control information is updated to add or remove write permissions for the storage item) may trigger an event where the cache entry associated with the storage item and the user is invalidated.

By frontloading retrieval of permission information for a user and storage item and storing the information in the form of a bitmask in the cache 810, subsequent access requests from the user may be more efficient in terms of cost and latency (e.g., (1) avoiding and/or streamlining the task of fetching the entire ACL for each access request, and (2) replacing the task of reading the entire ACL each time with the smaller task of simply reading the cached permissions profile for the user).

In particular embodiments, a system (VFS 202) for managing data access using a virtualized file server may comprise (1) a plurality of host machines 201a-c implementing a virtualization environment, wherein each of the host machines (e.g., host machines 201a-c) comprises a hypervisor (e.g., hypervisor 130) and at least one user virtual machine (user VMs 105a-c or client 330); and (2) a virtualized file server comprising a plurality of file server virtual machines (FSVMs 170a-c) and a storage pool (e.g., storage pool 160), wherein each of the FSVMs is running on one of the host machines, wherein the FSVMs conduct I/O transactions with the storage pool. One of the user VMs may send, to one of the FSVMs, a request to perform a storage access operation on a storage item for an identified user,

wherein the I/O request complies with a protocol for a distributed file server. A file system (e.g., the FileSystem-1A 364a) may then retrieve, from a cache (e.g., the cache 810), a user permissions profile for the storage item, wherein the user permissions profile consists of permissions information for the user with respect to the storage item. The file system may then determine whether the storage access operation is permissible based on the user permissions profile. Finally, the file system may send a response to the user VM with respect to the request.

Prior to the file system retrieving the user permissions profile for the storage item from the cache, the file system may determine that no cache entry exists for the permissions profile for the user and the storage item. The file system may retrieve access control information for the storage item (e.g., from the storage item or from the metadata database 820), where the access control information may include multiple (e.g., all) permissions information for all users having any access rights to the storage item. The file system may then create a user permissions profile for the identified user based on all permissions information extracted from the access control information that is relevant to the identified user. Finally, the file system may create a cache entry in a cache for the permissions profile.

In particular embodiments, wherein the user permissions profile is stored as a first bitmask representing all permissions information extracted from the access control information for the storage item and relevant to the identified user, the operation to determine whether the storage access operation is permissible based on the user permissions profile comprises performing one or more bitmask operations to compare the first bitmask to one or more second bitmasks representing the permissions required to perform the storage access operation upon the storage item.

In particular embodiments, the cache entry may include a key-value pair, wherein (1) a key of the key-value pair comprises one or more pieces of identifying information comprising: a session ID associated with the identified user, an identifier associated with the identified user, a share name associated with the storage item, or a name of the storage item; and (2) a value of the key-value pair comprises the user permissions profile for the storage item.

In particular embodiments, the FileSystem-1A 364a may detect access patterns and cache clusters of permission information accordingly. For example, if a significant percentage of users having write access to a storage item are detected as attempting to access the file within the past 10 minutes, the FileSystem-1A 364a may retrieve and cache user permissions profiles for all users having write access to the file. In another example, if the FileSystem-1A 364a detects that a user has listing all files of a particular type within a directory, the FileSystem-1A 364a may retrieve and cache user permissions profiles for the user for all files of that type within the directory.

In particular embodiments, the distributed data access coordinator 370a may store metadata associated with the storage items (e.g., Folder-1 312, File-1 318, Folder-2 314, etc.) in the metadata database 820. Such metadata may include, by way of example and not limitation: file type, permissions, owner, group, file size, file access time, file modification time, file deletion time, number of (soft/hard) links to the file/directory, extended attributes, or an ACL.

When a new storage item (e.g., a folder or file) is created in a directory, it may inherit the ACL of the directory. In such cases, oftentimes, many or most of the storage items in a directory have the same ACL information. For example, the ACL 830 may be the same as the ACL 831. During an access

51

operation where the ACLs of multiple and/or all files in a directory are desired to be read (e.g., a request to list all or multiple files in a directory), a conventional file server may need to scan the ACLs of all or multiple storage items in the directory in order to determine whether or not to list each of the storage items. When many of the ACLs contain duplicate information, such an operation may result in a substantial amount of latency.

In some examples, to reduce latency, the distributed data access coordinator 370a may perform a deduplication operation where only a parent directory ACL is stored in the metadata database 820, and for storage items in the directory that have an ACL that matches the parent directory ACL, the distributed data access coordinator 370a may store a pointer to the parent directory ACL (e.g., rather than a separate ACL for the storage item). For files having a different ACL than the parent directory ACL, the distributed data access coordinator 370a may create a separate ACL. For example, FIG. 9 depicts an operation 900 of a metadata database 820 according to particular embodiments. In the example of FIG. 9, the metadata database 820 includes an entry 910 for the Folder-1 312 and an entry 920 for the File 1 318. In the example of FIG. 9, the ACL 830 and the ACL 831 are the same (e.g., both equal to X). Therefore, the distributed data access coordinator 370a may store the ACL 831 as a pointer to the ACL 830 (as shown in the example 904), rather than storing the ACL 831 as a separate ACL (as shown in the example 902). The distributed data access coordinator 370a may perform the deduplication operation at the time of creation of a storage item in a directory, in some examples. Then, during an access, the distributed data access coordinator 370a may initially retrieve and cache the parent directory ACL, and when storage items ACLs from the directory are subsequently retrieved, the pointer results in a cache hit, and the data is more efficiently accessible and evaluated as compared with a separate retrieval process.

Herein, a computer-readable non-transitory storage medium or media may include one or more semiconductor-based or other integrated circuits (ICs) (such, as for example, field-programmable gate arrays (FPGAs) or application-specific ICs (ASICs)), hard disk drives (HDDs), hybrid hard drives (HHDs), optical discs, optical disc drives (ODDs), magneto-optical discs, magneto-optical drives, floppy diskettes, floppy disk drives (FDDs), magnetic tapes, solid-state drives (SSDs), RAM-drives, SECURE DIGITAL cards or drives, any other suitable computer-readable non-transitory storage media, or any suitable combination of two or more of these, where appropriate. A computer-readable non-transitory storage medium may be volatile, non-volatile, or a combination of volatile and non-volatile, where appropriate.

Herein, “or” is inclusive and not exclusive, unless expressly indicated otherwise or indicated otherwise by context. Therefore, herein, “A or B” means “A, B, or both,” unless expressly indicated otherwise or indicated otherwise by context. Moreover, “and” is both joint and several, unless expressly indicated otherwise or indicated otherwise by context. Therefore, herein, “A and B” means “A and B, jointly or severally,” unless expressly indicated otherwise or indicated otherwise by context.

The scope of this disclosure encompasses all changes, substitutions, variations, alterations, and modifications to the example embodiments described or illustrated herein that a person having ordinary skill in the art would comprehend. The scope of this disclosure is not limited to the example embodiments described or illustrated herein. Moreover, although this disclosure describes and illustrates respective

52

embodiments herein as including particular components, elements, feature, functions, operations, or steps, any of these embodiments may include any combination or permutation of any of the components, elements, features, functions, operations, or steps described or illustrated anywhere herein that a person having ordinary skill in the art would comprehend. Furthermore, reference in the appended claims to an apparatus or system or a component of an apparatus or system being adapted to, arranged to, capable of, configured to, enabled to, operable to, or operative to perform a particular function encompasses that apparatus, system, component, whether or not it or that particular function is activated, turned on, or unlocked, as long as that apparatus, system, or component is so adapted, arranged, capable, configured, enabled, operable, or operative.

What is claimed is:

1. A method comprising:

hosting a file server virtual machine (FSVM) on a host machine of a plurality cluster of host machines, the FSVM configured to manage at least one storage item of a namespace presented by the FSVM, the namespace including a plurality of storage items stored in a storage pool distributed across the plurality of host machines, the plurality of storage items comprising at least one storage item stored at a host machine of the plurality of host machines and another storage item stored at another host machine of the plurality of host machines, the storage pool including local storage coupled to at least the host machine, wherein the plurality of storage items include respective access control lists (ACLs) corresponding to permissions granted to users for the plurality of storage items;

receiving, at the FSVM, an access request associated with a user and directed to the at least one storage item stored in the storage pool;

listing the at least one storage item having one or more permissions granted to the user in a directory of the namespace;

determining whether the access request directed to the at least one storage item is permissible based on the permission listing within the directory; and

if the access request is permissible, performing the access request on the at least one storage item stored in the storage pool.

2. The method of claim 1, wherein the plurality of storage items of the namespace include folders, files, or portions thereof.

3. The method of claim 1, further comprising using a storage map to determine whether a particular storage item is located on a particular host machine.

4. The method of claim 1, further comprising hosting additional FSVMs on additional host machines of the plurality of host machines.

5. The method of claim 4, wherein each of the additional FSVMs is configured to present the namespace.

6. The method of claim 1, wherein at least one of the ACLs comprises an access control entity.

7. The method of claim 6, wherein the access control entity specifies access rights allowed, denied, or audited.

8. The method of claim 7, wherein the access control entity specifies a user associated with the access rights allowed, denied, or audited.

9. The method of claim 1, wherein the access request comprises a request to list storage items.

10. The method of claim 1, further comprising retrieving a user permission profile for multiple files in the directory.

53

11. The method of claim 1, the storage pool comprising an aggregation of storage devices from the plurality of host machines.

12. The method of claim 1, wherein the permission listing is configured to list one or more storage items that the user is permitted to access.

13. The method of claim 1, the storage pool including local storage coupled to at least multiple ones of the plurality of host machines.

14. A system comprising:

a plurality of host machines, including a first host machine;

a storage pool comprising at least a plurality of computer-readable non-transitory storage media distributed across the plurality of host machines, the storage pool including local storage coupled to at least one of the plurality of host machines; and

a file server virtual machine (FSVM) configured to execute on the first host machine, the FSVM configured to manage at least one storage item of a namespace presented by the FSVM, the namespace including a plurality of storage items,

wherein the plurality of storage items comprise at least one storage item stored at the first host machine and another storage item stored at a second host machine of the plurality of host machines,

wherein the plurality of storage items include respective access control lists (ACLs) corresponding to permissions granted to users for the storage items,

wherein the FSVM is configured to receive an access request associated with a user and directed to the at least one storage item stored in the storage pool; and

wherein the system is configured to list the storage item having one or more permissions granted to the user in a directory of the namespace, determine whether the access request directed to the storage item is permissible based on the permission listing within the directory, and if the access request is permissible, perform the access request on the storage item stored in the storage pool.

15. The system of claim 14, wherein the plurality of storage items of the namespace include folders, files, or portions thereof.

16. The system of claim 14, further comprising additional FSVMs on additional host machines of the cluster-plurality of host machines.

17. The system of claim 16, wherein each of the additional FSVMs is configured to present the namespace.

18. The system of claim 14, wherein at least one of the ACLs comprises an access control entity.

19. The system of claim 18, wherein the access control entity specifies access rights allowed, denied, or audited.

20. The system of claim 19, wherein the access control entity specifies a user associated with the access rights allowed, denied, or audited.

54

21. The system of claim 14, the storage pool comprising an aggregation of storage devices from the plurality of host machines.

22. The system of claim 14, wherein the permission listing is configured to list one or more storage items that the user is permitted to access.

23. The system of claim 14, the storage pool including local storage coupled to at least multiple ones of the plurality of host machines.

24. A method comprising:

receiving a request from a user to read a directory, wherein the directory is in a namespace presented by a file server virtual machine (FSVM) hosted on a host machine of a plurality of host machines, and wherein storage items of the directory are stored in a storage pool distributed across the plurality of host machines, and wherein the storage pool includes local storage coupled to at least one host machine of the plurality of host machines, wherein the storage items comprise at least one storage item stored at the host machine and another storage item stored at a second host machine of the plurality of host machines;

accessing permissions information associated with the user, the permissions information pertaining to multiple files in the directory; and

listing the storage items in the directory having permissions granted to the user based on the permissions information.

25. The method of claim 24, wherein the storage items include respective access control lists (ACLs) corresponding to the permissions information.

26. The method of claim 24, wherein the storage items of the namespace include folders, files, or portions thereof.

27. The method of claim 24, further comprising using a storage map to determine whether a particular storage item is located on a particular host machine of the plurality of host machines.

28. The method of claim 24, further comprising hosting additional FSVMs on additional host machines of the plurality of host machines.

29. The method of claim 28, wherein each of the additional FSVMs is configured to present the namespace.

30. The method of claim 24, the storage pool comprising an aggregation of storage devices from the plurality of host machines.

31. The method of claim 24, wherein the permissions information is configured to list one or more files that the user is permitted to access.

32. The method of claim 24, wherein the storage pool includes local storage coupled to at least multiple ones of the plurality of host machines.

* * * * *